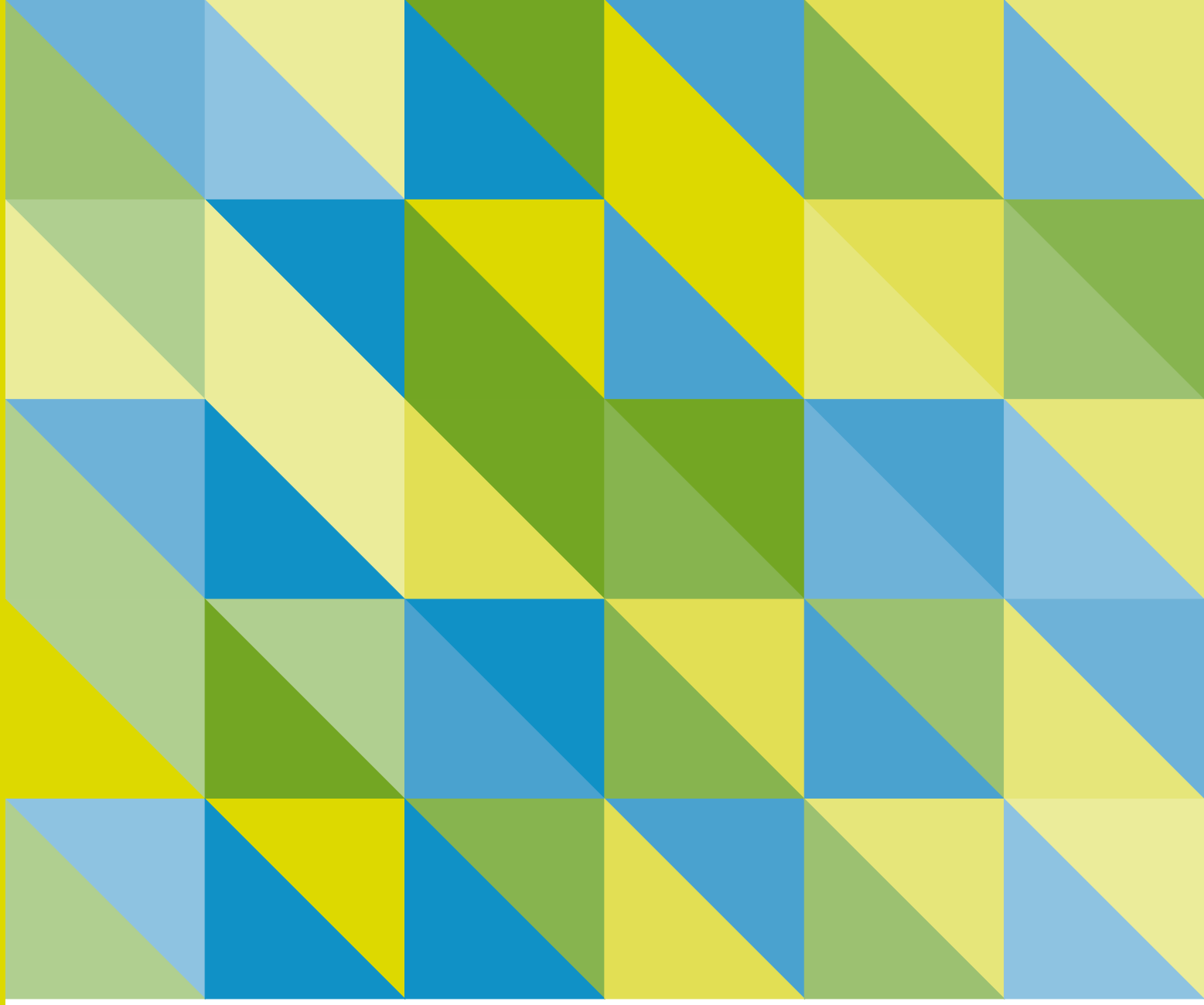


Apache Kafka in der Versicherung

Event Streaming mit Java und Spring Boot

Grundlagen, Spring Boot, Betrieb und Fallstudie – Apache Kafka 4.4

Marcus Konitzer · Konitzer Training · November 2026

konitzer

APACHE KAFKA IN DER VERSICHERUNG

© 2026 **Marcus Konitzer** · Konitzer Training

Dieses Handout wird im Rahmen einer Schulung ausgegeben und darf nur in diesem Kontext verwendet werden.

Diese Unterlage wurde nach bestem Wissen und Gewissen erstellt. Es wird keine Haftung für fehlerhafte Informationen übernommen. Versions- und Produktangaben beziehen sich auf den Stand November 2026.

Bei der Erstellung dieses Handbuchs kam Claude zum Einsatz.

Inhaltsverzeichnis

1 Einführung und Lab-Umgebung 7
1.1 Das Seminar im Überblick . . . 7
1.2 Grundlage: das Buch . . . 7
1.3 Die Lab-Umgebung . . . 7
1.4 Docker: Hintergrund und Installation . . . 10
1.5 Wenn etwas hakt . . . 14

2 Warum Kafka? 15
2.1 Punkt-zu-Punkt-Integration skaliert nicht . . . 15
2.2 Kafka ist ein verteiltes, persistentes Log . . . 15
2.3 Architektur auf einen Blick . . . 15
2.4 Vier Kommunikationsmuster . . . 16
2.5 Weitere Muster mit Kafka . . . 17
2.6 Kafka im Vergleich . . . 17
2.7 Kafka ja oder nein? . . . 18
2.8 Unser Szenario: die Bestandsverwaltung . . . 18
2.9 Zum Weiterdenken . . . 18

3 Erste Schritte mit der Kommandozeile 19
3.1 Das Beispiel: Beitragsänderungen . . . 19
3.2 In den Container wechseln . . . 19
3.3 Ein Topic anlegen . . . 19
3.4 Nachrichten schreiben . . . 20
3.5 Nachrichten lesen . . . 20
3.6 Was wir dabei sehen . . . 20
3.7 Dasselbe in Kafka UI . . . 21
3.8 Nachrichten mit Key . . . 21
3.9 Ein Topic löschen . . . 21
3.10 Neue Optionsnamen in Kafka 4.4 . . . 22
3.11 Kein automatisches Anlegen . . . 22

4 Das verteilte Log 23
4.1 Aufbau einer Nachricht . . . 23
4.2 Das Log . . . 23
4.3 Partitionen: Einheit für Skalierung . . . 26
4.4 Keys bestimmen die Partition . . . 26
4.5 Replikation: Leader und Follower . . . 28
4.6 Komponenten eines Clusters . . . 28
4.7 Lab: Partitionen und Replikation . . . 29
4.8 Woran sehe ich das im Betrieb? . . . 31
4.9 Weiterführende Links . . . 31

5 Zuverlässigkeit und Replikation 32
5.1 Drei Stellschrauben bestimmen die Haltbarkeit . . . 32
5.2 Acknowledgements: `acks` . . . 32
5.3 In-Sync Replicas (ISR) . . . 33

5.4 `min.insync.replicas` genauer erklärt . . . 34
5.5 Faustregel für Produktion . . . 35
5.6 Zustellgarantien . . . 36
5.7 Leader-Wahl und Datenverlust . . . 36
5.8 Lab: Broker-Ausfall . . . 37
5.9 Woran sehe ich das im Betrieb? . . . 38
5.10 Weiterführende Links . . . 38

6 Consumer Groups und Offsets 39
6.1 Consumer Groups . . . 39
6.2 Rebalancing . . . 40
6.3 Offsets . . . 41
6.4 Vorführung: Wie ein Consumer nach dem Neustart weitermacht . . . 42
6.5 Offset-Reset . . . 42
6.6 Daten erneut verarbeiten . . . 44
6.7 Fragen aus dem Seminar: Commit, Lücken, gezieltes Nachlesen . . . 44
6.8 Szenarien: Wer liest was, und warum eine Gruppe? . . . 45
6.9 Lab: Consumer Groups und Offsets . . . 51
6.10 Woran sehe ich das im Betrieb? . . . 51
6.11 Weiterführende Links . . . 52

7 Producer mit Spring Boot 53
7.1 Die minimale Anwendung . . . 53
7.2 Die Konfiguration . . . 54
7.3 Senden mit dem KafkaTemplate . . . 55
7.4 Unter der Haube: Idempotenz mit Producer-ID und Sequenznummer . . . 57
7.5 Topics als Code . . . 57
7.6 Ereignisse als JSON . . . 58
7.7 Header . . . 59
7.8 Producer-Ereignisse zentral protokollieren . . . 60
7.9 Mit welchem Broker spricht der Producer? . . . 60
7.10 Konfiguration gehört nicht ins JAR . . . 61
7.11 Was bei einem Absturz des Producers passiert . . . 61
7.12 Unterschiede zu Spring Boot 3.x . . . 61
7.13 Zusammenfassung . . . 62
7.14 Weiterführende Links . . . 62

8 Consumer mit Spring Boot 63
8.1 Die minimale Anwendung . . . 63
8.2 Der Listener . . . 63
8.3 Die Konfiguration . . . 64
8.4 Wann committet wird . . . 65
8.5 Skalieren: Instanzen und Threads . . . 66
8.6 Rebalances beobachten . . . 66
8.7 Idempotent verarbeiten . . . 67
8.8 Mehrere Gruppen, ein Topic . . . 68
8.9 JSON lesen . . . 68

8.10 Den Listener steuern . . . 69
8.11 Reihenfolge innerhalb einer Partition ändern? . . . 70
8.12 Alternative: Apache Camel . . . 70
8.13 Zusammenfassung . . . 70
8.14 Weiterführende Links . . . 70

9 Fehlerbehandlung und Zustellsicherheit 72
9.1 Zustellsicherheit ist eine Kette . . . 72
9.2 Wenn eine Nachricht nicht verarbeitet werden kann . . . 73
9.3 Anhalten oder Dead-Letter-Topic? . . . 74
9.4 Das Outbox-Muster . . . 77
9.5 Kafka-Transaktionen . . . 79
9.6 Fehler in der Deserialisierung . . . 82
9.7 Checkliste Zustellsicherheit . . . 82
9.8 Weiterführende Links . . . 82

10 Datenformate und Schemas 83
10.1 Ereignisse entwerfen . . . 83
10.2 Formate im Vergleich . . . 84
10.3 JSON mit Spring Kafka . . . 84
10.4 Schemas und Schema Registry . . . 85
10.5 Namenskonventionen für Topics . . . 86
10.6 Kompaktierte Topics . . . 86
10.7 Zusammenfassung . . . 87
10.8 Weiterführende Links . . . 87

11 Kafka Streams mit Spring Boot 88
11.1 Grundbegriffe . . . 88
11.2 Einrichten in Spring Boot . . . 88
11.3 Zwei Ereignisse zusammenführen: Antrag und Prüfung . . . 89
11.4 Weitere Join-Arten . . . 91
11.5 Zählen und Summieren . . . 92
11.6 Unter der Haube: Tasks, Threads und Zustand . . . 93
11.7 Betrieb einer Streams-Anwendung . . . 93
11.8 Kafka Streams oder selbst korrelieren? . . . 94
11.9 Zusammenfassung . . . 95
11.10 Weiterführende Links . . . 95

12 Kafka Connect 96
12.1 Konzept . . . 96
12.2 Typische Connectoren im Versicherungsumfeld . . . 97
12.3 Change Data Capture mit Debezium . . . 97
12.4 Ins Reporting schreiben: JDBC Sink . . . 98
12.5 Connect über die REST-API bedienen . . . 99
12.6 Woran sehe ich das im Betrieb? . . . 99
12.7 Zusammenfassung . . . 99
12.8 Weiterführende Links . . . 99

13 Kafka-Anwendungen testen 100
13.1 Die Testpyramide für Kafka-Anwendungen . . . 100
13.2 Abhängigkeiten . . . 100
13.3 Fachlogik ohne Kafka . . . 101
13.4 Integrationstest mit eingebettetem Broker . . . 102
13.5 Integrationstest mit Testcontainers . . . 103
13.6 Fehlerfälle testen . . . 104
13.7 Kafka Streams ohne Broker testen . . . 105
13.8 Verträge zwischen Teams prüfen . . . 106
13.9 Zusammenfassung . . . 106
13.10 Weiterführende Links . . . 106
14 Monitoring und Alerting 107
14.1 Die Metrik-Landkarte . . . 107
14.2 Woher kommen die Metriken? . . . 107
14.3 Metriknamen lesen . . . 108
14.4 Zustand: diese Werte müssen 0 bzw. 1 sein . . . 109
14.5 Auslastung und Requests . . . 109
14.6 Replikation, KRaft und Speicher . . . 110
14.7 Client-Metriken . . . 110
14.8 Spring Boot: Metriken mit Micrometer . . . 111
14.9 Lag von außen messen . . . 112
14.10 Verarbeitung nachverfolgen . . . 112
14.11 Monitoring mit Dynatrace und Splunk . . . 112
14.12 Vom Messwert zum Alarm . . . 117
14.13 Vom Alarm zur Lösung: das Runbook . . . 118
14.14 Lab: Dashboard lesen und Broker-Ausfall . . . 119
14.15 Lab: Lag-Alarm auslösen . . . 119
14.16 Zusammenfassung . . . 120
15 Performance und Tuning 121
15.1 Partitionen bestimmen die Parallelität . . . 121
15.2 Producer: Batching . . . 121
15.3 Producer: Kompression und acks . . . 124
15.4 Broker . . . 124
15.5 Consumer . . . 125
15.6 Messen mit Bordmitteln . . . 125
15.7 Lab: Perf-Test . . . 126
15.8 Quotas . . . 127
15.9 Woran sehe ich das im Betrieb? . . . 128
16 Sicherheit 129
16.1 Die vier Schichten . . . 129
16.2 Listener und Protokolle . . . 129
16.3 Authentifizierung . . . 130
16.4 Autorisierung mit ACLs . . . 130
16.5 Felder verschlüsseln . . . 131

16.6 Quotas als Schutz . . . 133
16.7 Checkliste . . . 134
16.8 Weiterführende Links . . . 134

17 Betrieb 135
17.1 Apache Kafka, Confluent, KRaft und ZooKeeper . . . 135
17.2 Zielbild für den Betrieb . . . 135
17.3 Hardware und Betriebssystem . . . 136
17.4 Kafka auf OpenShift und Kubernetes . . . 136
17.5 Topics und ACLs als Code . . . 137
17.6 Rolling Restart . . . 137
17.7 Cluster erweitern und Partitionen umziehen . . . 137
17.8 Upgrades . . . 138
17.9 Speicher: Kafka darf nie die Platte ausgehen . . . 138
17.10 Best Practices . . . 138
17.11 Die Plattform beim Kunden: Confluent, Terraform und OpenShift . . . 139
17.12 Vom Testprojekt zur produktiven Anwendung . . . 142
17.13 Weiterführende Links . . . 142

18 Fallstudie: Bestandsverwaltung 143
18.1 Warum die Queue hier an Grenzen stößt . . . 143
18.2 Kafka als Puffer . . . 143
18.3 Den Puffer dimensionieren . . . 144
18.4 Reihenfolge pro Vertrag . . . 144
18.5 Parallel verarbeiten, ohne die Reihenfolge zu brechen . . . 144
18.6 Lab: Skaliert die Bestandsverwaltung? . . . 145
18.7 Wenn der Producer abstürzt . . . 146
18.8 Wenn der Consumer abstürzt . . . 146
18.9 Die Poison Pill . . . 146
18.10 Die Spring-Boot-Anwendung bestand-demo . . . 146
18.11 Fallstricke für die Reihenfolge . . . 147
18.12 Den Puffer überwachen . . . 147
18.13 Checkliste: Puffer mit Reihenfolge . . . 148

19 Referenzarchitektur und Desaster Recovery 149
19.1 Was kann schiefgehen – und woran sehe ich es? . . . 149
19.2 Ausfallsicher über Standorte . . . 149
19.3 MirrorMaker 2 . . . 149
19.4 Lab: Spiegeln und Failover . . . 150
19.5 Desaster Recovery überwachen . . . 151
19.6 Backups in Kafka . . . 151
19.7 Neu seit dem Buch . . . 152
19.8 Ihr Zielbild . . . 152
19.9 Weiterführende Links . . . 152

20 Wenn-Dann: Situationen und ihre Folgen 153
20.1 Grundlagen . . . 153
20.2 Zuverlässigkeit . . . 154

20.3 Consumer Groups und Offsets . . . 155
20.4 Betrieb . . . 158
20.5 Typische Fehler beim Aufsetzen der Plattform . . . 161
20.6 Zielwerte für erhöhten Ausfallschutz . . . 163
20.7 Mehr Durchsatz zur Laufzeit . . . 167
20.8 Weiterführende Links . . . 168

Anhang: Befehlsreferenz **169**

Docker und Lab . . . 169
Topics . . . 169
Konfiguration . . . 170
Schreiben und lesen . . . 170
Consumer Groups und Offsets . . . 171
Cluster und KRaft . . . 171
Messen . . . 172
Spring-Boot-Einstellungen . . . 172

Anhang: Glossar **173**

1 Einführung und Lab-Umgebung

Dieses Buch begleitet das Seminar „Apache Kafka – Professionelles Event Streaming“ und geht an vielen Stellen darüber hinaus. Alle Beispiele stammen aus dem Versicherungswesen: Verträge, Anträge, Risikoprüfungen, Schadenmeldungen und Beitragszahlungen. Rund ein Drittel des Buchs zeigt die Umsetzung mit Java und Spring Boot.

Dieses Kapitel beschreibt den Rahmen des Seminars und die Lab-Umgebung: einen Kafka-Cluster mit drei Knoten und Monitoring, der komplett in Docker läuft. Es erklärt außerdem, wie Sie die Umgebung auf dem eigenen Rechner einrichten.

1.1 Das Seminar im Überblick

Tag 1 – Kafka verstehen, Cluster aufbauen, Zuverlässigkeit

- Einstieg: Vorstellung, Erwartungen, Technik-Check
- Warum Kafka? Event Streaming, Kafka ja oder nein, unser Szenario
- Erste Schritte: Topics, Nachrichten, Kommandozeile
- Das verteilte Log: Partitionen, Keys, Replikation, KRaft
- Zuverlässigkeit: acks, ISR, Zustellgarantien, Broker-Ausfall
- Consumer Groups und Offsets: Rebalancing, Lag, Offset-Reset

Tag 2 – Monitoring, Performance, Architektur

- Monitoring I: den Cluster lesen – Broker-, KRaft- und Speicher-Metriken
- Performance: Tuning live im Dashboard, Perf- und Ende-zu-Ende-Tests
- Monitoring II: Client-Metriken, Lag-Alert, Runbook
- Fallstudie Bestandsverwaltung: Puffer, Skalierung, Reihenfolge, Poison Pill
- Referenzarchitektur und Desaster Management mit MirrorMaker 2

1.2 Grundlage: das Buch

Anatoly Zelenin, Alexander Kropp: *Apache Kafka – Von den Grundlagen bis zum Produktiveinsatz*, 2. Auflage, Hanser. Die Kapitel dieses Handbuchs verweisen auf Kapitel und Seiten des Buchs zum Nachlesen. Ergänzend gilt die offizielle Kafka-Dokumentation zur Version 4.4: https://kafka.apache.org/documentation/

1.3 Die Lab-Umgebung

Im Seminar arbeiten alle auf einem eigenen Remote-Rechner. Darauf läuft per Docker:

Komponente	Zweck	Adresse im Browser
`kafka1`, `kafka2`, `kafka3`	Apache Kafka 4.4.0, je Knoten Broker und KRaft-Controller kombiniert	–
`kafka-ui`	Kafbat UI: Topics, Nachrichten, Consumer Groups	http://localhost:8080
`prometheus`	sammelt Metriken, wertet Alert-Regeln aus	http://localhost:9090
`alertmanager`	verschickt Alarme	http://localhost:9093
`grafana`	Dashboard „Kafka Broker“	http://localhost:3000
`jmx-init`	lädt einmalig den JMX-Exporter herunter, beendet sich dann („Exited (0)“ ist richtig)	–

Für Tag 2 kommt mit dem Profil `dr` ein zweiter Standort dazu: der Cluster `kafka-dr` und `mirror-maker` (MirrorMaker 2).

Die Werkzeuge sind bewusst auf Kafka-Bordmittel und das Monitoring aus dem Buch beschränkt (Anhang B: JMX-Exporter, Prometheus, Alertmanager, Grafana) sowie Kafbat UI (Buch Kap. 2.5 und 14.1.2).

Lab ist nicht Produktion

Im Lab laufen Broker und Controller im selben Prozess. Die Kafka-Dokumentation empfiehlt das nur für Entwicklung und Tests; in Produktion laufen drei oder fünf eigene Controller-Knoten. Deshalb gilt im Lab: **Nie zwei der drei Knoten gleichzeitig stoppen** – sonst fehlt dem KRaft-Quorum die Mehrheit.

1.3.1 Der Lab-Ordner

```
kafka-lab/
├── docker-compose.yml     Definition aller Container
├── README.md              Kurzanleitung
├── bin/                   Lab-Hilfsskripte (im Container unter /opt/lab/bin)
│   ├── mit-metriken       startet ein Kafka-Werkzeug mit JMX-Exporter
│   └── bestandsverwaltung simuliert eine Instanz der Bestandsverwaltung
├── jmx/                   Konfiguration des JMX-Exporters
├── prometheus/            prometheus.yml, rules.yml (Alert-Regeln),
alertmanager.yml
├── grafana/               Dashboard und Provisionierung
└── mm2/                   Konfiguration MirrorMaker 2 (Tag 2)
```

1.3.2 Starten und prüfen

Alle Befehle in diesem Abschnitt laufen in **PowerShell** auf Ihrem Rechner.

```
cd $HOME\Desktop\kafka-lab
```

```
docker compose up -d
```

```
docker compose ps
```

Nach etwa 30 Sekunden sind sieben Container „Up“; `jmx-init` steht auf „Exited (0)“.

1.3.3 Kafka-Befehle ausführen

Die Kafka-Werkzeuge laufen **im Container** des Brokers `kafka1`:

```
docker exec -it kafka1 bash
```

```
cd /opt/kafka/bin
```

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --list
```

Mit `exit` verlassen Sie den Container wieder. In diesem Handbuch ist bei jedem Befehl angegeben, ob er in PowerShell oder im Container läuft. Laufende Console-Producer beenden Sie mit Strg+D, Consumer mit Strg+C.

1.3.4 Neue Optionsnamen in Kafka 4.4

Die alten Namen funktionieren noch, erzeugen aber Warnungen. Befehle aus dem Buch oder aus dem Internet laufen also weiter.

Alt	Neu in 4.4
`--producer-property`, `--consumer-property`, `--producer-props`	`--command-property`
`--property` beim Console-Producer	`--reader-property`
`--property` beim Console-Consumer	`--formatter-property`
`--messages` beim Consumer-Perf-Test	`--num-records`

1.3.5 Topics werden nicht automatisch angelegt

Im Lab ist `auto.create.topics.enable=false` gesetzt, wie im Buch für Produktion empfohlen (Kap. 14.1.3: Topics als Code, nie nebenbei). Ein Tippfehler im Topic-Namen führt deshalb sofort zu `UNKNOWN_TOPIC_OR_PARTITION`, statt still ein neues Topic mit Standardwerten anzulegen.

1.3.6 Zwei Adressen für denselben Cluster

Wer verbindet sich?	Adresse
Werkzeuge im Container, andere Container	`kafka1:9092`, `kafka2:9092`, `kafka3:9092`
Programme auf dem Windows-Rechner (IntelliJ, Spring Boot)	`localhost:19092,localhost:19093,localhost:19094`

Der Grund: Ein Client nutzt die Bootstrap-Adresse nur für den Einstieg. Danach verbindet er sich direkt mit dem Leader jeder Partition – und zwar unter der Adresse, die der Broker selbst bekannt gibt (`advertised.listeners`). Jeder Broker hat deshalb zwei Listener: einen für das Docker-Netz,

einen für den Host. Ein klassischer Betriebsfehler ist ein falsch gesetzter Advertised Listener: Der Bootstrap klappt, danach gibt es nur Timeouts.

1.3.7 Ports

Port	Dienst
8080	Kafbat UI
3000	Grafana
9090	Prometheus
9093	Alertmanager
19092, 19093, 19094	Kafka-Broker für Clients auf dem Host
9101–9103	JMX-Exporter der Broker
29092	DR-Cluster (Profil `dr`)

1.3.8 Aufräumen

```
docker compose --profile dr down
```

Das stoppt und entfernt alle Container – Topics und Nachrichten sind danach weg. Zwischen Tag 1 und Tag 2 daher **nicht** `down` ausführen. Mit `down -v` wird zusätzlich das Volume mit dem heruntergeladenen JMX-Agenten entfernt.

Nach Performance-Tests Speicher freigeben:

```
docker exec kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:9092
--delete --topic perf
```

1.4 Docker: Hintergrund und Installation

Damit Sie die Lab-Umgebung nach dem Seminar auf dem eigenen Rechner nachbauen können, hier das Nötigste zu Docker.

1.4.1 Warum Docker für das Lab

- **Gleiche Versionen überall:** Die `docker-compose.yml` legt jedes Image mit fester Version fest (z. B. `apache/kafka:4.4.0`). Wer das Lab startet, bekommt exakt dieselbe Umgebung.
- **Ein Befehl für alles:** `docker compose up -d` startet sieben Container mit Netzwerk, Konfiguration und Monitoring.
- **Spurlos entfernbar:** `docker compose down` räumt alles weg, auf dem Rechner bleibt nichts installiert außer Docker selbst.

1.4.2 Begriffe in einem Satz

Begriff	Bedeutung
Image	Vorlage mit Betriebssystem-Dateien und Programm, z. B. `apache/kafka:4.4.0`
Container	laufende Instanz eines Images, z. B. `kafka1`
Volume	Speicherplatz, der einen Container überleben kann
Netzwerk	virtuelles Netz, in dem Container sich über ihren Namen finden (`kafka1`, `kafka2` …)
Docker Compose	beschreibt mehrere Container in einer YAML-Datei und startet sie gemeinsam
Registry	Ablage für Images, standardmäßig Docker Hub

1.4.3 Verwendete Images

Image	Version
`apache/kafka`	4.4.0
`kafbat/kafka-ui`	v1.2.0
`prom/prometheus`	v3.5.0
`prom/alertmanager`	v0.28.1
`grafana/grafana`	11.6.0
`curlimages/curl`	8.11.1 (nur zum Herunterladen des JMX-Exporters)

Alle Images zusammen sind etwa 2–3 GB groß. Beim ersten Start werden sie heruntergeladen; das kann je nach Netz einige Minuten dauern. Vorab laden lässt es sich mit `docker compose pull`.

1.4.4 Voraussetzungen

	Minimum	Empfohlen
Arbeitsspeicher des Rechners	16 GB	24 GB oder mehr
Arbeitsspeicher für Docker	6 GB	8 GB oder mehr
CPU-Kerne	4	8
Freier Plattenplatz	20 GB	40 GB (Performance-Tests schreiben pro Lauf rund 1,8 GB)
Virtualisierung	im BIOS/UEFI aktiviert (Intel VT-x bzw. AMD-V)	

Die drei Broker sind im Lab auf je 768 MB Java-Heap begrenzt. Mit DR-Standort, Monitoring und Kafbat UI kommt der Stack auf einige Gigabyte.

1.4.5 Installation unter Windows

1. **WSL 2 einrichten** (Windows-Subsystem für Linux). In einer PowerShell als Administrator:

```
wsl --install
```

Danach neu starten.

2. **Docker Desktop installieren:** Installer von https://docs.docker.com/desktop/setup/install/windows-install/ herunterladen und ausführen. Bei der Frage nach dem Backend „WSL 2“ wählen.
3. **Docker Desktop starten**, die Lizenzbedingungen lesen und annehmen (siehe unten) und warten, bis unten links „Engine running“ steht.
4. **Prüfen:**

```
docker version
```

```
docker compose version
```

5. **Speicher prüfen:** Mit dem WSL-2-Backend bekommt Docker standardmäßig einen Anteil des Arbeitsspeichers (in der Regel die Hälfte). Anpassen lässt sich das in der Datei `%UserProfile%\.wslconfig`:

```
[wsl2]
memory=8GB
processors=4
```

Danach `wsl --shutdown` und Docker Desktop neu starten.

6. **Lab entpacken und starten:** `kafka-lab.zip` auf den Desktop entpacken, dann wie oben unter „Starten und prüfen“.
7. **Beispielcode (optional, ab Kapitel 7):** Für die Java-Beispiele genügt ein JDK 17 oder neuer; Maven bringt der Wrapper `mvnw` mit. Alle Beispiele liegen in einem Maven-Projekt `labs`, ein Modul je Thema: von den Shell-Skripten des ersten Tages (`01-shell`) über Producer und Consumer ohne Framework (`02`, `03`) und Spring Boot minimal (`04` bis `06`) bis zu JSON, Fehlerbehandlung, Streams, Tests, Monitoring, Performance und der Fallstudie (`07` bis `19`). Jedes Modul hat eine `ANLEITUNG.md` mit Ablauf, Aufgaben und dem Weg zum Ändern und Bauen; `./mvnw verify` führt alle 33 Tests ohne Lab aus. Die Konsolen-Demos laufen gegen den Lab-Cluster unter `localhost:19092,19093,19094`.

1.4.6 Installation unter macOS und Linux

- **macOS:** Docker Desktop für Mac (Apple Silicon oder Intel) von https://docs.docker.com/desktop/setup/install/mac-install/ installieren. Speicher unter Settings → Resources einstellen. Im Lab ersetzen Sie `$HOME\Desktop\kafka-lab` durch `~/Desktop/kafka-lab`; die `docker`-Befehle sind identisch.
- **Linux:** Docker Engine und das Compose-Plugin aus dem offiziellen Docker-Repository der Distribution installieren (https://docs.docker.com/engine/install/). Den eigenen Benutzer zur Gruppe `docker` hinzufügen, damit `docker` ohne `sudo` läuft. Ein Arbeitsspeicher-Limit wie bei Docker Desktop gibt es nicht.

1.4.7 Lizenz: Docker Desktop im Unternehmen

Docker Desktop ist für private Nutzung, Ausbildung und kleine Unternehmen kostenlos. Für größere Unternehmen (laut Docker: ab 250 Beschäftigten oder 10 Mio. US-Dollar Jahresumsatz) ist ein kostenpflichtiges Abonnement nötig. Klären Sie vor der Installation mit Ihrer IT, welche Lösung im Haus freigegeben ist. Kostenfreie Alternativen, mit denen das Lab ebenfalls läuft:

- **Docker Engine direkt in WSL 2:** In einer Linux-Distribution unter WSL (z. B. Ubuntu) Docker Engine wie unter Linux installieren. Kein grafisches Programm, aber dieselben Befehle.
- **Rancher Desktop** oder **Podman Desktop:** grafische Oberflächen mit Docker-kompatibler Kommandozeile; `docker compose` bzw. `podman compose` verarbeiten die `docker-compose.yml`.

1.4.8 Firmennetz und Proxy

- Images kommen von Docker Hub (`registry-1.docker.io`). In vielen Firmennetzen ist das gesperrt oder nur über einen Proxy oder einen internen Spiegel (Registry-Mirror) erreichbar.
- Proxy in Docker Desktop: Settings → Resources → Proxies.
- Der Container `jmx-init` lädt den JMX-Exporter von Maven Central (`repo1.maven.org`). Auch das muss erreichbar sein.
- Gibt es nur einen internen Spiegel, die Image-Namen in der `docker-compose.yml` entsprechend anpassen.

1.4.9 Wichtige Docker-Befehle für das Lab

Befehl (PowerShell, im Ordner `kafka-lab`)	Wirkung
`docker compose up -d`	alle Container starten (im Hintergrund)
`docker compose ps`	Status aller Container
`docker compose logs --tail 30 kafka1`	letzte Log-Zeilen eines Containers
`docker stop kafka3` / `docker start kafka3`	einen Container stoppen bzw. starten
`docker compose restart kafka-ui`	einen Container neu starten
`docker compose pull`	alle Images vorab herunterladen
`docker compose --profile dr down`	alles stoppen und entfernen (Daten weg)
`docker system df`	Plattenverbrauch von Images, Containern, Volumes

1.4.10 Rechner prüfen

Für die Seminar-Rechner gibt es das Prüfskript `Test-Teilnehmerrechner.ps1`. Es prüft nur lesend Arbeitsspeicher, Docker, Java, Ports und Netz, startet dann das Lab und schreibt und liest 20.000 Testnachrichten. Den Bericht legt es auf dem Desktop ab.

```
powershell -ExecutionPolicy Bypass -File "$HOME\Desktop\Test-
Teilnehmerrechner.ps1"
```

Mit dem Schalter `-Stoppen` fährt es den Lab-Stack anschließend wieder herunter.

1.5 Wenn etwas hakt

Symptom	Lösung
`docker` meldet „failed to connect to the docker API“	Docker Desktop starten und warten, bis „Engine running“ steht
Ein Container ist nicht „Up“	`docker compose logs <name>` ansehen, dann `docker compose up -d`
Kafbat UI lädt nicht	`docker compose ps kafka-ui`, ggf. `docker compose up -d kafka-ui` oder `docker compose restart kafka-ui`
Grafana zeigt „No data“	1–2 Minuten warten; in Prometheus unter Status → Targets prüfen. Die Client-Ziele (9201–9213) sind „down“, solange kein Client läuft – das ist normal
`UNKNOWN_TOPIC_OR_PARTITION`	Topic fehlt oder ist falsch geschrieben – mit `kafka-topics.sh --list` prüfen, dann anlegen
Port belegt	`docker ps` – läuft noch ein anderer Kafka oder ein anderes Programm auf dem Port?
Images lassen sich nicht laden	Proxy bzw. Registry-Freigabe mit der IT klären (siehe oben)

2 Warum Kafka?

Bevor wir Kafka bedienen, klären wir, welches Problem es löst – und wann es nicht die richtige Wahl ist.

2.1 Punkt-zu-Punkt-Integration skaliert nicht

Integration ist oft gewachsen: Jedes System spricht direkt mit jedem anderen. Bei n Quellen und m Abnehmern entstehen bis zu $n \times m$ Schnittstellen. Jede Änderung an einer Quelle kann einen Abnehmer brechen, jeder neue Abnehmer braucht eine neue Schnittstelle.

Die Alternative ist ein zentraler Event-Hub: **Ein System schreibt ein Ereignis einmal, beliebig viele lesen es – ohne dass der Schreiber sie kennen muss.** Neue Abnehmer kommen hinzu, ohne die Quelle anzufassen.

Buch: Kap. 1.1–1.2, S. 3–7

2.2 Kafka ist ein verteiltes, persistentes Log

Kafka ist keine Warteschlange, sondern ein Log. Neue Ereignisse werden hinten angehängt, jedes bekommt eine fortlaufende Nummer, den *Offset*.

```
Offset:   0       1       2       3       4   ← neues Ereignis wird angehängt
        [Event] [Event] [Event] [Event] [Event]
```

Daraus folgen drei Eigenschaften:

- **Aufbewahren:** Ereignisse bleiben erhalten, bis die Retention sie löscht – typischerweise Tage (Standard: 7 Tage).
- **Unabhängig lesen:** Jeder Leser hat seine eigene Leseposition und sein eigenes Tempo.
- **Erneut verarbeiten (Replay):** Lesen löscht nichts. Solange die Ereignisse noch da sind, kann man sie erneut lesen.

2.3 Architektur auf einen Blick

- **Producer** schreiben Ereignisse in **Topics**.
- Ein Topic ist in **Partitionen** geteilt; die Partitionen liegen verteilt auf mehreren **Brokern**. So skaliert Kafka.
- **Consumer** lesen in **Consumer Groups**. Jede Gruppe liest alle Ereignisse, unabhängig von den anderen Gruppen – zum Beispiel Gruppe A die Bestandsverwaltung, Gruppe B ein Reporting.
- Das **KRaft-Controller-Quorum** verwaltet die Metadaten des Clusters und wählt Leader. Früher übernahm das ZooKeeper.

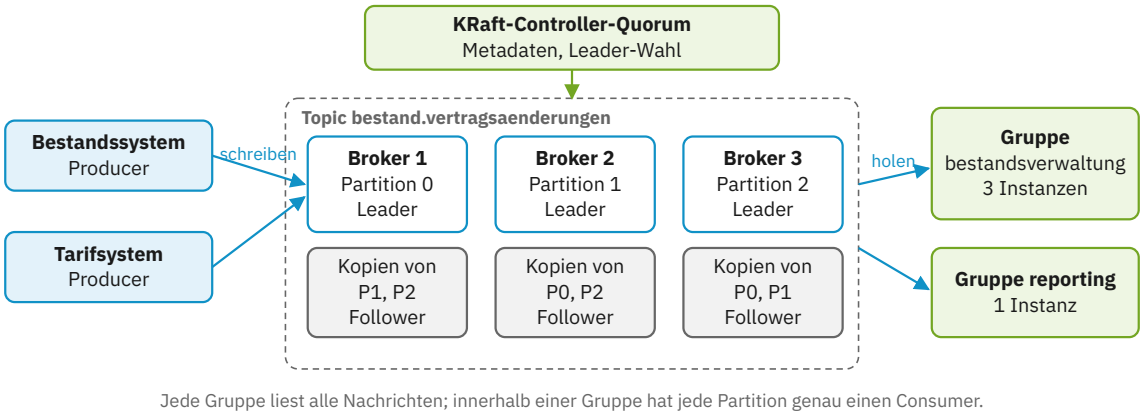


Abbildung 1: Producer schreiben in Partitionen auf mehreren Brokern; jede Consumer Group liest alles

Buch: Kap. 1.3, S. 8–10

2.4 Vier Kommunikationsmuster

Muster	Ablauf	Eigenschaft
Synchron (REST)	Aufrufer → Service → Antwort	Der Aufrufer wartet. Beide Seiten sind in Zeit und Verfügbarkeit eng gekoppelt.
Asynchron (Kafka)	Producer → Log → Consumer	Nach bestätigtem Schreiben ist der Producer fertig. Die Verarbeitung folgt später.
Publish/ Subscribe	ein Ereignis → viele Abnehmer	Mehrere Consumer Groups nutzen dieselben Ereignisse für unterschiedliche Aufgaben.
Active Polling	Consumer holt Daten ab	Kafka schiebt nicht. Consumer holen aktiv und steuern ihre Last selbst.

Wichtig für den Betrieb: Kafka schiebt nicht, Consumer holen. Ein überlasteter Consumer wird nicht überrollt, er holt einfach langsamer – die Daten warten im Log. Umgekehrt wartet Kafka aber auch nicht auf ihn: Ist ein Consumer langsamer, als die Retention erlaubt, werden ungelesene Daten gelöscht.

2.5 Weitere Muster mit Kafka

Muster	Umsetzung	Beispiel
Publish/ Subscribe	mehrere Consumer Groups lesen dasselbe Topic	Vertragsänderungen für Bestand, Inkasso, Reporting
Lastverteilung (Queue)	eine Consumer Group mit mehreren Instanzen	Bestandsverwaltung mit drei Instanzen
Request/Reply	Anfrage-Topic plus Antwort-Topic, Korrelations-ID im Header	Tarifierung asynchron anfragen
Retry- und Dead-Letter-Topics	fehlerhafte Nachrichten in eigene Topics	Dokumentaufträge, die scheitern
Kompaktierte Topics	je Key nur der letzte Stand	Kundenstammdaten

Request/Reply über Kafka ist möglich (Spring bietet dafür `ReplyingKafkaTemplate`), aber kein Ersatz für REST, wenn der Aufrufer sofort eine Antwort braucht: Die Antwort kommt asynchron, und der Aufrufer muss mit Zeitüberschreitung umgehen.

2.5.1 Von REST zu Ereignissen

Wer eine bestehende REST-Schnittstelle ablösen will – etwa die Meldung von Transaktionen an die Geldwäsche-Prüfung –, geht schrittweise vor:

1. Ereignis fachlich beschreiben: Was ist passiert? Welche Daten brauchen die Abnehmer? (Kapitel 10)
2. Producer ergänzt Kafka zusätzlich zum REST-Aufruf, idealerweise über eine Outbox (Kapitel 9).
3. Abnehmer stellen nacheinander auf Kafka um; der REST-Aufruf läuft parallel weiter.
4. Wenn alle umgestellt sind, entfällt der REST-Aufruf.

Was sich ändert: Der Sender erfährt nicht mehr, ob der Empfänger erfolgreich war. Fehler beim Empfänger werden dort behandelt (Kapitel 9), Rückmeldungen laufen bei Bedarf als eigene Ereignisse zurück.

2.6 Kafka im Vergleich

Ansatz	Typischer Schwerpunkt	Unterschied zu Kafka
Klassisches Messaging (MQ)	Nachrichten zustellen und nach Bestätigung löschen; der Broker führt Buch, wer was bekommen hat.	Ereignisse bleiben erhalten; jede Consumer Group führt ihre eigene Leseposition.
REST	Synchrones Request-Response, direkt zwischen zwei Systemen.	Zeitlich entkoppelte Verarbeitung über ein persistentes Log.
Relationale Datenbank	Aktuellen Zustand speichern und flexibel abfragen.	Historie der Änderungen als Ereignisstrom verteilen und erneut lesen.

Die Datenbank hält den aktuellen Zustand, Kafka die Geschichte der Änderungen – die beiden ergänzen sich. Ereignisse transportieren Änderungen; eine Datenbank hält den daraus aufgebauten, abfragbaren Zustand.

Buch: Kap. 17, S. 333–346

2.6.1 Kann Kafka inzwischen auch Queues?

Ja, eingeschränkt: Seit Kafka 4.2 gibt es *Queues for Kafka* (Share Groups, KIP-932), seit 4.4 auch mit Dead-Letter-Queue. Mehrere Consumer teilen sich dann eine Partition und bestätigen Nachrichten einzeln – allerdings **ohne Reihenfolge-Garantie**. Mehr dazu in der Fallstudie an Tag 2.

2.7 Kafka ja oder nein?

Gut geeignet:

- Viele Konsumenten derselben Ereignisse, Replay, Entkopplung eines Monolithen
- Hoher Durchsatz und lückenlose Historie, z. B. für Geldwäscheprävention
- Ereignisgetriebene Integration zwischen Teams

Eher nicht:

- Synchrones Request-Response mit kurzer Antwortzeit
- Einzelne Nachrichten gezielt löschen oder priorisieren
- Komplexe Ad-hoc-Abfragen

Kafka allein ist noch keine Revisionssicherheit

Audit-Anforderungen brauchen zusätzlich passende Retention, Zugriffsschutz und Governance.

2.8 Unser Szenario: die Bestandsverwaltung

Ein Szenario begleitet das ganze Seminar:

```
Producer (Vertragsänderungen) → Queue (heute) → Bestandsverwaltung (mehrere Consumer)
```

- Zu Spitzenzeiten liefert der Producer schneller, als die Consumer verarbeiten.
- Kafka soll puffern.
- Auch nach einem Absturz muss die Reihenfolge pro Vertrag stimmen.

Die Bausteine dafür sammeln wir in beiden Tagen: Partitionen und Keys (Kapitel 3), Consumer Groups und Offsets (Kapitel 4), Lag und Monitoring (Tag 2). Gelöst wird die Fallstudie am Nachmittag von Tag 2.

2.9 Zum Weiterdenken

Wo würden Sie in Ihrem Haus Kafka einsetzen – und wo bewusst nicht? Welche Systeme, welche Datenströme?

3 Erste Schritte mit der Kommandozeile

Bevor wir verstehen, was im Cluster passiert, schreiben und lesen wir die ersten Nachrichten. Alle Werkzeuge in diesem Kapitel gehören zu Kafka selbst und liegen im Lab im Container `kafka1` unter `/opt/kafka/bin`.

Die Beispiele zu diesem Kapitel

Die Befehle dieses Kapitels liegen als kommentierte Skripte in `labs/01-shell` (`.ps1` und `.sh`); dort heißt das Topic kurz `beitraege` statt `tarif.beitragsaenderungen`, sonst ist alles gleich. Dasselbe als Java-Programm ohne Framework zeigen `02-producer-plain` und `03-consumer-plain`: Properties, `KafkaProducer`, `KafkaConsumer`, die `poll`-Schleife. Mit Spring Boot: `04-producer-spring` und `05-consumer-spring`.

3.1 Das Beispiel: Beitragsänderungen

Ein Tarifsystem meldet jede Beitragsänderung eines Vertrags als Ereignis: Vertrag V-1001 kostet ab dem 1. Januar 48,90 Euro statt 45,20 Euro. Mehrere Systeme interessieren sich dafür – das Inkasso, das Kundenportal, das Reporting. Statt jedem System eine eigene Schnittstelle anzubieten, schreibt das Tarifsystem die Änderungen in ein Topic.

3.2 In den Container wechseln

Öffnen Sie zwei PowerShell-Fenster und führen Sie in beiden aus:

```
docker exec -it kafka1 bash
```

```
cd /opt/kafka/bin
```

Der Prompt lautet jetzt etwa `appuser@kafka1:/opt/kafka/bin$`. Mit `exit` verlassen Sie den Container wieder.

3.3 Ein Topic anlegen

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic
tarif.beitragsaenderungen --partitions 1 --replication-factor 1
```

Eine Partition und keine Replikation – bewusst minimal für den Einstieg. Warum man in Produktion anders vorgeht, zeigen Kapitel 4 und 5. Beim Anlegen eines Topics mit Punkt im Namen warnt Kafka, dass Punkte und Unterstriche in Metriknamen kollidieren können. Die Warnung ist harmlos, solange man nicht beides mischt.

Das Topic ansehen:

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic
tarif.beitragsaenderungen
```

```
Topic: tarif.beitragsaenderungen  PartitionCount: 1  ReplicationFactor: 1
Configs: min.insync.replicas=1
    Topic: tarif.beitragsaenderungen  Partition: 0  Leader: 3  Replicas: 3  Isr: 3
Elr:   LastKnownElr:
```

Die einzige Kopie liegt hier auf Broker 3; welcher Broker es wird, entscheidet Kafka beim Anlegen. `min.insync.replicas=1` ist der Standardwert, den Kafka 4.x clusterweit hinterlegt.

3.4 Nachrichten schreiben

Im rechten Fenster den Console-Producer starten:

```
./kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic
tarif.beitragsaenderungen
```

Jede Zeile wird eine Nachricht:

```
>V-1001 Beitrag ab 01.01. 48,90 EUR
>V-1002 Beitrag ab 01.01. 112,00 EUR
>V-1001 Beitrag ab 01.04. 49,50 EUR
```

Strg+D beendet den Producer.

3.5 Nachrichten lesen

Im linken Fenster einmal ohne und einmal mit `--from-beginning`:

```
./kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic
tarif.beitragsaenderungen
```

```
./kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic
tarif.beitragsaenderungen --from-beginning
```

Ohne `--from-beginning` zeigt der Consumer nur, was ab jetzt geschrieben wird. Mit der Option liest er alles, was im Topic liegt – auch Nachrichten, die vor seinem Start geschrieben wurden. Strg+C beendet den Consumer.

3.6 Was wir dabei sehen

- **Unabhängig lesen:** Zwei Consumer ohne gemeinsame Gruppe bekommen beide alle Nachrichten. Das Inkasso und das Kundenportal lesen dieselben Beitragsänderungen, ohne sich abzustimmen.
- **Lesen löscht nicht:** Ein Consumer kann jederzeit von vorn lesen. Kafka löscht erst, wenn die Retention greift.
- **Startposition:** Ein neuer Consumer braucht eine Startposition – den ersten Kontakt mit Offsets.
- **Parallel schreiben:** Mehrere Producer dürfen in dasselbe Topic schreiben.

Warum fehlen Nachrichten nach einem Neustart?

Ohne `--group` startet der Console-Consumer bei jedem Aufruf mit einer neuen Zufallsgruppe. Diese Gruppe hat keinen gespeicherten Offset, also beginnt er am Ende des Logs. Was während seiner Pause geschrieben wurde, sieht er nicht. Wie ein Consumer genau dort weitermacht, wo er aufgehört hat, zeigt Kapitel 6.

3.7 Dasselbe in Kafka UI

Kafbat UI unter http://localhost:8080 zeigt Topics, Partitionen und Nachrichten: Topics → `tarif.beitragsaenderungen` → Reiter *Messages*. Jede Nachricht erscheint mit Partition, Offset und Zeitstempel. Schreiben Sie im Producer eine weitere Zeile und aktualisieren Sie die Ansicht – der Offset zählt weiter.

Das UI ist bequem zum Nachschauen. Für Automatisierung und im Notfall direkt auf dem Server bleibt die Kommandozeile das Werkzeug.

3.8 Nachrichten mit Key

Bisher hatte jede Nachricht nur einen Wert. Die Vertragsnummer gehört aber als **Key** an die Nachricht – warum, erklärt Kapitel 4. Der Console-Producer trennt Key und Wert, wenn man ihn darum bittet:

```
./kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic
tarif.beitragsaenderungen --reader-property parse.key=true --reader-property
key.separator=:
```

```
>V-1001:Beitrag ab 01.07. 50,10 EUR
>V-1003:Beitrag ab 01.07. 23,40 EUR
```

Der Consumer zeigt den Key mit `print.key`:

```
./kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic
tarif.beitragsaenderungen --from-beginning --formatter-property print.key=true --
formatter-property print.offset=true
```

Eine Zeile ohne Trennzeichen lehnt der Producer bei `parse.key=true` ab.

3.9 Ein Topic löschen

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --delete --topic
tarif.beitragsaenderungen
```

Das Löschen läuft asynchron. Wer das Topic sofort neu anlegt, bekommt eventuell die Meldung „marked for deletion“ und wartet ein paar Sekunden. Der Name bei `--delete` darf ein regulärer Ausdruck sein – vorher mit `--list` prüfen, was alles darauf passt.

3.10 Neue Optionsnamen in Kafka 4.4

Befehle aus Büchern und dem Internet nutzen oft noch die alten Optionen. Sie funktionieren weiterhin, erzeugen aber Warnungen.

Alt	Neu in 4.4
`--producer-property`, `--consumer-property`, `--producer-props`	`--command-property`
`--property` beim Console-Producer	`--reader-property`
`--property` beim Console-Consumer	`--formatter-property`
`--messages` beim Consumer-Perf-Test	`--num-records`

3.11 Kein automatisches Anlegen

Im Lab ist `auto.create.topics.enable=false` gesetzt. Ein Tippfehler im Topic-Namen – `tarif.beitragsaenderung` statt `tarif.beitragsaenderungen` – führt deshalb sofort zu `UNKNOWN_TOPIC_OR_PARTITION`, statt still ein neues Topic mit Standardwerten anzulegen. In Produktion ist das die empfohlene Einstellung: Topics entstehen bewusst, über eine Pipeline oder als Code in der Anwendung (Kapitel 7 und 17).

Buch Zelenin/Kropp: Kap. 2, S. 13–19

4 Das verteilte Log

Topics, Partitionen, Keys, Replikation und KRaft. Dieses Kapitel erklärt, was im Cluster passiert, wenn eine Vertragsänderung geschrieben wird – und welche Folgen das für Anwendungsdesign und Betrieb hat.

Die Beispiele zu diesem Kapitel

Dass der Key die Partition bestimmt, zeigt `02-producer-plain` am Ergebnis von `send()`: gleicher Key, gleiche Partition, Offset zählt hoch. `04-producer-spring` legt das Topic mit Partitionen und Replikation als Code an (`TopicBuilder`).

4.1 Aufbau einer Nachricht

Feld	Inhalt	Beispiel
Key	optional; bestimmt die Partition	`V-1001` (Vertragsnummer)
Value	die eigentlichen Daten	`{"version":3,"art":"ADRESSE","plz":"50667"}`
Header	Metadaten	`traceparent`, `quelle=bestandssystem`
Timestamp	Zeitstempel	Zeitpunkt der Änderung

Buch Zelenin/Kropp: Kap. 3.2, S. 32–38

4.1.1 Hintergrund und Konsequenzen

- **Für den Broker sind Key und Value nur Bytes.** Er prüft weder Format noch Schema. Das Format ist ein Vertrag zwischen dem Team des Bestandssystems und den Teams, die lesen. Bricht ein Producer das Format, scheitert erst der Consumer – eine „Poison Pill“ (Kapitel 9). Absicherung: Schemas und eine Schema Registry (Kapitel 10).
- **Größe:** Die Standardgrenze liegt bei etwa 1 MB pro Batch (`message.max.bytes` am Broker, `max.message.bytes` am Topic, `max.request.size` am Producer). Ein gescanntes Schadengutachten gehört nicht in Kafka, sondern in ein Dokumentenarchiv; die Nachricht trägt nur die Referenz (Claim-Check-Muster).
- **Timestamp:** Standard ist `CreateTime`, also die Uhr des Producers; alternativ `LogAppendTime` (Uhr des Brokers) über `message.timestamp.type`. Zeitbasierte Retention, Offset-Reset mit `--to-datetime` und Zeitfenster in Kafka Streams richten sich danach.
- **Header** beeinflussen die Partition nicht. Sie eignen sich für Trace-IDs, die Quelle oder einen Schema-Hinweis.
- **Key = null** ist erlaubt, schließt aber Log Compaction („letzter Stand je Vertrag“) und die Reihenfolge pro Vertrag aus.

4.2 Das Log

Das Log ist *append-only* und unveränderlich: Neue Nachrichten werden hinten angehängt, geschriebene nie geändert.

```
Partition 0:  [0] [1] [2] [3] [4] [5] [6] [7]  → neue Nachrichten hinten anhängen
```

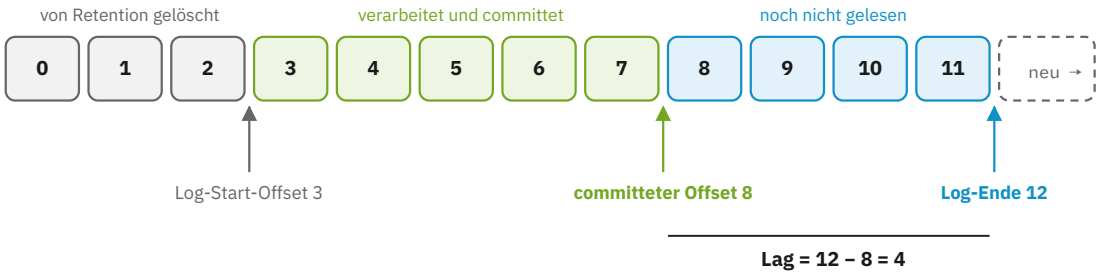


Abbildung 2: Eine Partition mit Log-Start, committetem Offset, Log-Ende und Lag

Ein Offset ist die Position innerhalb einer Partition, nicht im ganzen Topic. Auf der Platte ist jede Partition ein Verzeichnis mit Segment-Dateien. Kafka schreibt nur ans Ende – sequenzielles Schreiben ist sehr schnell, auch auf normalen Platten.

Buch Zelenin/Kropp: Kap. 4.1, S. 41–45

4.2.1 Hintergrund und Konsequenzen

- **Offset nur pro Partition:** Offset 5 gibt es in jeder Partition. Eine Position ist immer das Tripel Topic + Partition + Offset. Daraus folgt:
  1. Offsets verschiedener Partitionen sind nicht vergleichbar; es gibt keine globale Reihenfolge über alle Verträge.
  2. Der Fortschritt einer Consumer Group ist eine Liste – ein Offset je Partition.
  3. Lag wird je Partition gerechnet und für die Gruppe summiert. Eine einzelne hängende Partition verschwindet in der Summe leicht.
  4. Ein Reset auf einen bestimmten Offset braucht die Partition: `--topic bestand.vertragsaenderungen:2`.
- **Offsets werden nie wiederverwendet** und steigen nur. Löscht die Retention alte Segmente, wandert der Anfang des Logs nach vorn. Liegt der gespeicherte Offset einer Gruppe davor, greift `auto.offset.reset` – bei `latest` werden Daten still übersprungen. Betriebsregel: Der Lag, in Zeit ausgedrückt, muss deutlich unter der Retention bleiben.
- **Lücken sind normal:** Transaktions-Marker und Compaction belegen bzw. entfernen Offsets. Die Offset-Differenz ist daher eine Näherung für die Zahl der Nachrichten.
- **Segmente:** Standard 1 GiB oder 7 Tage pro Segment (`log.segment.bytes`, `segment.ms`). Die Retention löscht nur ganze, geschlossene Segmente – das aktive Segment nie. Daten leben also oft länger als eingestellt. Jedes Segment hat `.log`, `.index` und `.timeindex`; viele Partitionen bedeuten viele offene Dateien.
- **Warum schnell:** Kafka schreibt sequenziell in den Page Cache des Betriebssystems, ohne fsync pro Nachricht. Die Haltbarkeit kommt aus der Replikation. Für das Sizing heißt das: JVM-Heap klein halten (einige GB), den Rest des Arbeitsspeichers dem Page Cache lassen.
- **Unveränderlich:** Einzelne Nachrichten lassen sich nicht ändern oder löschen. Für Löschpflichten nach DSGVO gibt es zwei Wege: Compaction mit Tombstone (Key + `null`) oder Verschlüsselung personenbezogener Felder mit einem Schlüssel je Kunde, der bei Löschung vernichtet wird (Crypto-Shredding).

4.2.2 Unter der Haube: Wie ein Broker eine Nachricht findet

Ein Consumer fragt: „Gib mir Partition 1 ab Offset 1.234.567." Der Broker findet die Stelle in drei Schritten, ohne die Datei zu durchsuchen:

1. **Segment finden:** Jede Segment-Datei heißt nach ihrem ersten Offset (`00000000000001200000.log`). Eine binäre Suche über die Dateinamen liefert das Segment, in dem der Offset liegt.
2. **Position im Segment finden:** Die Datei `.index` ist ein dünn besetzter Index von Offset auf Byte-Position – etwa alle 4 KB ein Eintrag (`log.index.interval.bytes`). Eine binäre Suche darin liefert den nächstkleineren Eintrag.
3. **Kurz vorwärts lesen:** Ab dieser Position liest der Broker Batch für Batch bis zum gesuchten Offset.

Die Datei `.timeindex` bildet Zeitstempel auf Offsets ab – damit funktionieren `--to-datetime` und `offsetsForTimes()`. Beim Ausliefern kopiert der Broker die Bytes nicht durch die JVM, sondern lässt das Betriebssystem sie direkt aus dem Page Cache ins Netz schicken (*Zero-Copy*, `sendfile`). Mit TLS geht das nicht, weil die Daten verschlüsselt werden müssen – ein Grund, warum TLS Broker-CPU kostet.

Die Nachrichten liegen auf der Platte in genau dem Format, in dem der Producer sie geschickt hat: als komprimierte Batches. Der Broker packt sie in der Regel nicht aus. Deshalb lohnt sich Kompression im Producer doppelt – für Netz und Platte.

4.2.3 Partitionen auf der Platte ansehen

```
ls /tmp/kraft-combined-logs | grep bestand
```

```
ls -l /tmp/kraft-combined-logs/bestand.vertragsaenderungen-0
```

```
./kafka-dump-log.sh --files /tmp/kraft-combined-logs/
bestand.vertragsaenderungen-0/00000000000000000000.log --print-data-log
```

Der Dateiname eines Segments ist sein erster Offset. `kafka-dump-log.sh` zeigt jeden Batch mit Offset, Zeitstempel, Key und Inhalt. `/tmp/kraft-combined-logs` ist der Pfad im Lab; in Produktion steht er in `log.dirs`.

Jeder Broker hat nur die Partitionen, von denen er eine Kopie hält. Bei einem Topic mit RF 1 hat nur ein Broker ein Verzeichnis dafür – welcher, zeigt die Spalte `Replicas` von `--describe`.

4.2.4 Offsets werden nie zurückgesetzt

Der Offset zählt je Partition für immer nach oben. Er ist ein `long`, also Platz für rund 9 Trillionen Nachrichten; bei einer Million pro Sekunde reicht das etwa 290.000 Jahre. Eine Nummer wird nie wiederverwendet, auch nicht nach dem Löschen alter Daten.

Was beim Löschen passiert: Die Retention entfernt ganze Segmente, dadurch wandert der **kleinste** vorhandene Offset nach oben (`log-start-offset`), der Zähler läuft weiter. Ein Topic mit zehn Millionen geschriebenen und nur noch tausend vorhandenen Nachrichten hat Offsets von 9.999.000 bis 9.999.999, nicht von 0 bis 999. Beide Grenzen zeigt `kafka-get-offsets.sh`: `--time -2` den ältesten, `--time -1` den nächsten freien Offset.

Wieder bei 0 anfangen geht nur, indem das Topic gelöscht und neu angelegt wird. Ein Reset der Consumer-Offsets (Kapitel 6) ändert nur das Lesezeichen der Gruppe, nicht die Nummern der Nachrichten.

Zwei Folgerungen für die Anwendung

Offsets nie als fortlaufende Geschäftsnummer verwenden: Es gibt Lücken, jede Partition zählt für sich, und nach einem Topic-Neuanlegen beginnt alles bei 0. Und beim Vergleich zweier Consumer immer Partition **und** Offset nennen, denn Offset 4 gibt es in jeder Partition einmal.

4.3 Partitionen: Einheit für Skalierung

Ein Topic besteht aus einer oder mehreren Partitionen, verteilt auf die Broker:

```
Broker 1:  Partition 0   V-1002, V-1005, …
Broker 2:  Partition 1   V-1001, V-1004, …
Broker 3:  Partition 2   V-1003, V-1006, …
```

So können viele parallel schreiben und lesen. Der Preis: **Reihenfolge gibt es nur innerhalb einer Partition, nicht über das ganze Topic.** Das ist kein Mangel, sondern das Prinzip, das Kafka skalierbar macht – und für die Bestandsverwaltung genügt es, wenn die Änderungen eines Vertrags in Reihenfolge bleiben.

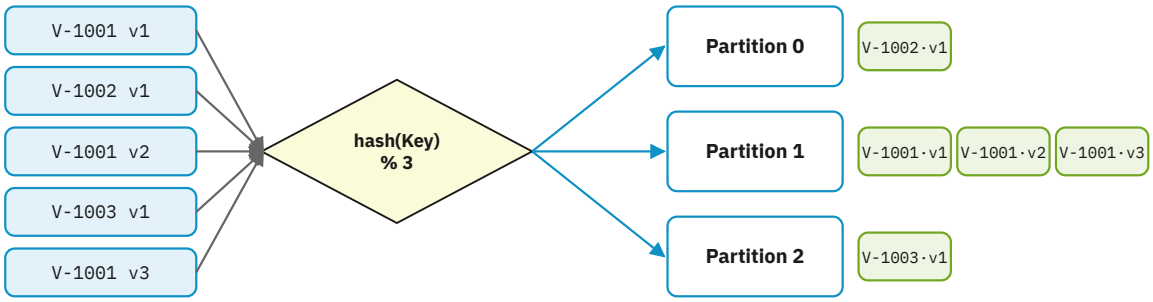
Buch Zelenin/Kropp: Kap. 4.2, S. 46–52

4.3.1 Hintergrund und Konsequenzen

- **Partitionen begrenzen die Parallelität:** In einer Consumer Group liest jede Partition genau ein Consumer. Mit 6 Partitionen arbeiten höchstens 6 Instanzen der Bestandsverwaltung.
- **Partitionen lassen sich nur erhöhen, nie verringern.** Das Erhöhen ändert die Zuordnung Key → Partition. Für eine Übergangszeit liegen alte Änderungen an V-1001 in Partition 1, neue in Partition 4 – die Reihenfolge pro Vertrag kann brechen. Die Partitionszahl daher vorab planen (Kapitel 15).
- **Zu viele Partitionen kosten auch:** mehr Dateien, mehr Metadaten, mehr Leader-Wechsel bei einem Broker-Ausfall, mehr Speicher für Producer-Batches.
- **Neue Broker bekommen nichts automatisch.** Die Verteilung wird beim Anlegen festgelegt. Nach einer Cluster-Erweiterung müssen Partitionen mit `kafka-reassign-partitions.sh` umgezogen werden (Kapitel 17).
- **Hot Partition:** Ungleich verteilte Keys belasten eine Partition – und damit einen Broker und einen Consumer – überproportional.

4.4 Keys bestimmen die Partition

```
Partition = hash(Key) % Anzahl Partitionen
```



Alle Änderungen an V-1001 liegen in Partition 1 – in der Reihenfolge, in der sie geschrieben wurden.

Abbildung 3: Der Hash der Vertragsnummer bestimmt die Partition

Gleicher Key → gleiche Partition → Reihenfolge pro Key. Mit der Vertragsnummer als Key kommen alle Änderungen eines Vertrags in der richtigen Reihenfolge an: erst die Adressänderung, dann die Beitragsanpassung, dann die Kündigung. Ohne Key verteilt der Producer die Nachrichten selbst über die Partitionen.

Der Key wählt **nicht das Topic**, sondern die Partition innerhalb eines Topics. Ein Topic je Art von Daten (Vertragsänderungen, Schadenmeldungen, Zahlungen), ein Key je Fachobjekt (Vertrag, Schaden, Kunde).

Topic	Key	Warum
`bestand.vertragsaenderungen`	Vertragsnummer	Änderungen eines Vertrags in Reihenfolge
`schaden.meldungen`	Schadennummer	Statuswechsel eines Schadens in Reihenfolge
`inkasso.zahlungen`	Vertragsnummer	Zahlungen passend zu den Vertragsänderungen partitioniert
`kunden.stammdaten`	Kundennummer	letzter Stand je Kunde (kompaktiertes Topic)
`dokumente.erzeugen`	kein Key	unabhängige Aufträge, beste Verteilung

4.4.1 Hintergrund und Konsequenzen

- **Der Java-Client** rechnet murmur2 über die serialisierten Key-Bytes. Derselbe Key einmal als String, einmal als Zahl serialisiert ergibt verschiedene Partitionen.
- **Andere Clients rechnen anders:** librdkafka (Basis für C, Python, Go, .NET) nutzt standardmäßig CRC32. Schreiben ein Java- und ein Python-Producer mit derselben Vertragsnummer ins selbe Topic, landen die Nachrichten in verschiedenen Partitionen. Abhilfe in librdkafka: `partitioner=murmur2_random`.
- **Ohne Key:** Der Sticky Partitioner füllt einen Batch für eine Partition und wechselt dann – große Batches, wenige Requests. Seit Kafka 3.3 weicht er zusätzlich langsamen Brokern aus.
- **Reihenfolge braucht auch den Producer:** Der idempotente Producer (Standard seit Kafka 3.0) hält die Reihenfolge auch bei Wiederholungen ein.
- **Wer bestimmt die Partition?** Immer der Producer: eine ausdrücklich angegebene Partition, sonst der Hash des Keys, sonst der Sticky Partitioner. Eigene Logik ist über das Interface `Partitioner` möglich – etwa, um Großkunden auf eigene Partitionen zu legen. Der Consumer bestimmt nur, welche Partitionen er liest: automatisch über die Consumer Group oder manuell mit `assign()`.

- **Kafka filtert nicht nach Keys:** Ein Consumer liest immer ganze Partitionen in Offset-Reihenfolge. „Erst alle Änderungen an V-1001“ geht nicht; Teilmengen trennt man beim Schreiben in eigene Topics.
- **Gute Keys:** viele verschiedene Werte, gleichmäßig verteilt, fachlich die Einheit, deren Reihenfolge zählt. Vertragsnummer: ja. Sparte oder Bundesland als Key: Hot Partition – 70 % aller Änderungen in der KFZ-Sparte landen dann in einer Partition.

4.5 Replikation: Leader und Follower

Jede Partition hat einen **Leader** und **Follower** auf anderen Brokern. Producer und Consumer sprechen mit dem Leader, die Follower kopieren vom Leader.

Partition	Broker 1	Broker 2	Broker 3
0	**Leader**	Follower	Follower
1	Follower	**Leader**	Follower
2	Follower	Follower	**Leader**

Replication Factor 3 ist in Produktion der Standard. Die Leader sind auf die Broker verteilt, damit die Last gleichmäßig ist.

Buch Zelenin/Kropp: Kap. 4.2.3, S. 56–59

4.5.1 Hintergrund und Konsequenzen

- **Follower holen sich die Daten selbst** – per Fetch wie ein Consumer. Wer zu lange hinterherhängt (`replica.lag.time.max.ms`, Standard 30 s), fällt aus der ISR (Kapitel 5).
- **High Watermark:** Consumer sehen nur Nachrichten, die alle Repliken in der ISR haben. Ein langsamer Follower verzögert also auch das Lesen.
- **Replikation ist kein Backup.** Ein versehentlich gelöschtes Topic oder fehlerhafte Daten werden genauso repliziert. Gegen Fehlbedienung hilft nur eine Kopie außerhalb des Clusters (Kapitel 19).
- **Kosten:** RF 3 bedeutet dreifachen Plattenplatz; jede Nachricht geht zweimal zusätzlich über das Netz.
- **Rack Awareness:** Mit `broker.rack` verteilt Kafka die Kopien auf Racks oder Rechenzentren. Consumer können mit `client.rack` vom nächstgelegenen Follower lesen.
- **Preferred Leader:** die erste Replik in der Liste. Nach einem Broker-Neustart wandern die Leader nicht sofort zurück; `auto.leader.rebalance.enable` (Standard an) korrigiert nach einigen Minuten, sofort geht es mit `kafka-leader-election.sh`.

4.6 Komponenten eines Clusters

Komponente	Aufgabe
Broker	speichern Partitionen und bedienen Clients
Controller	verwalten Metadaten und wählen Leader; mehrere Controller bilden mit KRaft ein Quorum
Clients	Producer, Consumer, Admin-Clients, Kafka Connect, Kafka Streams

Ab Kafka 4.0 gibt es nur noch KRaft, ZooKeeper ist abgelöst. Im Lab laufen Broker und Controller kombiniert; in Produktion empfiehlt die Kafka-Dokumentation eigene Controller-Knoten (3 oder 5).

Buch Zelenin/Kropp: Kap. 4.3, S. 60–63; Kap. 7, S. 116–121

4.6.1 Hintergrund und Konsequenzen

- **Quorum = Mehrheit:** 3 Controller vertragen 1 Ausfall, 5 vertragen 2. Eine gerade Anzahl bringt nichts.
- **Metadaten als Log:** Alle Metadaten stehen im internen Topic `__cluster_metadata`; die Broker lesen es mit. Gegenüber ZooKeeper: schnelleres Failover, deutlich mehr Partitionen pro Cluster möglich.
- **Ohne Controller-Mehrheit** bedienen bestehende Leader zunächst weiter – aber es gibt keine Leader-Wahl, keine neuen Topics, keine Konfigurationsänderungen. Fällt dann ein Broker aus, bleiben seine Partitionen offline.
- **Clients sprechen nie mit Controllern,** immer nur mit Brokern (Port 9092). Die Bootstrap-Adresse ist nur der Einstieg; danach spricht der Client direkt mit den Leadern. Deshalb in jedem Client **mehrere Bootstrap-Adressen** angeben – Producer wie Consumer.
- **Migration von ZooKeeper:** Kafka 4.x kann ZooKeeper nicht mehr. Der Weg führt über Kafka 3.9 als Brückenversion.

Was wird verteilt?	Wer entscheidet?
Nachrichten auf Partitionen	der Producer (über den Key)
Partitionen auf Consumer einer Gruppe	der Group Coordinator, ein Broker
Partitionen und Leader auf Broker	der Controller

4.6.2 Unter der Haube: das Metadaten-Log

Im KRaft-Modus sind alle Metadaten – Topics, Partitionen, Leader, ISR, Konfigurationen, ACLs – Einträge in einem eigenen Log, dem internen Topic `__cluster_metadata` mit genau einer Partition.

- Die **Controller** replizieren dieses Log untereinander mit dem Raft-Verfahren. Ein Eintrag gilt als festgeschrieben, wenn die Mehrheit ihn hat. Einer der Controller ist der **aktive Controller** (Raft-Leader) und schreibt alle Änderungen.
- Die **Broker** lesen das Log wie ein Consumer mit und halten den aktuellen Stand im Speicher. Ändert sich der Leader einer Partition, steht das als neuer Eintrag im Log; jeder Broker sieht es beim nächsten Fetch.
- **Heartbeats:** Jeder Broker meldet sich regelmäßig beim aktiven Controller. Bleiben die Heartbeats länger als `broker.session.timeout.ms` aus, wird der Broker ausgesperrt (*fenced*) und seine Partitionen bekommen neue Leader.
- **Snapshots:** Damit das Metadaten-Log nicht endlos wächst, schreiben Controller und Broker regelmäßig einen Snapshot des Standes.

Gegenüber ZooKeeper bedeutet das: Ein neuer Controller muss beim Wechsel nicht erst alle Metadaten laden – er hat sie schon. Failover dauert Sekunden statt Minuten, auch bei Hunderttausenden Partitionen.

4.7 Lab: Partitionen und Replikation

Das zentrale Topic der Bestandsverwaltung mit 6 Partitionen und Replication Factor 3:

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic
bestand.vertragsaenderungen --partitions 6 --replication-factor 3
```

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic
bestand.vertragsaenderungen
```

Consumer mit Anzeige von Key und Partition:

```
./kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic
bestand.vertragsaenderungen --from-beginning --formatter-property print.key=true
--formatter-property print.partition=true
```

Producer mit Vertragsnummer als Key:

```
./kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic
bestand.vertragsaenderungen --reader-property parse.key=true --reader-property
key.separator=:
```

```
V-1001:1|Adresse geändert
V-1002:1|Beitrag angepasst
V-1001:2|Bankverbindung neu
V-1003:1|Kündigung zum 31.12.
V-1001:3|Tarifwechsel
```

Der Wert enthält vorn eine Versionsnummer je Vertrag. Wozu, zeigt die Fallstudie in Kapitel 18.

KRaft-Quorum und Plattenbelegung ansehen:

```
./kafka-metadata-quorum.sh --bootstrap-server kafka1:9092 describe --status
```

```
./kafka-log-dirs.sh --bootstrap-server kafka1:9092 --describe --topic-list
bestand.vertragsaenderungen
```

4.7.1 Die Ausgaben lesen

- `--describe`**:** `Leader` = Broker-ID des aktuellen Leaders. `Replicas` = alle Kopien; die erste ist der Preferred Leader. `Isr` = aktuelle Kopien. `Elr`/`LastKnownElr` sind leer, solange alles gesund ist.
- **Erwartung bei 6 Partitionen auf 3 Brokern:** je Broker 2 Leader.
- `describe --status`**:** `LeaderId` = aktiver Controller, `HighWatermark` = Stand des Metadaten-Logs, `MaxFollowerLag` sollte nahe 0 sein, `CurrentVoters` = Controller-Quorum, `CurrentObservers` = reine Broker.
- **Schneller Gesundheitscheck:** `./kafka-topics.sh --bootstrap-server kafka1:9092 --describe --under-replicated-partitions` – eine leere Ausgabe ist gut.

4.8 Woran sehe ich das im Betrieb?

Frage	Metrik	Gesund
Ist genau ein Controller aktiv?	`ActiveControllerCount`, über alle Knoten summiert	Summe = 1
Hat jede Partition einen Leader?	`OfflinePartitionsCount`	0
Sind die Leader gleichmäßig verteilt?	`LeaderCount` je Broker, `PreferredReplicaImbalanceCount`	gleich viele bzw. 0

Im Lab zeigt das Grafana-Dashboard „Kafka Broker“ diese Werte in den Zeilen „Zustand“ und „Replikation und KRaft“. Kapitel 14 baut daraus Alarme.

4.9 Weiterführende Links

- Kafka-Dokumentation: https://kafka.apache.org/documentation/
- KRaft: https://kafka.apache.org/documentation/#kraft
- Kafbat UI: https://github.com/kafbat/kafka-ui

5 Zuverlässigkeit und Replikation

Geht eine bestätigte Vertragsänderung verloren, wenn ein Broker ausfällt? Die ehrliche Antwort: Es kommt auf drei Einstellungen an. Dieses Kapitel erklärt sie, zeigt die Faustregel für Produktion und lässt im Lab einen Broker ausfallen.

Die Beispiele zu diesem Kapitel

`acks=all` und `min.insync.replicas=2` stehen in `04-producer-spring` (`application.properties` und `TopicBuilder`). Aufgabe 2 der `ANLEITUNG.md` dort führt den Broker-Ausfall mit `NOT_ENOUGH_REPLICAS` vor; `delivery.timeout.ms` bestimmt, wann der Producer aufgibt.

5.1 Drei Stellschrauben bestimmen die Haltbarkeit

Stellschraube	Frage	Wo eingestellt
Replication Factor	Wie viele Kopien einer Partition gibt es?	beim Anlegen des Topics
`min.insync.replicas`	Wie viele aktuelle Kopien braucht ein Schreibvorgang mindestens?	am Topic oder Broker
`acks`	Worauf wartet der Producer?	im Code bzw. in der Konfiguration der Anwendung

Nur alle drei zusammen entscheiden, ob Daten einen Ausfall überleben. Eine allein nützt wenig: RF 3 mit `acks=1` kann trotzdem Daten verlieren.

5.1.1 Hintergrund und Konsequenzen

- **Drei Stellschrauben, drei Zuständige:** Das Plattformteam kann RF und `min.insync.replicas` erzwingen, `acks` nur per Vorgabe an die Entwicklung. Haltbarkeit ist eine Absprache zwischen Teams.
- `min.insync.replicas` **wirkt nur bei** `acks=all`**.** Ein Producer mit `acks=1` ignoriert es.
- **Der Auslieferungszustand ist unsicher:** Broker-Standard ist `default.replication.factor=1` und `min.insync.replicas=1`. In Produktion daher am Broker setzen: `default.replication.factor=3`, `min.insync.replicas=2`.
- **Den Replication Factor nachträglich ändern** geht nur über `kafka-reassign-partitions.sh` (neue Replikaliste), nicht über `kafka-configs.sh`.

5.2 Acknowledgements: `acks`

Einstellung	Producer wartet auf	Risiko / Latenz
`acks=0`	keine Bestätigung	schnell; Verlust möglich
`acks=1`	Bestätigung des Leaders	Verlust möglich, wenn der Leader ausfällt, bevor die Follower kopiert haben
`acks=all`	alle In-Sync-Repliken	hohe Sicherheit bei ausreichender ISR; etwas mehr Latenz

`acks=all` **ist die sichere Einstellung und seit Kafka 3.0 der Standard**, zusammen mit dem idempotenten Producer. Für Vertragsänderungen, Schadenmeldungen oder Zahlungen gibt es keinen Grund, davon abzuweichen.

Buch Zelenin/Kropp: Kap. 5.1.1, S. 69–71

5.2.1 Hintergrund und Konsequenzen

- `acks=all` **kostet Latenz, kaum Durchsatz:** Der Producer wartet auf das langsamste ISR-Mitglied. Durchsatz holt man über Batching und parallele Requests zurück (Kapitel 15).
- **Idempotenz hängt an** `acks=all`**:** Setzt jemand im Java-Client ausdrücklich `acks=1`, wird die Idempotenz stillschweigend abgeschaltet. Ein „Tuning“ auf `acks=1` bringt neben dem Verlustrisiko auch Duplikate bei Wiederholungen zurück.
- **Fehler kommen asynchron:** Der Producer wiederholt bis `delivery.timeout.ms` (Standard 2 Minuten) und meldet erst dann einen Fehler im Callback. Code, der `send()` aufruft und das Ergebnis nie prüft, verliert Daten trotz `acks=all` – still (Kapitel 7).
- `acks=0` nur für Daten, deren Verlust egal ist, etwa Telemetrie von Telematik-Boxen mit hoher Frequenz.

5.3 In-Sync Replicas (ISR)

ISR = die Repliken, die mit dem Leader aktuell sind.

- Ein Follower, der länger als `replica.lag.time.max.ms` hinterherhängt, fällt aus der ISR.
- `acks=all` wartet auf alle ISR-Mitglieder.
- **Ist die ISR kleiner als** `min.insync.replicas`**, lehnt der Broker das Schreiben mit** `acks=all` **ab** – der Producer bekommt `NOT_ENOUGH_REPLICAS`. Das ist kein Fehler, sondern der Schutz: lieber nicht schreiben als nur auf eine einzige Kopie.

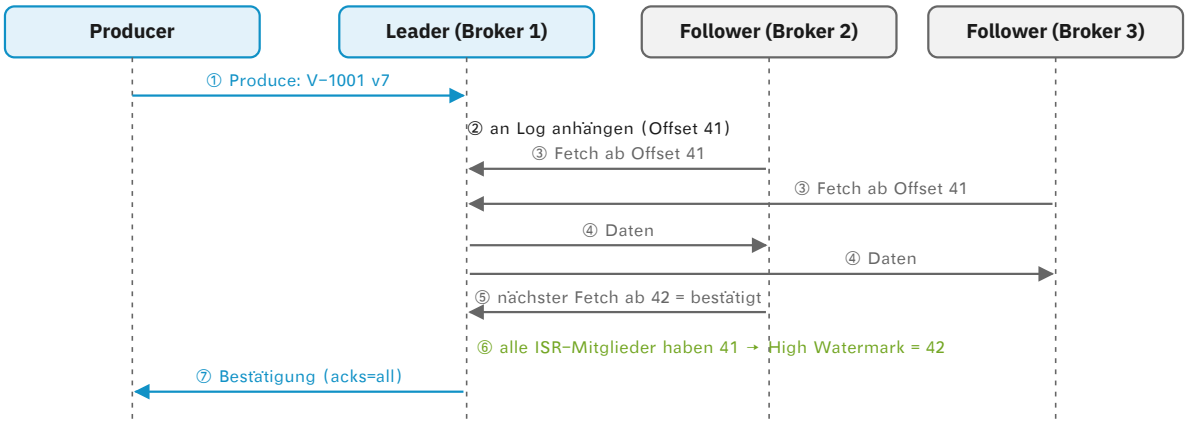


Abbildung 4: Ablauf eines Schreibvorgangs mit acks=all

Buch Zelenin/Kropp: Kap. 5.1.2, S. 71–74

5.3.1 Hintergrund und Konsequenzen

- `replica.lag.time.max.ms` (Standard 30 s): Entscheidend ist, wie lange ein Follower nicht aufgeholt hat, nicht wie viele Nachrichten fehlen. Ein Follower außerhalb der ISR repliziert weiter und kehrt automatisch zurück.
- **Flattern:** Ständiges Raus und Rein (`IsrShrinksPerSec`/`IsrExpandsPerSec`) deutet auf Netz-, Platten- oder GC-Probleme eines Brokers hin.

- **Schleichende Gefahr:** Mit `min.insync.replicas=1` kann die ISR auf den Leader allein schrumpfen – dann ist `acks=all` faktisch `acks=1`.
- **Was der Producer bei `NOT_ENOUGH_REPLICAS` tut:** Er wiederholt bis `delivery.timeout.ms`; der Puffer (`buffer.memory`) läuft voll, `send()` blockiert bis `max.block.ms` (60 s). Für die Anwendung sieht das aus wie „Kafka hängt“. Anwendungen brauchen ein Konzept für Rückstau.
- **Lesen geht weiter:** Consumer lesen alles bereits Bestätigte. Nur das Schreiben mit `acks=all` stoppt.

5.3.2 Unter der Haube: High Watermark, Leader-Epoche und Truncation

Jede Replik kennt zwei Positionen:

- **Log End Offset (LEO):** der nächste Offset, den sie schreiben würde.
- **High Watermark (HW):** der kleinste LEO aller ISR-Mitglieder – bis hierhin haben alle aktuellen Kopien die Daten.

Der Leader erfährt den LEO der Follower aus ihren Fetch-Anfragen: Wer „ab Offset 42“ anfragt, hat 41 sicher. Liegen alle ISR-Mitglieder bei mindestens 42, rückt die HW auf 42, und erst dann bestätigt der Leader einen Schreibvorgang mit `acks=all`. Consumer lesen nur bis zur HW – sie sehen nie eine Nachricht, die bei einem Leader-Wechsel verschwinden könnte.

Was bei einem Leader-Wechsel passiert: Jeder Leader bekommt eine fortlaufende **Leader-Epoche**. Der neue Leader schreibt ab seinem eigenen LEO weiter. Kommt der alte Leader als Follower zurück, fragt er den neuen: „Wo endete deine Epoche?“ Alles, was er darüber hinaus hat, schneidet er ab (*Truncation*). Mit `acks=all` und `min.insync.replicas=2` waren diese abgeschnittenen Nachrichten nie bestätigt – kein Producer hat sie als geschrieben gemeldet. Mit `acks=1` dagegen schon: Das ist der Datenverlust, vor dem die Faustregel schützt.

5.4 `min.insync.replicas` genauer erklärt

- **Was es ist:** die Mindestgröße der ISR (Leader mitgezählt), damit der Broker einen Schreibvorgang mit `acks=all` überhaupt annimmt. Es ist eine Untergrenze, keine Zahl, die Kafka herstellt.
- **Warum man es braucht:** `acks=all` heißt „alle Repliken, die *gerade* in der ISR sind“ – nicht „alle drei“. Schrumpft die ISR auf den Leader allein, bestätigt `acks=all` nach einer einzigen Kopie. Erst `min.insync.replicas` macht `acks=all` belastbar.
- **Zwei Prüfzeitpunkte:** Vor dem Anhängen: ISR kleiner als das Minimum → Ablehnung mit `NOT_ENOUGH_REPLICAS`, nichts wird geschrieben. Nach dem Anhängen, falls die ISR während des Wartens auf die Follower schrumpft: `NOT_ENOUGH_REPLICAS_AFTER_APPEND`. Die Nachricht steht dann schon im Log des Leaders, der Producer wiederholt – dank idempotentem Producer ohne Duplikat.
- **Die Rechnung dahinter:** Eine bestätigte Nachricht liegt auf mindestens `min.insync.replicas` Kopien. Ausfälle ohne Datenverlust = Minimum − 1. Ausfälle ohne Schreibstopp = RF − Minimum.

Merksatz

`acks=all` fragt: „Wer ist gerade da?“ – `min.insync.replicas` fragt: „Sind genug da?“

RF / Minimum	Schreiben läuft bis	Jede Bestätigung liegt auf	Einsatz
3 / 1	2 Ausfälle	eventuell nur 1 Kopie	nur wenn Verfügbarkeit alles ist
3 / 2	**1 Ausfall**	**mindestens 2 Kopien**	**Standard-Empfehlung**
3 / 3	0 Ausfälle	3 Kopien	jede Wartung stoppt das Schreiben – praktisch unbrauchbar
5 / 3	2 Ausfälle	mindestens 3 Kopien	sehr kritische Daten, mehrere Standorte

- **Wo es gesetzt wird:** am Broker als Standard für alle Topics oder pro Topic mit `kafka-configs.sh`. Der Topic-Wert gewinnt. Kafka 4.x hinterlegt den Wert zusätzlich clusterweit (nötig für ELR) – deshalb zeigt `--describe` bei jedem Topic `Configs: min.insync.replicas=1`, auch ohne eigene Einstellung.
- **Typischer Denkfehler:** „Minimum = RF, dann ist es am sichersten." Dann reicht ein einziger Broker-Neustart für einen Schreibstopp.

5.5 Faustregel für Produktion

RF 3, `min.insync.replicas=2`**,** `acks=all`**.** Dann darf ein Broker ausfallen, das Schreiben läuft weiter, und bestätigte Daten liegen auf zwei Kopien. Fallen zwei aus, stoppt das Schreiben – gewollt.

Bei RF 3	Ein Broker ausgefallen	Zwei Broker ausgefallen
`acks=all`, Minimum 2	Schreiben läuft weiter	Schreibstopp; Schutz vor Einzelkopie
`acks=all`, Minimum 1	Schreiben läuft weiter	Einzelkopie; Verlustrisiko bei weiterem Ausfall
`acks=1`, Minimum 2	Leader reicht; Verlustrisiko	letzter Leader schreibt allein

Annahme: Die verbleibenden Repliken sind in der ISR, die Leader-Wahl ist sauber.

5.5.1 Hintergrund und Konsequenzen

- **Wartung frisst die Reserve:** Beim Rolling Restart ist immer ein Broker weg. In diesem Fenster führt ein weiterer Ausfall zum Schreibstopp. Wartung daher nur bei gesundem Cluster beginnen und erst weitermachen, wenn die ISR wieder voll ist (Kapitel 17).
- **Geordnetes Herunterfahren:** Ein normaler Dienst-Stopp löst einen *Controlled Shutdown* aus – der Broker gibt seine Leader vorher ab. Ein harter Absturz führt zu einer Leader-Wahl nach Ablauf der Broker-Session (wenige Sekunden).
- **Zwei Rechenzentren:** RF 3 auf zwei Standorte heißt 2 + 1 Kopien. Fällt der Standort mit zwei Kopien aus, ist `min.insync.replicas=2` unterschritten. Ein Cluster über Standorte braucht daher drei Standorte (mindestens für die Controller) oder getrennte Cluster mit Spiegelung (Kapitel 19).

- **Interne Topics** (`__consumer_offsets`, `__transaction_state`) haben eigene Einstellungen. In Testumgebungen werden sie oft auf RF 1 gesetzt – beim Übernehmen von Konfigurationen in Produktion prüfen.

5.6 Zustellgarantien

Garantie	Bedeutung	Voraussetzung
At-most-once	höchstens einmal – Verlust möglich	Commit vor der Verarbeitung
At-least-once	mindestens einmal – Duplikate möglich; der Normalfall	Commit nach der Verarbeitung
Exactly-once	genau einmal im Transaktionsumfang	idempotenter Producer + Transaktionen + `isolation.level=read_committed`

At-least-once ist der Normalfall – darauf baut man idempotente Verarbeitung. Exactly-once gibt es in Kafka nur innerhalb von Kafka. Sobald eine Datenbank im Spiel ist, braucht die Anwendung selbst Idempotenz – etwa eine Versionsnummer je Vertrag (Kapitel 9 und 18).

Buch Zelenin/Kropp: Kap. 5.1.3 und 5.2, S. 74–82

5.6.1 Hintergrund und Konsequenzen

- **Idempotenter Producer:** Jeder Producer bekommt eine ID, jede Nachricht pro Partition eine Sequenznummer; der Broker verwirft doppelte Wiederholungen. Das gilt nur innerhalb einer Producer-Sitzung. Startet das Bestandssystem neu und sendet dieselbe Änderung erneut, ist das für Kafka eine neue Nachricht.
- **Woher Duplikate beim Consumer kommen:** verarbeitet, aber vor dem Commit abgestürzt → nach dem Neustart wird ab dem letzten Commit erneut gelesen.
- **At-most-once entsteht oft ungewollt:** Commit vor der Verarbeitung oder Auto-Commit bei Verarbeitung in eigenen Threads.

5.7 Leader-Wahl und Datenverlust

	Verhalten
Saubere Wahl	Nur ISR-Repliken dürfen übernehmen. Ohne geeignete Replik bleibt die Partition offline. Konsistenz vor Verfügbarkeit.
Unclean Election (`unclean.leader.election.enable=true`)	Auch Repliken außerhalb der ISR dürfen übernehmen. Die Partition ist wieder verfügbar, aber Daten können verloren sein.
Eligible Leader Replicas (ELR), standardmäßig aktiv seit Kafka 4.1	Kafka merkt sich Repliken, die sicher alle bestätigten Daten haben, auch wenn sie aus der ISR gefallen sind. Leader-Wahl ohne Datenverlust.

Die Abwägung lautet: Verfügbarkeit gegen Konsistenz. In einer Versicherung ist die Antwort fast immer: Unclean Election aus lassen. Die ELR zeigt `kafka-topics.sh --describe` in den Spalten `Elr` und `LastKnownElr`.

Buch Zelenin/Kropp: Kap. 5.3, S. 83–88

5.7.1 Hintergrund und Konsequenzen

- **Warum ELR funktioniert:** Fällt die ISR unter `min.insync.replicas`, werden `acks=all`-Schreibvorgänge abgelehnt – die High Watermark bleibt stehen. Die Repliken, die in diesem Moment aus der ISR fallen, haben also alle bestätigten Daten und dürfen später sicher Leader werden.
- **ELR braucht** `min.insync.replicas` ≥ **2** – ein weiterer Grund für die Faustregel.
- **Unclean Election im Notfall gezielt auslösen** statt dauerhaft einschalten: `kafka-leader-election.sh --election-type unclean --topic … --partition …`. Dann ist es eine bewusste, dokumentierte Entscheidung.
- **Was verloren geht:** Der neue Leader hat ein kürzeres Log. Kommt der alte Leader zurück, schneidet er seine überzähligen Nachrichten ab (Truncation). Bei `acks=1` waren diese Nachrichten bereits bestätigt.

5.8 Lab: Broker-Ausfall

Voraussetzung: Topic `bestand.vertragsaenderungen` mit RF 3 und vollständiger ISR (Kapitel 4).

1. Mindestens zwei synchrone Kopien verlangen (Container):

```
./kafka-configs.sh --bootstrap-server kafka1:9092 --alter --entity-type topics --
entity-name bestand.vertragsaenderungen --add-config min.insync.replicas=2
```

```
./kafka-configs.sh --bootstrap-server kafka1:9092 --describe --entity-type topics
--entity-name bestand.vertragsaenderungen
```

2. Producer mit `acks=all` **starten** (Container):

```
./kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic
bestand.vertragsaenderungen --reader-property parse.key=true --reader-property
key.separator=: --command-property acks=all
```

3. Broker 3 ausfallen lassen (PowerShell, nicht im Container):

```
docker stop kafka3
```

4. Wirkung ansehen: Die ISR schrumpft auf 1,2; die Partitionen von Broker 3 bekommen neue Leader. Der Producer schreibt weiter – zwei Kopien reichen.

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --describe --under-replicated-
partitions
```

5. Drei Kopien verlangen, obwohl nur zwei da sind:

```
./kafka-configs.sh --bootstrap-server kafka1:9092 --alter --entity-type topics --
entity-name bestand.vertragsaenderungen --add-config min.insync.replicas=3
```

Im Producer eine Änderung schreiben: Es erscheinen wiederholt Warnungen mit `NOT_ENOUGH_REPLICAS`, die Nachricht wird nicht bestätigt. Nach `delivery.timeout.ms` (2 Minuten) kommt der eigentliche Fehler.

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --describe --under-min-isr-
partitions
```

6. Broker 3 wieder starten (PowerShell). Der Follower holt auf und kehrt in die ISR zurück; der Producer liefert die wartende Nachricht nach, falls die Zeit noch nicht abgelaufen ist.

```
docker start kafka3
```

7. Leader neu verteilen: Die ISR ist wieder vollständig – aber Broker 3 ist nirgends Leader.

```
./kafka-leader-election.sh --bootstrap-server kafka1:9092 --election-type
preferred --all-topic-partitions
```

8. Zurück zur Faustregel:

```
./kafka-configs.sh --bootstrap-server kafka1:9092 --alter --entity-type topics --
entity-name bestand.vertragsaenderungen --add-config min.insync.replicas=2
```

Nie zwei Broker gleichzeitig stoppen

Im Lab sind Broker und KRaft-Controller kombiniert. Zwei von drei Controllern weg bedeutet: keine Mehrheit, der Cluster kann keine Leader mehr wählen.

5.9 Woran sehe ich das im Betrieb?

Frage	Metrik	Gesund
Sind alle Kopien aktuell?	`UnderReplicatedPartitions`, summiert	0 (kurz mehr beim Neustart ist normal)
Kann mit `acks=all` geschrieben werden?	`UnderMinIsrPartitionCount`	0 – sonst stehen die Producer
Ist das Replizieren stabil?	`IsrShrinksPerSec`, `IsrExpandsPerSec`	0 außerhalb von Wartung

Kapitel 14 macht daraus Alert-Regeln.

5.10 Weiterführende Links

- Kafka-Dokumentation, Replikation: https://kafka.apache.org/documentation/#replication
- KIP-966 Eligible Leader Replicas: https://cwiki.apache.org/confluence/display/KAFKA/KIP-966%3A+Eligible+Leader+Replicas

6 Consumer Groups und Offsets

Die Bestandsverwaltung, das Inkasso und das Reporting lesen dieselben Vertragsänderungen – jedes System in seinem Tempo, und die Bestandsverwaltung mit mehreren Instanzen parallel. Dafür gibt es Consumer Groups. Wie weit eine Gruppe gekommen ist, merkt sich Kafka in Offsets.

Die Beispiele zu diesem Kapitel

`03-consumer-plain` zeigt `group.id`, `poll` und `commitSync` im Klartext. Für die Frage „Wo liegt ein Key?“ gibt es in `01-shell` die Skripte `10-partition-fuer-key` (vorab rechnen wie der Producer), `11-partition-lesen` (eine Partition ohne Gruppe, `assign`) und `12-nachrichten-zu-key` (lesen und filtern, der Normalfall). `06-consumer-groups` macht die Gruppe aus der Spring-App heraus sichtbar: `ZUGEWIESEN/ABGEGEBEN` beim Rebalance (`ConsumerSeekAware`), manuelles `acknowledge()`, Lag über `--app.dauer-ms`, Reset über `seekToBeginning`. Die Shell-Gegenstücke `05-consumer-gruppe`, `06-gruppe-ansehen` und `07-offsets-zuruecksetzen` liegen in `01-shell`.

6.1 Consumer Groups

Innerhalb einer Gruppe teilen sich die Consumer die Partitionen: **Jede Partition hat genau einen Consumer.** Verschiedene Gruppen lesen unabhängig voneinander.

```
Topic bestand.vertragsaenderungen · 6 Partitionen

Gruppe bestandsverwaltung · 3 Instanzen     Gruppe reporting · 1 Instanz
  P0, P1 → Instanz A                          P0 … P5 → Instanz R1
  P2, P3 → Instanz B
  P4, P5 → Instanz C
```

Mehr Consumer als Partitionen bringen nichts – der überzählige wartet. Die Zahl der Partitionen begrenzt, wie parallel die Bestandsverwaltung arbeiten kann. Und weil eine Partition nur einen Consumer hat, bleibt die Reihenfolge darin erhalten.

Buch Zelenin/Kropp: Kap. 4.2.2, S. 53–55; Kap. 9.4, S. 157–165

6.1.1 Hintergrund und Konsequenzen

- **Group Coordinator:** Jede Gruppe wird von einem Broker koordiniert – dem Leader der Partition von `__consumer_offsets`, auf die der Hash der `group.id` fällt. Fällt dieser Broker aus, übernimmt ein anderer, die Gruppe stockt kurz.
- **Mehrere Gruppen = Fan-out:** Jede Anwendung bekommt ihre eigene `group.id`. Zwei Anwendungen mit versehentlich gleicher `group.id` teilen sich die Daten – das Inkasso sieht dann nur die Hälfte der Vertragsänderungen. Ein typischer Fehler beim Kopieren von Konfigurationen.
- **Überzählige Consumer** sind eine Reserve: Sie übernehmen sofort, wenn ein anderer ausfällt.
- **Ein Consumer-Objekt ist nicht thread-safe.** Parallelität innerhalb einer Anwendung entsteht durch mehrere Consumer-Instanzen (in Spring Kafka über `concurrency`, Kapitel 8), begrenzt durch die Partitionszahl.

- **Zuweisungsstrategie:** Im klassischen Protokoll entscheidet `partition.assignment.strategy` im Client (Standard Range + CooperativeSticky). Im neuen Protokoll wählt der Broker (`uniform` oder `range`).

6.2 Rebalancing

Kommt ein Consumer hinzu oder fällt weg, werden die Partitionen neu verteilt: Änderung → Neuverteilung → Fortsetzen. **Während des Rebalancings stockt die Verarbeitung.**

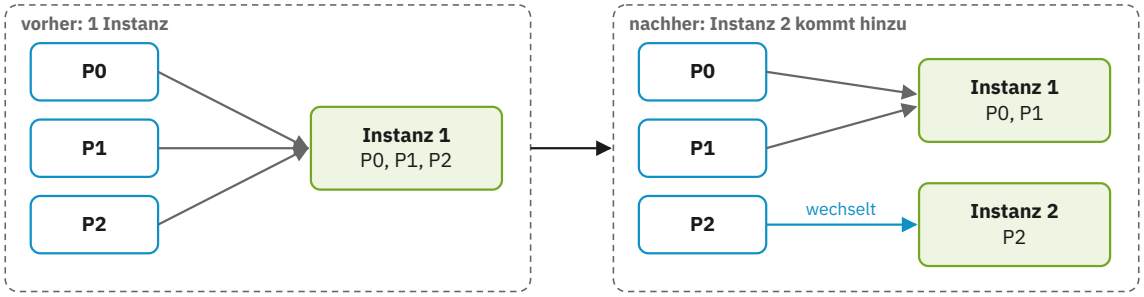


Abbildung 5: Rebalance mit dem neuen Gruppenprotokoll

- Static Membership und Cooperative Rebalancing reduzieren die Unterbrechung.
- **Neues Gruppenprotokoll (KIP-848, produktionsreif seit Kafka 4.0):** Mit `group.protocol=consumer` koordiniert der Broker die Zuteilung, und nur die betroffenen Partitionen wechseln.
- `kafka-groups.sh --bootstrap-server kafka1:9092 --list` zeigt Typ und Protokoll jeder Gruppe.

6.2.1 Hintergrund und Konsequenzen

- **Auslöser:** Ein Consumer kommt oder geht (Deployment, Skalierung); ein Absturz wird nach `session.timeout.ms` erkannt (Standard 45 s); oder ein Consumer ruft `poll()` nicht rechtzeitig auf (`max.poll.interval.ms`, Standard 5 Minuten) und wird aus der Gruppe geworfen.
- **Rebalance-Sturm:** Dauert die Verarbeitung eines Poll-Batches länger als `max.poll.interval.ms`, fliegt der Consumer raus, der nächste übernimmt denselben Batch und fliegt ebenfalls raus. Ergebnis: kaum Fortschritt, viele Duplikate. Abhilfe: `max.poll.records` senken oder die Verarbeitung beschleunigen.
- **Eager vs. Cooperative:** Eager (alt) nimmt allen Consumern alle Partitionen weg – kompletter Stopp. Cooperative bewegt nur die betroffenen Partitionen.
- **Neues Protokoll im Detail:** Der Broker berechnet die Zuweisung, Heartbeat-Intervall und Session-Timeout werden zentral am Broker gesteuert, `partition.assignment.strategy` im Client wird ignoriert. Eine bestehende Gruppe kann im laufenden Betrieb umgestellt werden, wenn die Instanzen nacheinander neu starten.
- **Static Membership** (`group.instance.id`): Ein Consumer, der innerhalb des Session-Timeouts mit derselben ID zurückkommt, behält seine Partitionen – kein Rebalance beim Rolling Restart.
- **Beobachten:** `kafka-consumer-groups.sh --describe --group … --state` zeigt den Gruppenstatus (Stable, PreparingRebalance, Empty …).

6.2.2 Unter der Haube: das Gruppenprotokoll

Klassisches Protokoll:

1. Jeder Consumer schickt dem Group Coordinator ein `JoinGroup`.
2. Der Coordinator wartet, bis alle bekannten Mitglieder beigetreten sind, und bestimmt einen von ihnen zum Gruppenleiter.
3. Der Gruppenleiter – ein Client! – berechnet die Zuteilung mit seiner `partition.assignment.strategy` und schickt sie per `SyncGroup` zurück.
4. Alle Mitglieder bekommen ihre Partitionen. Bei der Eager-Strategie haben vorher alle ihre Partitionen abgegeben; bei Cooperative Sticky geben sie nur ab, was wechselt – dafür braucht es eine zweite Runde.

Jeder Rebalance ist ein globaler Synchronisationspunkt: Ein einziger langsamer Consumer hält die ganze Gruppe auf.

Neues Protokoll (KIP-848):

1. Jeder Consumer schickt regelmäßig ein `ConsumerGroupHeartbeat` mit seinen Abonnements.
2. Der Coordinator auf dem Broker berechnet die Ziel-Zuteilung selbst und erhöht die **Gruppen-Epoche**.
3. Jedes Mitglied erfährt im nächsten Heartbeat, was es abgeben und was es übernehmen soll, und meldet zurück, wenn es fertig ist. Eine Partition wird erst neu vergeben, wenn ihr alter Besitzer sie freigegeben hat.

Es gibt keine globale Pause mehr; jedes Mitglied gleicht sich einzeln an. Deshalb sind Rebalances ruhiger und skalieren auf große Gruppen.

6.3 Offsets

Begriff	Bedeutung
Committed Offset	Fortschritt der Gruppe pro Partition: die nächste zu lesende Position. Gespeichert im internen Topic `__consumer_offsets`.
Commit	automatisch (Auto-Commit) oder manuell
`auto.offset.reset`	Startposition ohne gültigen Offset, etwa bei einer neuen Gruppe: `earliest` / `latest`
Lag	Abstand zwischen dem Ende des Logs und dem committeten Offset – wie weit die Gruppe hinterherhängt

Regel für at-least-once: erst verarbeiten, dann committen – und die Verarbeitung idempotent gestalten.

Buch Zelenin/Kropp: Kap. 9.3, S. 153–157

6.3.1 Hintergrund und Konsequenzen

- **Committeter Offset = nächste zu lesende Position,** also letzte verarbeitete + 1. Nach dem Lesen der Offsets 0–2 zeigt `--describe` den Wert 3.
- **Auto-Commit:** Alle 5 s (`auto.commit.interval.ms`) committet `poll()` die Offsets der zuvor gelieferten Records. Bei Verarbeitung direkt in der Poll-Schleife ergibt das at-least-once. Wird die Verarbeitung an andere Threads übergeben, kann der Commit vor der Verarbeitung liegen → Verlust bei Absturz.

- **Offsets verfallen:** `offsets.retention.minutes` (Standard 7 Tage) zählt ab dem Moment, in dem die Gruppe leer ist. Ist eine Anwendung länger als 7 Tage aus – etwa eine Test-Umgebung über die Feiertage –, sind ihre Offsets weg und `auto.offset.reset` greift beim Neustart.
- **Lag von außen messen:** Die Client-Metrik `records-lag-max` gibt es nur, solange der Consumer läuft. Ist er abgestürzt, verschwindet die Metrik – und ein Alarm darauf schweigt genau dann, wenn er gebraucht wird (Kapitel 14).
- **Lag in Zeit ist aussagekräftiger als in Nachrichten:** 10.000 Vertragsänderungen Rückstand sind im Monatsabschluss nichts, an einem ruhigen Sonntag viel.
- `__consumer_offsets` hat 50 Partitionen und ist kompaktiert – es bleibt je Gruppe, Topic und Partition nur der letzte Commit.

6.4 Vorführung: Wie ein Consumer nach dem Neustart weitermacht

Ohne `--group` startet der Console-Consumer jedes Mal mit einer neuen Zufallsgruppe. Mit einer festen Gruppe macht er dort weiter, wo er aufgehört hat. Drei Fenster im Container:

1. **Fenster A** – Consumer mit fester Gruppe:

   ```
   ./kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic
   tarif.beitragsaenderungen --group inkasso
   ```

   Fenster B – Producer; ein paar Zeilen senden, A liest mit:

   ```
   ./kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic
   tarif.beitragsaenderungen
   ```

2. Fenster A mit Strg+C beenden. In Fenster B drei neue Beitragsänderungen senden.
3. **Fenster C** – Rückstand ansehen: `CURRENT-OFFSET` steht, `LOG-END-OFFSET` ist 3 weiter, `LAG` = 3.

   ```
   ./kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group
   inkasso
   ```

4. In Fenster A denselben Befehl wie in Schritt 1 erneut starten: Genau die drei verpassten Änderungen erscheinen. In Fenster C noch einmal `--describe`: `LAG` = 0.

Gut zu wissen: `--from-beginning` wirkt nur, solange die Gruppe noch keinen Commit hat. Wird kurz nach einer Nachricht abgebrochen, war der Auto-Commit eventuell noch nicht durch – dann kommt die Nachricht doppelt: at-least-once.

Dieselbe Vorführung mit einer Spring-Boot-Anwendung, die jeden Commit auf der Konsole anzeigt, folgt in Kapitel 8.

6.5 Offset-Reset

Manchmal muss man zurückspulen, etwa nachdem ein Fehler in der Beitragsberechnung behoben ist und die betroffenen Änderungen erneut verarbeitet werden sollen. **Zwei Regeln: Die Gruppe muss gestoppt sein, und immer erst `--dry-run`, dann `--execute`.**

```
./kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group
bestandsverwaltung
```

```
./kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --group
bestandsverwaltung --topic bestand.vertragsaenderungen --reset-offsets --to-
datetime 2026-11-02T10:00:00.000 --dry-run
```

```
./kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --group
bestandsverwaltung --topic bestand.vertragsaenderungen --reset-offsets --to-
datetime 2026-11-02T10:00:00.000 --execute
```

Nur Partition 2 auf Offset 10:

```
./kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --group
bestandsverwaltung --topic bestand.vertragsaenderungen:2 --reset-offsets --to-
offset 10 --execute
```

Option	Ziel
`--to-earliest` / `--to-latest`	Anfang bzw. Ende des Logs
`--to-offset 10`	fester Offset (meist mit Partitionsangabe)
`--shift-by -5`	relativ zurück oder vor
`--to-datetime 2026-11-02T10:00:00.000`	erster Offset ab diesem Zeitpunkt
`--by-duration PT1H`	eine Stunde zurück
`--topic t`, `--topic t:2`, `--all-topics`	Geltungsbereich

6.5.1 Hintergrund und Konsequenzen

- **Die Gruppe muss leer sein:** Läuft noch ein Mitglied, verweigert das Werkzeug den Reset. In Produktion heißt das: alle Instanzen der Anwendung stoppen.
- **Vorher sichern:** `--reset-offsets … --dry-run --export > stand.csv` schreibt den aktuellen Stand als CSV. Mit `--from-file stand.csv --execute` lässt sich der Reset rückgängig machen.
- **Offsets gelten pro Partition:** `--to-offset 500` setzt **jede** Partition auf 500, obwohl dieselbe Zahl in jeder Partition einen anderen Zeitpunkt bedeutet. Einzelne Partitionen gezielt wählen mit `--topic bestand.vertragsaenderungen:0,2`. Für einen einheitlichen Neustart über alle Partitionen ist `--to-datetime` meist die richtige Wahl.
- **Vor der Retention gibt es nichts mehr:** Liegt der Zielpunkt vor dem Anfang des Logs, wird ab dem ältesten noch vorhandenen Offset gelesen.
- **Auch beim Replay gibt es keine Reihenfolge über Partitionen hinweg.**
- **Erneut verarbeiten heißt Duplikate:** Ein Reset ist nur sicher, wenn die Verarbeitung idempotent ist, und kann nachgelagerte Systeme mit Last fluten.
- **Ganz zurücksetzen:** `kafka-consumer-groups.sh --delete --group …` löscht die Gruppe samt Offsets; beim nächsten Start greift `auto.offset.reset`.

- **Alternative im Code:** `consumer.seek()` bzw. in Spring `ConsumerSeekAware` – für gezielte Wiederholung einzelner Partitionen, ohne die Anwendung zu stoppen.

6.6 Daten erneut verarbeiten

„Topics noch mal schicken“ heißt in Kafka meist: nicht neu senden, sondern **neu lesen**. Die Daten liegen bis zum Ende der Retention im Topic.

Weg	Wann
Offsets der bestehenden Gruppe zurücksetzen	dieselbe Anwendung soll einen Zeitraum erneut verarbeiten
neue Consumer Group mit `auto.offset.reset=earliest`	eine neue Anwendung oder ein Test soll alles lesen; die produktive Gruppe bleibt unberührt
im Code mit `seek()`, `seekToBeginning()`, `offsetsForTimes()`	gezielt, ohne die Anwendung zu stoppen
in ein anderes Topic oder einen anderen Cluster kopieren	eigener Consumer/Producer oder MirrorMaker 2

Jeder Weg erzeugt Duplikate in den nachgelagerten Systemen – sie müssen idempotent sein.

6.7 Fragen aus dem Seminar: Commit, Lücken, gezieltes Nachlesen

6.7.1 Der Commit gehört der Gruppe, nicht der Nachricht

Zwei Consumer sollen dieselbe Nachricht bekommen, und einer committet. Wird der andere noch aktiv? Ja, solange beide in **verschiedenen Gruppen** sind. Der Commit landet in `__consumer_offsets` unter dem Schlüssel (Gruppe, Topic, Partition). Gruppe `inkasso` committet Offset 42, Gruppe `reporting` hat davon nichts mitbekommen und liest Offset 42 beim nächsten Poll ganz normal. Die Nachricht bleibt im Log, bis die Retention sie löscht, egal wie viele Gruppen sie gelesen haben.

Sind beide Consumer in **derselben** Gruppe, bekommt nur einer die Partition zugeteilt. Der zweite sieht die Nachricht nicht wegen des Commits, sondern wegen der Zuweisung beim Rebalance. Der Commit entscheidet dann nur, wo der zweite weitermacht, falls der erste abstürzt und er die Partition übernimmt. Im Lab: `06-consumer-groups` einmal mit der Standardgruppe und einmal mit `--app.gruppe=statistik` starten; beide bekommen alles, `06-gruppe-ansehen` zeigt je Gruppe einen eigenen `CURRENT-OFFSET`.

6.7.2 Lücken gibt es nicht: 3, 5 und 7 verarbeiten, 4 und 6 später?

Nicht mit dem normalen Commit. Kafka merkt sich je Gruppe und Partition nur **eine Zahl**, den nächsten zu lesenden Offset. Wer nach Nachricht 7 committet, sagt „bis 7 bin ich fertig“, und 4 und 6 gelten als erledigt. Nach dem Neustart beginnt die Gruppe bei 8. Vier Auswege:

Weg	Was passiert	Wofür
Anhalten	4 wird wiederholt, erst danach 5, 6, 7; Lag wächst, Mensch greift ein	Nachrichten mit Reihenfolge je Key (Versionen eines Vertrags)
Weglegen und committen	4 und 6 ins Dead-Letter-Topic oder in eine Tabelle „offen“, später einzeln verarbeiten	unabhängige Aufträge; Durchsatz bleibt, Reihenfolge nicht
Zurückspringen	`seek(partition, 4)` liest ab 4 alles noch einmal, auch 5 und 7	nur mit idempotenter Verarbeitung (Versionsprüfung)
Share Group	jede Nachricht einzeln bestätigen, Lücken erlaubt	Queue-Semantik ohne Reihenfolge je Key (Kapitel 18)

Für die Bestandsverwaltung ist nur der erste Weg richtig: Wenn Version 4 fehlt, darf Version 5 nicht verbucht werden. Was passiert, wenn 4 auch in der Wiederholung scheitert, steht in Kapitel 9.

6.7.3 Eine Nachricht gezielt holen

Jede Nachricht ist über Partition und Offset adressierbar, und Lesen verändert nichts. Per Shell, ohne Gruppe und ohne Nebenwirkung:

```
./kafka-console-consumer.sh --bootstrap-server kafka1:9092 \
  --topic beitraege --partition 1 --offset 4 --max-messages 1 \
  --formatter-property print.key=true \
  --formatter-property print.offset=true
```

Die Partition muss bekannt sein; das Log des Consumers oder die Header einer DLT-Nachricht (`kafka_dlt-original-partition`, `kafka_dlt-original-offset`) nennen sie. In Java mit einem eigenen Consumer ohne `group.id`:

```
var tp = new TopicPartition("beitraege", 1);
consumer.assign(List.of(tp)); // assign statt subscribe: keine Gruppe
consumer.seek(tp, 4);
var record = consumer.poll(Duration.ofSeconds(2)).records(tp).get(0);
```

`assign` statt `subscribe` ist der Punkt: kein Rebalance, kein Commit, der laufende Consumer wird nicht gestört. In Spring geht dasselbe über `ConsumerSeekCallback.seek(topic, partition, offset)` im Listener (`06-consumer-groups`; `seekToBeginning` ist der Spezialfall Offset 0). Kafbat UI kann die Nachricht ebenfalls anzeigen und bei Bedarf in ein Topic kopieren. Nicht mehr holbar ist sie nur, wenn die Retention sie gelöscht hat oder ein kompaktiertes Topic eine neuere Nachricht mit demselben Key kennt.

6.8 Szenarien: Wer liest was, und warum eine Gruppe?

Die Fragen aus dem Seminar kreisen um dieselben drei Entscheidungen: Wer legt die Partition fest, wer liest sie, und wer merkt sich den Fortschritt? Die folgenden Szenarien gehen sie der Reihe nach durch, jedes mit dem Skript aus `labs/01-shell`, mit dem man es nachstellt.

6.8.1 Szenario 1: Nachschauen, ohne Spuren zu hinterlassen

Ein Administrator will sehen, was in `beitraege` liegt. Er startet den Console-Consumer ohne `--group`. Kafka gibt ihm eine Wegwerf-Gruppe `console-consumer-12345`, er liest alles, committet

nichts, und nach dem Beenden ist nichts von ihm übrig. Startet er noch einmal, sieht er wieder alles beziehungsweise nur Neues, je nach `--from-beginning`.

Frage	Antwort
Fortschritt gespeichert?	nein
Zweiter Consumer gleichzeitig?	sieht ebenfalls alles
Richtig für	Werkzeuge, Fehlersuche, Kafbat UI
Falsch für	jede Anwendung, die verarbeitet
Lab	`03-consumer.ps1`, zweimal in zwei Fenstern

6.8.2 Szenario 2: Eine Anwendung, ein Neustart

Die Bestandsverwaltung liest `beitraege` mit der Gruppe `beitragsrechner`. Sie verarbeitet drei Nachrichten, committet, wird neu gestartet. Beim Start fragt sie den Koordinator nach dem Offset der Gruppe und macht bei der vierten Nachricht weiter. Nichts fehlt, nichts kommt doppelt.

Frage	Antwort
Woher weiß Kafka, wo sie war?	`__consumer_offsets`, je Gruppe und Partition
Was, wenn der Offset fehlt (neue Gruppe)?	`auto.offset.reset`: `earliest` oder `latest`
Was, wenn sie zwischen Verarbeiten und Commit abstürzt?	die letzte Nachricht kommt noch einmal: at least once
Lab	`05-consumer-gruppe.ps1`, Strg+C, drei Zeilen mit `02-producer.ps1`, `06-gruppe-ansehen.ps1` zeigt LAG 3, Consumer neu starten

Das ist der Unterschied zu Szenario 1: Der Name der Gruppe macht aus einem Fremden einen Bekannten.

6.8.3 Szenario 3: Zwei Instanzen derselben Anwendung

Die Last steigt, die Bestandsverwaltung wird ein zweites Mal gestartet, gleiche Gruppe. Der Koordinator verteilt die drei Partitionen: zwei an die erste Instanz, eine an die zweite. Jede Nachricht wird genau einmal gelesen. Welche Instanz welche Partition bekommt, entscheidet der Koordinator, nicht die Anwendung.

Frage	Antwort
Lesen beide alles?	nein, jede Partition hat genau ein Mitglied
Kann ich mir eine Partition aussuchen?	nicht mit Gruppe; das macht der Koordinator
Was ist ein Mitglied?	ein Consumer-Thread: ein Prozess, oder ein Thread bei `concurrency` größer 1
Wie viele Mitglieder sind sinnvoll?	höchstens so viele wie Partitionen; der Rest wartet
Lab	zwei Fenster `05-consumer-gruppe.ps1`; in Java `06-consumer-groups` mit `ZUGEWIESEN` im Log oder `--app.threads=3`

Mehrere Mitglieder stehen nicht im Code. `05-consumer-spring` hat eine Listener-Methode und einen Gruppennamen in `application.properties`; Mitglieder entstehen, indem man die Anwendung mehrmals startet oder `spring.kafka.listener.concurrency` setzt.

6.8.4 Szenario 4: Eine Instanz fällt aus

Instanz 2 stürzt ab, hart, ohne sich abzumelden. Nach `session.timeout.ms` (45 Sekunden) erklärt der Koordinator sie für tot und gibt ihre Partition an Instanz 1. Die beginnt dort beim letzten Commit von Instanz 2. Hatte Instanz 2 zehn Nachrichten verarbeitet, aber noch nicht committet, verarbeitet Instanz 1 diese zehn noch einmal.

Frage	Antwort
Geht etwas verloren?	nein, die Partition wechselt nur den Leser
Kommt etwas doppelt?	ja, bis zum letzten Commit; deshalb Versionsnummer je Vertrag (Kapitel 18)
Bleibt die Reihenfolge?	ja, die Partition ist dieselbe, nur das Mitglied wechselt
Warum darf ich die Partition nicht festnageln?	genau deshalb: Sonst könnte niemand übernehmen
Lab	`06-consumer-groups` in zwei Fenstern, eines mit Strg+C beenden: `ABGEGEBEN` und `ZUGEWIESEN`; in der Fallstudie `18-fallstudie`, Szene Absturz

6.8.5 Szenario 5: Mehrere Anwendungen lesen dasselbe Topic

Die Bestandsverwaltung verbucht, das Inkasso rechnet, das Reporting liest nachts. Jede Anwendung hat ihre eigene Gruppe: `beitragsrechner`, `inkasso`, `reporting`. Jede Gruppe hat eigene Offsets und eigene Mitglieder; keine stört die andere. Das Reporting kann eine Woche stehen und holt dann alles nach, solange die Retention reicht.

Frage	Antwort
Eine Gruppe pro was?	pro Anwendung, pro Zweck
Eine Gruppe pro Key oder Partition?	nein; Keys gehören zur Partition, Partitionen zur Gruppe
Was unterscheidet das von einer Queue?	die Nachricht wird nicht verbraucht; jede Gruppe liest sie
Lab	`05-consumer-gruppe.ps1 -Gruppe statistik` neben `beitragsrechner`; in Java `06-consumer-groups -- app.gruppe=statistik`

6.8.6 Szenario 6: Wo liegt V-1001?

Ein Entwickler will wissen, in welcher Partition die Änderungen zu Vertrag V-1001 liegen. Die Antwort kennt der Producer: Er hat beim Senden `murmur2(Key) modulo Partitionsanzahl` gerechnet. Ein Consumer rechnet nie; er liest die Partition aus jeder Nachricht (`ConsumerRecord.partition()`). Wer es vorab wissen will, ruft dieselbe Funktion auf, die der Producer benutzt.

Frage	Antwort
Als Consumer alle Nachrichten zu V-1001?	lesen und nach Key filtern; alle liegen in einer Partition, in Reihenfolge
Vorab wissen, welche Partition?	`BuiltInPartitioner.partitionForKey`, dieselbe Rechnung wie im Producer
Ändert sich das?	ja, wenn die Partitionszahl geändert wird (Kapitel 15)
Lab	`10-partition-fuer-key.ps1` rechnet, `02-producer.ps1` sendet, `03-consumer.ps1` zeigt dieselbe Partition, `12-nachrichten-zu-key.ps1` filtert

6.8.7 Szenario 7: Genau eine Partition lesen

Für die Fehlersuche soll jemand Partition 1 ab Offset 0 lesen, nichts anderes, ohne die Verarbeitung zu stören. Dafür gibt es `assign` statt `subscribe`: Der Consumer nimmt sich die Partition selbst, ohne Gruppe, ohne Commit. In der Shell ist das `--partition` mit `--offset`.

Frage	Antwort
Mit Gruppe möglich?	nein, `--partition` und `--group` schließen sich aus
Stört es die laufende Gruppe?	nein, es gibt keinen Rebalance und keinen Commit
Richtig für	Werkzeuge, Replay einer Partition, Fehlersuche
Falsch für	Verarbeitung, weil niemand übernimmt, wenn der Prozess stirbt
Lab	`11-partition-lesen.ps1 -Partition 1 -Offset 0`; in Java `consumer.assign(List.of(new TopicPartition(topic, 1)))` und `seek`

6.8.8 Szenario 8: Die Pods einer Anwendung lesen alle alles

Das Inkasso läuft mit drei Pods auf OpenShift. Jeder Pod bekommt jede Vertragsänderung, die Last wird nicht verteilt, jede Änderung wird dreimal verbucht. Für Kafka sind die drei Pods drei verschiedene Anwendungen, weil sie nicht dieselbe Gruppe haben. Das ist der häufigste Konfigurationsfehler beim ersten Deployment, und er erzeugt keine Fehlermeldung.

Ursache	Woran man es erkennt	Behebung
kein `group-id` gesetzt, oder der Listener hat eine `id` ohne `idIsGroup = false`	`kafka-consumer-groups.sh --list` zeigt je Pod eine eigene Gruppe, etwa `console-consumer-4711` oder den Namen der Listener-`id`	`spring.kafka.consumer.group-id=inkasso`; bei `@KafkaListener(id = …)` immer `idIsGroup = false`
der Gruppenname enthält etwas Pod-Spezifisches	Gruppen heißen `inkasso-pod-0`, `inkasso-pod-1`	der Name ist in allen Pods identisch; der Pod-Name gehört in `group.instance.id`, nicht in `group-id`
`assign` statt `subscribe`, oder Console-Consumer mit `--partition`	kein Rebalance, `--members` zeigt keine Mitglieder	`subscribe` mit Gruppe; `assign` nur für Werkzeuge

Prüfen, während die Pods laufen:

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --list
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 \
    --describe --group inkasso --members
```

Richtig ist: eine Gruppe `inkasso`, so viele Mitglieder wie Pods, jede Partition genau einmal in der Liste.

6.8.9 Was passiert, sobald die Gruppe stimmt

Mit identischem `group-id` in allen Pods läuft der folgende Ablauf von selbst; nichts davon steht im Anwendungscode.

1. **Start:** Jeder Pod meldet sich beim Gruppen-Koordinator als Mitglied von `inkasso`. Der Koordinator verteilt die Partitionen: sechs Partitionen und zwei Pods ergeben je drei, drei Pods je zwei, sieben Pods lassen einen warten.
2. **Betrieb:** Jeder Pod liest nur seine Partitionen, dort alle Nachrichten in Reihenfolge. Liegt Partition 1 bei Pod B, sieht Pod A die Änderungen zu V-1001 nie. Keine Nachricht erreicht zwei Pods.
3. **Commit:** Jeder Pod committet nach der Verarbeitung seinen Offset je Partition unter dem Gruppennamen. Der Fortschritt gehört der Gruppe, nicht dem Pod.
4. **Pod kommt dazu:** Ein anderer Pod gibt eine oder zwei Partitionen ab; mit `group.protocol=consumer` nur diese, die übrigen laufen weiter. Der neue Pod liest ab dem committeten Offset.
5. **Pod stirbt:** Ein anderer Pod bekommt seine Partitionen, nach dem Session-Timeout (45 Sekunden) oder sofort, wenn der Pod sich sauber abgemeldet hat. Er beginnt beim letzten Commit; was der tote Pod verarbeitet, aber nicht committet hatte, kommt noch einmal. Die Versionsprüfung je Vertrag macht das folgenlos.
6. **Neustart aller Pods:** Für jede Partition geht es genau dort weiter, wo die Gruppe stand. Nichts wird übersprungen, nichts von vorn gelesen.

So sieht der Zustand dann aus, hier mit zwei Pods und `group.instance.id` aus dem Pod-Namen:

```
GROUP    TOPIC      PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  CONSUMER-ID
inkasso  beitraege  0          1830            1830            0    inkasso-
pod-0-…
inkasso  beitraege  1          1795            1801            6    inkasso-
pod-1-…
inkasso  beitraege  2          1811            1811            0    inkasso-
pod-0-…
```

Jede Partition genau einmal, jede mit genau einem Consumer, Lag je Partition. Das ist die Ausgabe von `01-shell/06-gruppe-ansehen -Gruppe inkasso`; die Schritte 4 und 5 zeigen die Zeilen `ZUGEWIESEN` und `ABGEGEBEN` im Log von `06-consumer-groups`.

Für die Anforderung „Reihenfolge je Vertrag, Last verteilt, kein Pod nimmt einem anderen etwas weg“ sind damit die Einstellungen festgelegt:

Einstellung	Wert	Warum
`group-id`	`inkasso`, in allen Pods gleich, nur für diese Anwendung	eine Gruppe je Anwendung
Key	Vertragsnummer	alles zu einem Vertrag in einer Partition, in Reihenfolge
`enable-auto-commit=false`, `ack-mode=record`	Commit nach der Verarbeitung	at least once
Verarbeitung	idempotent über die Versionsnummer	Duplikate nach Rebalance sind folgenlos
`group.protocol`	`consumer`	nur betroffene Partitionen wandern
`group.instance.id`	Pod-Name (StatefulSet)	zurückkehrender Pod behält seine Partitionen, kein Rebalance beim Rolling Update
`terminationGracePeriodSeconds`	größer als `max.poll.interval.ms` plus Verarbeitung	sauberes Abmelden statt 45 Sekunden Stillstand
Threads und Pods	Pods × `concurrency` höchstens Partitionszahl, HPA-Maximum = Partitionszahl	Mitglieder über der Partitionszahl arbeiten nicht

Nicht tun: Pods fest auf Partitionen setzen (`assign`), dann übernimmt niemand beim Ausfall. Nachrichten im Pod an einen Thread-Pool weiterreichen, dann überholen sich Änderungen desselben Vertrags. Eine Gruppe je Vertrag oder Sparte anlegen, die Gruppe ist die Anwendung, nicht der Datensatz.

6.8.10 Die drei Ebenen

Ebene	Wer entscheidet	Wofür
Key → Partition	der Producer, über den Hash des Keys	Reihenfolge je Vertrag
Partition → Mitglied	der Gruppen-Koordinator	Lastverteilung und Ausfallsicherheit
Topic → Gruppe	die Anwendung, ein Name je Zweck	eigener Fortschritt je Anwendung

Die Regel zum Mitnehmen: Keine Gruppe für Werkzeuge und Nachschauen. Eine Gruppe für jede Anwendung, die verarbeitet; ihr Name ist der Name der Anwendung. Mitglieder so viele wie nötig, höchstens so viele wie Partitionen. Welche Partition ein Mitglied bekommt, ist nie Sache der Anwendung.

6.9 Lab: Consumer Groups und Offsets

Drei Fenster im Container: zwei Consumer, ein Producer.

```
./kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic
bestand.vertragsaenderungen --group bestandsverwaltung --command-property
group.protocol=consumer --formatter-property print.key=true --formatter-property
print.partition=true
```

```
./kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic
bestand.vertragsaenderungen --reader-property parse.key=true --reader-property
key.separator=:
```

```
./kafka-groups.sh --bootstrap-server kafka1:9092 --list
```

```
./kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group
bestandsverwaltung --members
```

Ausprobieren:

1. Gruppe `bestandsverwaltung` mit 1, 2 und dann 7 Consumern betreiben – wie ändert sich die Zuordnung? Was macht der siebte?
2. Beide Consumer stoppen, weiter schreiben, den Lag in `--describe` und in Kafka UI ansehen.
3. Eine zweite Gruppe `reporting` starten: Sie liest alles noch einmal, unabhängig von der ersten.
4. Offsets zurücksetzen – auf einen Zeitpunkt und gezielt auf Partition 2 – und die Consumer neu starten.

6.10 Woran sehe ich das im Betrieb?

Die Kennzahlen dieses Kapitels leben an zwei Orten. Broker-Metriken liegen im Broker und sind immer abrufbar. Client-Metriken wie `records-lag-max` liegen im Consumer-Prozess: Sie existieren nur, solange er läuft und sie herausgibt, in der Shell über den JMX-Agenten (`mit-metriken`), in

Spring Boot über Micrometer (Kapitel 14). Deshalb gehört der Lag doppelt überwacht: als Client-Metrik und von außen aus den Commits.

Frage	Metrik	Shell (Container `kafka1`, `/opt/kafka/bin`)	Lab-Skript
Sind alle Repliken aktuell?	`UnderReplicatedPartitions`	`kafka-topics.sh --describe --under-replicated-partitions`; roh: `kafka-jmx.sh --object-name kafka.server:type=ReplicaManager,name=UnderReplicatedPartitions`	`14-monitoring/shell/01-gesundheit`, `08-metriken-ansehen`
Blockieren `acks=all`-Producer?	`UnderMinIsrPartitionCount`	`kafka-topics.sh --describe --under-min-isr-partitions`	dieselben
Flattert die ISR?	`IsrShrinksPerSec`	`kafka-jmx.sh … --name=IsrShrinksPerSec --attributes OneMinuteRate`	`08-metriken-ansehen`
Kommen die Consumer hinterher?	Lag aus den Commits	`kafka-consumer-groups.sh --describe --group <gruppe>` (Spalte LAG), auch ohne laufenden Consumer	`01-shell/06-gruppe-ansehen`
Kommen die Consumer hinterher?	`records-lag-max` (Client)	Shell-Consumer mit `mit-metriken 9211 …`, dann `curl kafka1:9211/metrics`; Spring: `curl localhost:8085/actuator/prometheus`	`14-monitoring`, `08-metriken-ansehen`
Ist die Gruppe stabil?	Status, Rebalances	`kafka-consumer-groups.sh --describe --group <gruppe> --state` (Stable, PreparingRebalance, Empty, Dead) und `--members`; Rebalance-Zähler nur im Client (`rebalance_total`)	`08-metriken-ansehen`

Alle `kafka-jmx.sh`-Aufrufe brauchen `--jmx-url service:jmx:rmi:///jndi/rmi://kafka1:9999/jmxrmi --one-time true`; das Skript `08-metriken-ansehen` setzt das voraus und zeigt alle sechs Zeilen in einem Lauf. Mit `docker stop kafka3` dazwischen sieht man die ersten drei Werte steigen. In Kapitel 14 werden dieselben Metriken zu Dashboard und Alarmen.

6.11 Weiterführende Links

- KIP-848 Neues Consumer-Gruppenprotokoll: https://cwiki.apache.org/confluence/display/KAFKA/KIP-848%3A+The+Next+Generation+of+the+Consumer+Rebalance+Protocol

7 Producer mit Spring Boot

Das Bestandssystem soll jede Vertragsänderung als Ereignis nach Kafka schreiben. Dieses Kapitel baut den Producer mit Spring Boot auf – von der minimalen Anwendung bis zu JSON-Ereignissen, Headern und Topics als Code. Die Beispiele laufen gegen das Lab und verwenden Spring Boot 4.1 mit Spring Kafka 4.1 (Kafka-Client 4.2).

Die Beispiele zu diesem Kapitel

Zwei Projekte im Code-Ordner: `kafka-einfach/producer-app` für die ersten Schritte mit Text-Nachrichten und `labs/07-producer-json` für alles ab dem Abschnitt „Ereignisse als JSON“. Die Codeausschnitte in den Kapiteln 7 bis 16 stammen aus `labs`; jedes Modul dort hat Tests, die mit `./mvnw verify` laufen. Nur die ausdrücklich als Skizze markierten Blöcke (Camel, Avro) sind nicht Teil der Projekte.

7.1 Die minimale Anwendung

Die `producer-app` aus `kafka-einfach.zip` ist bewusst klein: ein Starter, eine Klasse, eine Konfigurationsdatei. Sie liest Zeilen von der Konsole und schreibt sie als Vertragsänderungen in das Topic `vertraege`.

```
producer-app/
├── pom.xml                          ← nur spring-boot-starter-kafka
├── mvnw, mvnw.cmd                   ← Maven Wrapper, kein installiertes Maven nötig
└── src/main/java/de/konitzer/einfach/
    │   └── ProducerApp.java         ← Topic als Code, Senden, Ergebnis auswerten
└── src/main/resources/
    │   └── application.properties   ← alle Kafka-Einstellungen, kommentiert
```

Die einzige Abhängigkeit:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-kafka</artifactId>
</dependency>
```

Der Starter bringt Spring Kafka und den Kafka-Client mit und konfiguriert einen `KafkaTemplate` aus den Einstellungen unter `spring.kafka.*`. Starten im Lab:

```
cd $HOME\Desktop\kafka-einfach\producer-app
java -jar producer-app.jar
```

```
Producer bereit. Topic: vertraege
Eingabe: Vertrag:Änderung, z. B.  V-1001:Adresse geändert   (ohne Doppelpunkt =
ohne Key)

V-1001:Adresse geändert
16:55:06  gesendet key=V-1001   "Adresse geändert" -> Partition 1, Offset 0
```

```
V-1001:Bankverbindung neu
16:55:06  gesendet key=V-1001   "Bankverbindung neu" -> Partition 1, Offset 1
```

Gleiche Vertragsnummer, gleiche Partition, steigende Offsets – das Prinzip aus Kapitel 4, jetzt aus Java.

7.2 Die Konfiguration

```
# mehrere Adressen: Start klappt auch, wenn ein Broker fehlt
spring.kafka.bootstrap-servers=\
    localhost:19092,localhost:19093,localhost:19094

spring.kafka.producer.key-serializer=\
    org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=\
    org.apache.kafka.common.serialization.StringSerializer

# Zuverlässigkeit (Kapitel 5): Default seit Kafka 3.0, hier bewusst
# ausgeschrieben
spring.kafka.producer.acks=all
spring.kafka.producer.properties.enable.idempotence=true
spring.kafka.producer.properties.delivery.timeout.ms=120000

# Batching (Kapitel 15): kurz sammeln
spring.kafka.producer.properties.linger.ms=5

# sprechender Name für Metriken und Broker-Logs
spring.kafka.producer.client-id=bestandssystem
```

Spring Boot kennt für die wichtigsten Kafka-Einstellungen eigene Schlüssel (`spring.kafka.producer.acks`). Alles andere geht über `spring.kafka.producer.properties.<kafka-name>` direkt an den Kafka-Client.

Einstellung	Wert	Warum
`spring.kafka.bootstrap-servers`	mehrere Broker	Einstieg auch bei Ausfall eines Brokers
`spring.kafka.producer.acks`	`all`	auf alle Repliken in der ISR warten
`enable.idempotence`	`true`	keine Duplikate durch Wiederholungen, Reihenfolge bleibt
`delivery.timeout.ms`	120000	so lange wiederholt der Producer, bevor er einen Fehler meldet
`linger.ms`	5–20	kurz sammeln, größere Batches
`spring.kafka.producer.client-id`	Anwendungsname	Zuordnung in Metriken und Logs
`spring.kafka.producer.compression.type`	`lz4` oder `zstd`	weniger Netz und Platte (Kapitel 15)

7.3 Senden mit dem KafkaTemplate

```
@Service
class VertragsaenderungVeroeffentlicher {

    private static final Logger log =
        LoggerFactory.getLogger(VertragsaenderungVeroeffentlicher.class);

    private final KafkaTemplate<String, String> kafka;

    VertragsaenderungVeroeffentlicher(
            KafkaTemplate<String, String> kafka) {
        this.kafka = kafka;
    }

    void veroeffentliche(String vertragsnummer, String aenderung) {
        kafka.send("bestand.vertragsaenderungen", vertragsnummer,
                aenderung)
             .whenComplete((ergebnis, fehler) -> {
                 if (fehler != null) {
                     log.error("Vertrag {} nicht veröffentlicht: {}",
                             vertragsnummer,
                             fehler.getMessage());
                     // hier entscheiden: erneut einplanen, Fehlertabelle,
                             Alarm
                     return;
                 }
                 var meta = ergebnis.getRecordMetadata();
                 log.info("Vertrag {} -> Partition {}, Offset {}",
                         vertragsnummer,
                         meta.partition(), meta.offset());
             });
    }
}
```

`send(topic, key, value)` – der Key ist die Vertragsnummer. Damit landen alle Änderungen eines Vertrags in derselben Partition.

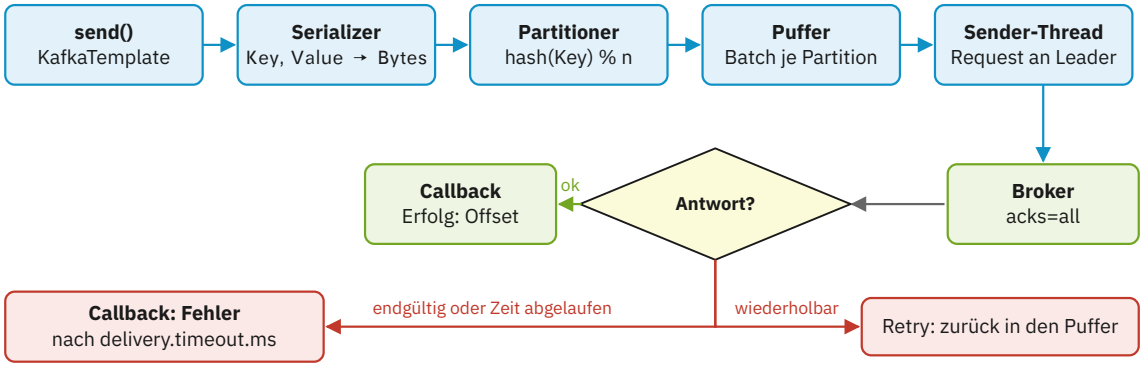


Abbildung 6: Vom Aufruf send() bis zum Callback

7.3.1 Das Ergebnis immer auswerten

`send()` ist asynchron: Die Methode kehrt sofort zurück und liefert einen `CompletableFuture`. Ob die Nachricht wirklich angekommen ist, steht erst im Ergebnis.

Fire and forget verliert Daten still

Wer `kafka.send(…)` aufruft und das Ergebnis ignoriert, bemerkt nie, dass eine Nachricht nicht geschrieben wurde – etwa weil zu wenige Repliken verfügbar waren. `acks=all` schützt nur, wenn jemand auf die Bestätigung schaut.

Im Lab lässt sich das zeigen. Topic strenger stellen, einen Broker stoppen, mit kurzer Wartezeit senden:

```
./kafka-configs.sh --bootstrap-server kafka1:9092 --alter --entity-type topics --
entity-name vertraege --add-config min.insync.replicas=3
```

```
docker stop kafka3
```

```
java -jar producer-app.jar --
spring.kafka.producer.properties.delivery.timeout.ms=35000
```

```
17:00:26 WARN  [Producer clientId=producer-app-1] Got error produce response ...
Error: NOT_ENOUGH_REPLICAS
17:00:27 WARN  [Producer clientId=producer-app-1] Got error produce response ...
Error: NOT_ENOUGH_REPLICAS
17:00:29  FEHLER   key=V-1001   "Test bei fehlender Replik" -> TimeoutException:
Expiring 1 record(s) for vertraege-1:35001 ms has passed since batch creation
```

Erst wiederholt der Producer im Sekundentakt, dann gibt er auf. Danach `docker start kafka3` und `min.insync.replicas` zurück auf 2.

7.3.2 Synchron senden – wenn das Ergebnis vor dem nächsten Schritt feststehen muss

```
SendResult<String, String> ergebnis = kafka.send(topic, vertragsnummer,
        aenderung)
                                           .get(10, TimeUnit.SECONDS);
```

Das blockiert den aufrufenden Thread bis zur Bestätigung. Sinnvoll, wenn etwa ein REST-Aufruf erst antworten darf, wenn die Änderung sicher in Kafka liegt. Der Durchsatz sinkt dabei deutlich, weil kein Batching mehr möglich ist. Der Timeout sollte unter `delivery.timeout.ms` liegen oder bewusst gleich gewählt werden.

7.4 Unter der Haube: Idempotenz mit Producer-ID und Sequenznummer

Warum entstehen mit `enable.idempotence=true` keine Duplikate, obwohl der Producer wiederholt?

1. Beim Start bekommt der Producer vom Broker eine **Producer-ID** (PID) und eine Epoche.
2. Jeder Batch trägt PID, Epoche und eine **Sequenznummer**, die je Partition fortlaufend zählt.
3. Der Leader merkt sich für jede PID die Sequenznummern der letzten fünf Batches je Partition. Kommt ein Batch mit einer bekannten Nummer erneut – weil die Bestätigung unterwegs verloren ging –, bestätigt er ihn, ohne ihn noch einmal zu schreiben.
4. Kommt ein Batch mit einer Lücke in der Nummerierung, lehnt der Leader ihn ab. So bleibt auch mit bis zu fünf parallelen Requests die Reihenfolge erhalten.

Die Grenze: Die PID gilt nur für eine Producer-Sitzung. Startet das Bestandssystem neu und sendet eine Änderung noch einmal, bekommt es eine neue PID – für den Broker ist das eine neue Nachricht. Gegen solche Duplikate helfen nur die Versionsnummer im Ereignis und der idempotente Consumer.

7.5 Topics als Code

Das Lab legt keine Topics automatisch an. Eine Anwendung kann ihre Topics aber selbst beschreiben – Spring Boot legt beim Start an, was fehlt:

```
@Bean
NewTopic vertragsaenderungen() {
    return TopicBuilder.name("bestand.vertragsaenderungen")
            .partitions(6)
            .replicas(3)
            .config("min.insync.replicas", "2")
            .config("retention.ms",
                    String.valueOf(Duration.ofDays(14).toMillis()))
            .build();
}
```

Der `KafkaAdmin` aus der Auto-Konfiguration prüft beim Start jede `NewTopic`-Bean. Existiert das Topic schon, ändert er nichts an Partitionen oder Replikation – nur fehlende Topics werden angelegt.

Wer legt Topics an?

In Produktion ist meist das Plattformteam zuständig, und Topics entstehen über eine Pipeline (Kapitel 17). Dann braucht die Anwendung Leserechte auf die Topic-Beschreibung, aber kein Recht zum Anlegen. `NewTopic`-Beans sind gut für Entwicklung, Tests und Teams, die ihre Topics selbst verantworten.

7.6 Ereignisse als JSON

Bisher waren die Werte einfacher Text. In der Praxis sind Vertragsänderungen strukturiert. Ein Java-Record beschreibt das Ereignis:

```
public record Vertragsaenderung(
        String vertragsnummer,
        // fortlaufend je Vertrag, für idempotente Verarbeitung
        int version,
        // ADRESSE, BEITRAG, BANKVERBINDUNG, KUENDIGUNG …
        String art,
        Map<String, String> daten,
        Instant zeitpunkt) {
}
```

Das Record liegt im Modul `ereignisse` – es ist der Vertrag zwischen Producer und Consumer und wird von beiden Seiten eingebunden. Spring Kafka 4 serialisiert mit Jackson 3; dafür braucht das Projekt neben dem Kafka-Starter den Jackson-Starter:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jackson</artifactId>
</dependency>
```

```
spring.kafka.producer.value-serializer=\
    org.springframework.kafka.support.serializer.JacksonJsonSerializer
# keine Java-Klassennamen in die Header schreiben –
# Consumer in anderen Sprachen brauchen sie nicht
spring.kafka.producer.properties.spring.json.add.type.headers=false
```

Der Service aus `07-producer-json` sendet das Record und wertet das Ergebnis aus:

```
@Service
public class Bestandssystem {

    private final KafkaTemplate<String, Vertragsaenderung> kafka;
    private final String topic;

    Bestandssystem(KafkaTemplate<String, Vertragsaenderung> kafka,
                   @Value("${app.topic}") String topic) {
        this.kafka = kafka;
```

```
        this.topic = topic;
    }

    public CompletableFuture<SendResult<String, Vertragsaenderung>>
            veroeffentliche(Vertragsaenderung aenderung) {
        return kafka.send(topic, aenderung.vertragsnummer(), aenderung)
                .whenComplete((ergebnis, fehler) -> {
                    if (fehler != null) {
                        log.error("Vertrag {} v{} nicht gesendet: {}",
                                aenderung.vertragsnummer(),
                                aenderung.version(), ursache(fehler));
                        return;
                    }
                    var meta = ergebnis.getRecordMetadata();
                    log.info("Vertrag {} v{} -> Partition {}, Offset {}",
                            aenderung.vertragsnummer(),
                            aenderung.version(),
                            meta.partition(), meta.offset());
                });
    }

    public CompletableFuture<SendResult<String, Vertragsaenderung>>
            adresseGeaendert(String vertrag, int version, String plz) {
        return veroeffentliche(new Vertragsaenderung(vertrag, version,
                "ADRESSE", Map.of("plz", plz), Instant.now()));
    }
}
```

Auf dem Topic liegt dann:

```
Key:   V-1001
Value: {"vertragsnummer":"V-1001","version":3,"art":"ADRESSE","daten":
{"plz":"50667"},"zeitpunkt":"2026-11-03T09:14:02Z"}
```

Der Integrationstest `BestandssystemIT` liest genau das mit einem rohen `KafkaConsumer` zurück und prüft, dass kein `__TypeId__`-Header dabei ist (Kapitel 13).

Wie man JSON-Ereignisse gegen Formatbrüche absichert und wann Avro die bessere Wahl ist, zeigt Kapitel 10.

7.7 Header

Metadaten gehören in Header, nicht in den Wert:

```
ProducerRecord<String, Vertragsaenderung> nachricht =
        new ProducerRecord<>("bestand.vertragsaenderungen", vertrag,
                ereignis);
nachricht.headers().add("quelle",
        "bestandssystem".getBytes(StandardCharsets.UTF_8));
```

```
nachricht.headers().add("korrelations-id",
        korrelationsId.getBytes(StandardCharsets.UTF_8));
kafka.send(nachricht);
```

Header beeinflussen die Partition nicht. Ist Micrometer Tracing aktiv (Kapitel 14), schreibt Spring den Trace-Kontext (`traceparent`) automatisch als Header.

7.8 Producer-Ereignisse zentral protokollieren

Statt in jedem Aufruf das Ergebnis zu protokollieren, kann ein `ProducerListener` alle Erfolge und Fehler eines Templates sehen:

```
@Bean
ProducerListener<Object, Object> producerProtokoll() {
    return new ProducerListener<>() {
        @Override
        public void onError(ProducerRecord<Object, Object> nachricht,
                RecordMetadata meta, Exception fehler) {
            log.error("Senden fehlgeschlagen: Topic {}, Key {}: {}",
                    nachricht.topic(), nachricht.key(),
                            fehler.getMessage());
        }
    };
}
```

Der Listener ersetzt nicht die fachliche Reaktion auf einen Fehler – er sorgt nur dafür, dass kein Fehler unprotokolliert bleibt.

7.9 Mit welchem Broker spricht der Producer?

Mit dem Leader jeder Partition, in die er schreibt, und die Leader liegen über alle Broker verteilt. Nur mit einem Broker zu sprechen ist weder sinnvoll noch möglich. `bootstrap.servers` ist der erste Kontakt: Der Producer holt dort die Metadaten (welche Partition hat welchen Leader), baut Verbindungen zu allen betroffenen Brokern auf und schickt jedes Batch direkt an den richtigen Leader. Ein Broker, der etwas für eine fremde Partition bekommt, antwortet mit `NOT_LEADER_OR_FOLLOWER`, der Client holt neue Metadaten und wiederholt.

Daraus folgen zwei Regeln. Erstens: In `bootstrap.servers` **mehrere Broker** eintragen, am besten drei bis fünf oder einen DNS-Namen mit mehreren Adressen. Mit nur einem Eintrag startet die Anwendung nicht, wenn genau dieser Broker gerade neu startet, obwohl der Cluster läuft. Nach dem ersten Kontakt braucht der Client die Liste nicht mehr. Zweitens: Nach einem Broker-Ausfall ohne anschließende Preferred Leader Election liegen alle Leader auf den übrigen Brokern, und der Producer spricht tatsächlich nur noch mit ihnen. Das ist der Grund für `kafka-leader-election.sh` (Kapitel 14).

Änderungen am Cluster zur Laufzeit erledigt der Client allein: Er frischt die Metadaten alle `metadata.max.age.ms` (fünf Minuten) auf und sofort bei jedem Fehler. Neue Broker, Leader-Wechsel, ausgefallene Broker: kein Code, kein Neustart. Im Lab: `04-producer-spring --app.automatisch=true` laufen lassen, `docker stop kafka3`, `docker start kafka3`. Die App sendet durchgehend, im Log höchstens ein paar `WARN … retrying`.

7.10 Konfiguration gehört nicht ins JAR

Die Broker-Adressen stehen nicht in der `pom.xml` (die bringt nur die Bibliothek) und nicht fest im Code, sondern dort, wo sie sich je Umgebung ändern lassen. Spring Boot liest alle diese Quellen, die weiter unten stehenden gewinnen:

Quelle	Wann
`application.properties` im JAR	Default für die Entwicklung, im Lab `localhost:19092,19093,19094`
Profil `application-prod.properties` oder Config-Server	wenn im Haus üblich
Umgebungsvariable `SPRING_KAFKA_BOOTSTRAP_SERVERS`	Produktion, Container, Kubernetes
Kommandozeile `--spring.kafka.bootstrap-servers=…`	Test, Vorführung, DR-Umschaltung

Eine geänderte Bootstrap-Liste übernimmt ein laufender Client nicht; ein Wechsel ist ein Neustart ohne Build. Ohne Spring gilt dieselbe Idee von Hand: `System.getProperty("bootstrap", "…")`, überschrieben mit `-Dbootstrap=…` (`02-producer-plain`). Stabiler als Broker-Namen ist ein DNS-Eintrag wie `kafka.intern` mit mehreren A-Records (`client.dns.lookup=use_all_dns_ips` ist seit 2.6 Default) oder ein Load Balancer nur für den Bootstrap; die `advertised.listeners` der Broker müssen von der Anwendung aus erreichbar bleiben, denn der Datenverkehr geht danach direkt zu den Leadern. In Kubernetes mit Strimzi bleibt der Bootstrap-Service gleich, die Broker-Pods darunter wechseln.

7.11 Was bei einem Absturz des Producers passiert

Situation	Folge	Lösung
Netzwerkfehler, Leader-Wechsel	Producer wiederholt automatisch	idempotenter Producer: keine Duplikate, keine Umsortierung
Absturz vor der Bestätigung	Nachricht gilt als nicht geschrieben	Quelle muss erneut senden → Outbox-Muster (Kapitel 9)
Absturz nach der Bestätigung	Nachricht ist sicher; ein erneutes Senden erzeugt ein Duplikat	Consumer verarbeitet idempotent (Kapitel 8 und 18)
Datenbank geschrieben, Senden scheitert	Bestand und Ereignisse laufen auseinander	Outbox oder Kafka-Transaktion (Kapitel 9)

Wie der Fehler bei nicht erreichbarem Broker ankommt, zeigt der Test `BrokerNichtErreichbarIT`: Entweder wirft `send()` sofort eine `KafkaException`, wenn der Producer nach `max.block.ms` (Standard 60 s) keine Metadaten bekommen hat – oder die Zukunft scheitert nach `delivery.timeout.ms`. In beiden Fällen ist die Ursache eine `TimeoutException`, und in beiden Fällen braucht der aufrufende Code eine Reaktion.

7.12 Unterschiede zu Spring Boot 3.x

Die Beispiele sind für Spring Boot 4.1 geschrieben. Auf Spring Boot 3.x heißen einige Dinge anders:

Spring Boot 4.1 / Spring Kafka 4.1	Spring Boot 3.x / Spring Kafka 3.x
`JacksonJsonSerializer`, `JacksonJsonDeserializer` (Jackson 3, Package `tools.jackson`)	`JsonSerializer`, `JsonDeserializer` (Jackson 2)
`spring-boot-starter-jackson`	`spring-boot-starter-json`
`ExponentialBackOff` mit `setMaxAttempts()` (Kapitel 9)	`ExponentialBackOffWithMaxRetries`
`@RetryableTopic(backOff = @BackOff(delayString = …))` (Kapitel 9)	`backoff = @Backoff(delayExpression = …)` aus Spring Retry
Testcontainers 2: `testcontainers-kafka`, `org.testcontainers.kafka.KafkaContainer` (Kapitel 13)	Testcontainers 1: `kafka`, `org.testcontainers.containers.KafkaContainer`

7.13 Zusammenfassung

- Ein Starter, ein `KafkaTemplate`, eine Handvoll Einstellungen – mehr braucht ein Producer nicht.
- Key = Vertragsnummer. Topics als Code mit `NewTopic`, wenn das Team sie verantwortet.
- `acks=all` und Idempotenz bleiben an; das Ergebnis von `send()` wird **immer** ausgewertet.
- Strukturierte Ereignisse als Records mit `JacksonJsonSerializer`; Metadaten in Header.

7.14 Weiterführende Links

- Spring for Apache Kafka, Referenz: https://docs.spring.io/spring-kafka/reference/
- Spring Boot, Kafka-Unterstützung: https://docs.spring.io/spring-boot/reference/messaging/kafka.html
- Kafka-Client (Producer API, Konfiguration): https://kafka.apache.org/documentation/#producerapi und https://kafka.apache.org/documentation/#producerconfigs
- Javadoc `kafka-clients` 4.x: https://kafka.apache.org/44/javadoc/index.html
- Spring Boot, `spring.kafka.*`-Properties: https://docs.spring.io/spring-boot/appendix/application-properties/index.html#appendix.application-properties.integration
- Apache Camel, Kafka-Komponente: https://camel.apache.org/components/next/kafka-component.html
- Camel Spring Boot Starter für Kafka: https://camel.apache.org/camel-spring-boot/next/kafka-starter.html

8 Consumer mit Spring Boot

Die Bestandsverwaltung liest Vertragsänderungen, verbucht sie und darf dabei weder Änderungen verlieren noch ihre Reihenfolge vertauschen. Dieses Kapitel zeigt, wie ein Consumer in Spring Boot aufgebaut ist, wann er committet, wie er skaliert und wie er Duplikate verkraftet.

Die Beispiele zu diesem Kapitel

`kafka-einfach/consumer-app` für die Konsolenausgaben der ersten Abschnitte, `labs/08-consumer-json` ab „Idempotent verarbeiten“. Die Tests zu diesem Modul liegen in `labs/13-tests` (Kapitel 13).

8.1 Die minimale Anwendung

Die `consumer-app` aus `kafka-einfach.zip` liest das Topic `vertraege` und zeigt auf der Konsole jeden Schritt: Zuteilung von Partitionen, Empfang, Verarbeitung, Commit.

```
cd $HOME\Desktop\kafka-einfach\consumer-app
java -jar consumer-app.jar
```

```
Consumer gestartet. Topic: vertraege, Gruppe: vertragsverarbeitung – wartet auf
die Zuteilung von Partitionen …

16:57:41  ZUGEWIESEN: vertraege-0, vertraege-1, vertraege-2   -> liest jetzt:
vertraege-0, vertraege-1, vertraege-2

16:55:13  empfangen  key=V-1001   "Adresse geändert"  (Partition 1, Offset 0)
16:55:13  committet Partition 1: nächster Offset 1
16:55:13  empfangen  key=V-1001   "Bankverbindung neu"  (Partition 1, Offset 1)
16:55:13  committet Partition 1: nächster Offset 2
```

8.2 Der Listener

```
@Component
class Vertragsverarbeitung {

    @KafkaListener(topics = "vertraege")
    void verarbeite(ConsumerRecord<String, String> nachricht,
            Acknowledgment commit) {
        System.out.printf("empfangen key=%s \"%s\" (P %d, Offset %d)%n",
                nachricht.key(), nachricht.value(), nachricht.partition(),
                        nachricht.offset());

        // fachliche Verarbeitung
        verbuche(nachricht.key(), nachricht.value());

        // erst danach: Offset nach __consumer_offsets
        commit.acknowledge();
```

```
    }
}
```

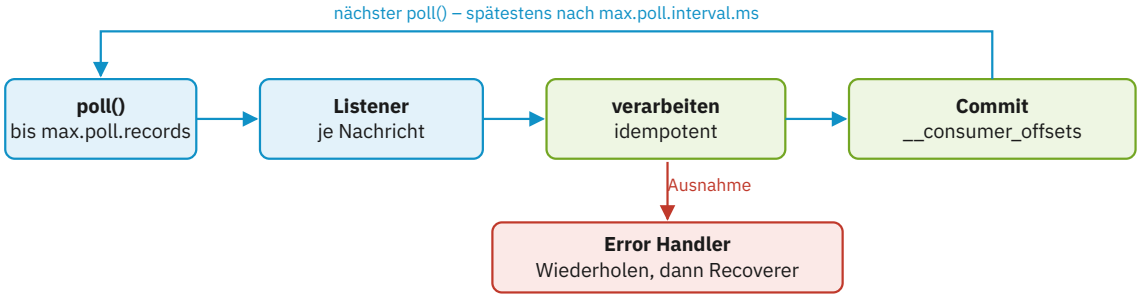


Abbildung 7: Die Poll-Schleife: holen, verarbeiten, committen

`@KafkaListener` erzeugt einen Listener-Container: Er ruft `poll()` in einer Schleife auf, übergibt jede Nachricht an die Methode und kümmert sich um Commits, Rebalances und Fehler. Die Methode kann den ganzen `ConsumerRecord` bekommen oder nur Teile davon:

```
@KafkaListener(topics = "bestand.vertragsaenderungen")
void verarbeite(@Payload Vertragsaenderung aenderung,
                @Header(KafkaHeaders.RECEIVED_KEY) String vertragsnummer,
                @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
                @Header(KafkaHeaders.OFFSET) long offset) {
    // …
}
```

8.3 Die Konfiguration

```
spring.kafka.bootstrap-servers=\
    localhost:19092,localhost:19093,localhost:19094

spring.kafka.consumer.key-deserializer=\
    org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer=\
    org.apache.kafka.common.serialization.StringDeserializer

# Instanzen derselben Gruppe teilen sich die Partitionen
spring.kafka.consumer.group-id=vertragsverarbeitung
# Startposition, solange die Gruppe keinen gespeicherten Offset hat
spring.kafka.consumer.auto-offset-reset=earliest
# selbst committen, erst nach der Verarbeitung
spring.kafka.consumer.enable-auto-commit=false
spring.kafka.listener.ack-mode=manual_immediate
# kleine Pakete bei langsamer Verarbeitung
spring.kafka.consumer.max-poll-records=10
# neues Gruppenprotokoll (KIP-848)
spring.kafka.consumer.properties.group.protocol=consumer
```

```
# Threads je Instanz – höchstens so viele wie Partitionen
spring.kafka.listener.concurrency=1
spring.kafka.consumer.client-id=vertragsverarbeitung
```

Einstellung	Wert	Warum
`group-id`	Name der Anwendung	Instanzen derselben Gruppe teilen sich die Partitionen; jede andere Anwendung bekommt eine eigene Gruppe
`auto-offset-reset`	`earliest`	eine neue Gruppe liest den Bestand von vorn; `latest` überspringt still
`enable-auto-commit`	`false`	Commit erst nach der Verarbeitung
`listener.ack-mode`	`manual_immediate`, `record` oder `batch`	wann der Container committet (siehe unten)
`max-poll-records`	10–100	Arbeit pro `poll()` muss sicher in `max.poll.interval.ms` passen
`group.protocol`	`consumer`	ruhigere Rebalances
`listener.concurrency`	≤ Partitionen	Threads in dieser Instanz

8.4 Wann committet wird

Spring Kafka schaltet Auto-Commit des Clients standardmäßig ab und committet selbst. Den Zeitpunkt bestimmt der `ack-mode`:

`ack-mode`	Commit	Typischer Einsatz
`batch` (Standard)	nach allen Nachrichten eines `poll()`	guter Kompromiss; bei Absturz höchstens ein Batch doppelt
`record`	nach jeder einzelnen Nachricht	wenige, teure Nachrichten
`time`, `count`, `count_time`	nach Zeit oder Anzahl	hoher Durchsatz
`manual`	`acknowledge()` im Code, gesammelt beim nächsten `poll()`	Commit hängt an fachlicher Bedingung
`manual_immediate`	`acknowledge()` im Code, sofort	Demonstration, sehr seltene Nachrichten

In allen Fällen gilt: **erst verarbeiten, dann committen.** Stürzt die Instanz zwischen beiden Schritten ab, kommt die Nachricht nach dem Neustart noch einmal – at-least-once.

8.4.1 Vorführung: Weitermachen nach einem Neustart

1. consumer-app laufen lassen, im Producer einige Änderungen senden.
2. consumer-app mit Strg+C beenden, im Producer drei weitere Änderungen senden.
3. Rückstand im Container ansehen:

```
./kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group
vertragsverarbeitung
```

`LAG` steht bei 3.

4. consumer-app neu starten: Sie liefert genau die drei verpassten Änderungen und meldet je „committet … nächster Offset“. `LAG` ist wieder 0.

8.5 Skalieren: Instanzen und Threads

Jede Instanz der Anwendung ist ein Mitglied der Consumer Group. Startet man die consumer-app ein zweites und drittes Mal, verteilt Kafka die Partitionen neu:

```
Instanz 1:
16:56:50  ZUGEWIESEN: vertraege-0, vertraege-1, vertraege-2   -> liest jetzt:
vertraege-0, vertraege-1, vertraege-2
16:57:01  ABGEGEBEN:  vertraege-2   -> liest jetzt: vertraege-0, vertraege-1
16:57:07  ABGEGEBEN:  vertraege-1   -> liest jetzt: vertraege-0

Instanz 2:
16:57:02  ZUGEWIESEN: vertraege-2   -> liest jetzt: vertraege-2

Instanz 3:
16:57:14  ZUGEWIESEN: vertraege-1   -> liest jetzt: vertraege-1
```

Mit dem neuen Gruppenprotokoll gibt Instanz 1 nur die Partition ab, die wechselt – die anderen verarbeitet sie ohne Unterbrechung weiter. Eine vierte Instanz bekommt bei drei Partitionen nichts und wartet als Reserve.

Innerhalb einer Instanz lässt `spring.kafka.listener.concurrency` mehrere Threads laufen. Jeder Thread ist ein eigener Consumer in derselben Gruppe:

Partitionen	Instanzen × `concurrency`	Ergebnis
6	1 × 3	3 Threads mit je 2 Partitionen
6	2 × 3	6 Threads mit je 1 Partition – Maximum
6	3 × 3	6 Threads arbeiten, 3 warten

Nie an einen Thread-Pool weiterreichen

Wer Nachrichten aus dem Listener an einen `ExecutorService` übergibt, um schneller zu sein, verliert die Reihenfolge pro Vertrag: Die zweite Änderung an V-1001 kann vor der ersten fertig sein. Außerdem committet der Container dann Offsets von Nachrichten, die noch nicht verarbeitet sind. Parallelität entsteht über Partitionen, Instanzen und `concurrency` – nicht im Listener.

8.6 Rebalances beobachten

Ein Listener-Bean kann `ConsumerSeekAware` implementieren und wird dann bei jeder Zuteilung und Abgabe benachrichtigt:

```
@Component
class Vertragsverarbeitung implements ConsumerSeekAware {

    @Override
    public void onPartitionsAssigned(
            Map<TopicPartition, Long> partitionen,
            ConsumerSeekCallback callback) {
        log.info("Zugewiesen: {}", partitionen.keySet());
    }

    @Override
    public void onPartitionsRevoked(Collection<TopicPartition> p) {
        log.info("Abgegeben: {}", p);
    }
}
```

Mit dem `ConsumerSeekCallback` kann eine Anwendung auch gezielt zurückspulen, etwa nach einer Fehlerkorrektur: `callback.seek("bestand.vertragsaenderungen", 2, 1500)`.

8.7 Idempotent verarbeiten

At-least-once heißt: Duplikate kommen vor – nach einem Absturz, nach einem Rebalance oder weil die Quelle eine Änderung zweimal gesendet hat. Die Verarbeitung muss sie erkennen. Das Modul `08-consumer-json` führt dafür eine Versionsnummer je Vertrag – und zwar in einer Klasse **ohne Kafka**, damit sie sich mit einem einfachen Unit-Test prüfen lässt:

```
@Component
public class Verbuchung {

    public enum Ergebnis { VERBUCHT, DUPLIKAT, LUECKE }

    private final Bestand bestand;   // letzte Version je Vertrag

    public Ergebnis verarbeite(Vertragsaenderung aenderung) {
        int letzte = bestand.letzteVersion(aenderung.vertragsnummer());
        if (aenderung.version() <= letzte) {
            duplikate.incrementAndGet();
            return Ergebnis.DUPLIKAT;
        }
        Ergebnis ergebnis = Ergebnis.VERBUCHT;
        if (aenderung.version() > letzte + 1) {
            luecken.incrementAndGet();      // eine frühere Änderung fehlt
            ergebnis = Ergebnis.LUECKE;
        }
        // hier: Adresse, Beitrag, Bankverbindung übernehmen
        bestand.speichere(aenderung.vertragsnummer(),
                aenderung.version());
        verbucht.incrementAndGet();
```

```
            return ergebnis;
        }
    }
```

Der Listener bleibt dünn: Er nimmt das Record entgegen, ruft die Verbuchung auf und committet danach.

```
@KafkaListener(id = "bestand", idIsGroup = false, topics = "${app.topic}")
void verarbeite(@Payload Vertragsaenderung aenderung,
                @Header(KafkaHeaders.RECEIVED_KEY) String vertragsnummer,
                @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
                @Header(KafkaHeaders.OFFSET) long offset,
                Acknowledgment commit) {
    Verbuchung.Ergebnis ergebnis = verbuchung.verarbeite(aenderung);
    log.info("P{} @{}  {} v{} {}  -> {}", partition, offset,
            vertragsnummer, aenderung.version(), aenderung.art(),
            ergebnis);

    commit.acknowledge();   // erst nach der Verarbeitung
}
```

Die `bestand-demo` (Kapitel 18) macht dasselbe mit Text-Werten der Form `"<version>|<änderung>"` und speichert die Versionen in Dateien, damit sie einen Absturz überleben.

Muster	Wie	Passt für
Versionsprüfung	Nur anwenden, wenn Version > gespeicherte Version	Änderungen an einem Fachobjekt (Vertrag, Schaden)
Upsert	`INSERT … ON CONFLICT UPDATE` mit fachlichem Schlüssel	Zustände, die überschrieben werden dürfen
Dedupe-Tabelle	Ereignis-ID speichern, bei Treffer überspringen	Ereignisse ohne natürliche Version (Zahlungseingang)

Wichtig: Speichern der Version und fachliche Änderung gehören in dieselbe Datenbank-Transaktion.

8.8 Mehrere Gruppen, ein Topic

```
java -jar consumer-app.jar --spring.kafka.consumer.group-id=reporting
```

Die Gruppe `reporting` hat noch keinen Offset und liest wegen `auto-offset-reset=earliest` alles ab Offset 0 – unabhängig von der Gruppe `vertragsverarbeitung`. So bekommen Inkasso, Kundenportal und Reporting dieselben Vertragsänderungen, ohne sich abzustimmen.

8.9 JSON lesen

Zum JSON-Producer aus Kapitel 7 passt:

```
spring.kafka.consumer.value-deserializer=\
    org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
spring.kafka.consumer.properties.spring.deserializer.value.delegate.class=\
    org.springframework.kafka.support.serializer.JacksonJsonDeserializer
spring.kafka.consumer.properties.spring.json.value.default.type=\
    de.konitzer.beispiele.ereignisse.Vertragsaenderung
spring.kafka.consumer.properties.spring.json.use.type.headers=false
spring.kafka.consumer.properties.spring.json.trusted.packages=\
    de.konitzer.beispiele.ereignisse
```

Der `ErrorHandlingDeserializer` fängt Nachrichten ab, die sich nicht deserialisieren lassen. Ohne ihn würde eine einzige kaputte Nachricht den Consumer bei jedem `poll()` erneut scheitern lassen – die Partition stünde still. Mit ihm landet der Fehler beim Error Handler. Im Modul `08-consumer-json` protokolliert der nur und überspringt die Nachricht:

```
@Bean
CommonErrorHandler errorHandler() {
    return new DefaultErrorHandler((datensatz, fehler) ->
            log.warn("ÜBERSPRUNGEN: P{} @{} key={}: {}",
                    datensatz.partition(), datensatz.offset(),
                    datensatz.key(), ursache(fehler)),
            new FixedBackOff(0L, 0L));   // keine Wiederholung
}
```

Der Test `BestandsverwaltungIT` schickt `"das ist kein JSON"` und direkt danach eine gültige Änderung – die gültige wird verbucht. Wiederholen, anhalten und Dead-Letter-Topic: Kapitel 9.

Bean-Namen

Eine `@Configuration`-Klasse `Fehlerbehandlung` und eine `@Bean`-Methode `fehlerbehandlung()` ergeben denselben Bean-Namen – Spring Boot bricht dann mit `BeanDefinitionOverrideException` ab. Methoden daher anders benennen als die Klasse.

8.10 Den Listener steuern

Listener lassen sich zur Laufzeit anhalten und starten, etwa für Wartung oder wenn ein nachgelagertes System ausfällt:

```
@Autowired KafkaListenerEndpointRegistry registry;

void bestandsverwaltungAnhalten() {
    // poll() läuft weiter, liefert aber nichts
    registry.getListenerContainer("bestand").pause();
}

void bestandsverwaltungFortsetzen() {
    registry.getListenerContainer("bestand").resume();
}
```

`pause()` hält die Partitionen und vermeidet einen Rebalance; `stop()` verlässt die Gruppe. Die ID kommt aus `@KafkaListener(id = "bestand", idIsGroup = false)`.

8.11 Reihenfolge innerhalb einer Partition ändern?

Kafka liefert immer in Offset-Reihenfolge; ändern lässt sich das im Broker nicht. Braucht die Anwendung eine andere Reihenfolge – etwa nach dem fachlichen Gültigkeitsdatum einer Änderung –, sortiert sie selbst: in einem Puffer nach Zeitstempel oder Versionsnummer, in Kafka Streams über Fenster. Die Versionsprüfung aus diesem Kapitel erkennt zumindest, wenn eine Änderung zu früh kommt.

8.12 Alternative: Apache Camel

Wer Integrationsrouten konfigurierbar halten will, kann statt `@KafkaListener` die Kafka-Komponente von Apache Camel nutzen. Der folgende Block ist eine **Skizze** und nicht Teil der Beispielprojekte:

```
@Component
class BestandRoute extends RouteBuilder {
    @Override
    public void configure() {
        from("kafka:bestand.vertragsaenderungen?groupId=bestandsverwaltung"
                + "&autoCommitEnable=false"
                + "&allowManualCommit=true&maxPollRecords=10")
            .routeId("bestand")
            .bean(Verbuchung.class, "verarbeite")
            .process(austausch -> austausch.getIn()
                    .getHeader(KafkaConstants.MANUAL_COMMIT,
                            KafkaManualCommit.class).commit());
    }
}
```

Es gelten dieselben Regeln: Commit nach der Verarbeitung, idempotente Verarbeitung, keine parallele Verarbeitung innerhalb einer Partition. Die Optionen der Endpunkt-URI entsprechen den Kafka-Einstellungen; Camel unterstützt ebenfalls Tracing über OpenTelemetry.

8.13 Zusammenfassung

- `@KafkaListener` + Container: Poll-Schleife, Commits und Rebalances übernimmt Spring.
- Erst verarbeiten, dann committen (`ack-mode`); Auto-Commit des Clients bleibt aus.
- Skalieren über Partitionen, Instanzen und `concurrency` – nie über einen Thread-Pool im Listener.
- Duplikate gehören dazu: idempotent verarbeiten, z. B. mit Versionsnummer je Vertrag.
- `ErrorHandlingDeserializer` immer vor einen Deserializer für strukturierte Daten schalten.

8.14 Weiterführende Links

- Spring for Apache Kafka, Receiving Messages: https://docs.spring.io/spring-kafka/reference/kafka/receiving-messages.html

- Kafka-Client (Consumer API, Konfiguration): https://kafka.apache.org/documentation/#consumerapi und https://kafka.apache.org/documentation/#consumerconfigs
- Spring for Apache Kafka, Fehlerbehandlung im Container: https://docs.spring.io/spring-kafka/reference/kafka/annotation-error-handling.html
- Apache Camel, Kafka-Komponente (Consumer-Optionen, manuelles Commit, Idempotent Consumer): https://camel.apache.org/components/next/kafka-component.html und https://camel.apache.org/components/next/eips/idempotentConsumer-eip.html
- Camel, Kafka-Beispiele: https://github.com/apache/camel-spring-boot-examples/tree/main/kafka

9 Fehlerbehandlung und Zustellsicherheit

„Wie stelle ich sicher, dass eine Vertragsänderung ankommt – genau einmal und in der richtigen Reihenfolge?“ Die Antwort ist keine einzelne Einstellung, sondern eine Kette von der Datenbank des Bestandssystems bis zur Verbuchung. Dieses Kapitel geht die Glieder durch und zeigt, wie man in Spring Boot mit Nachrichten umgeht, die nicht verarbeitet werden können.

Die Beispiele zu diesem Kapitel

Drei Module in `labs`, jedes mit Tests: `09a-fehlerbehandlung` (Poison Pill, anhalten oder DLT, `@RetryableTopic`), `09b-outbox` (Transactional Outbox mit JPA und H2) und `09c-transaktionen` (Kafka-Transaktionen, `read_committed`).

9.1 Zustellsicherheit ist eine Kette

Glied	Risiko	Lösung in Spring Boot
Anwendung → Producer	`send()` ohne Ergebnisprüfung	Ergebnis mit `whenComplete(…)` auswerten (Kapitel 7)
Datenbank + Ereignis	Datenbank geschrieben, Ereignis nicht – oder umgekehrt	Transactional Outbox
Broker	Kopie verloren	RF 3, `min.insync.replicas=2`, `acks=all` (Kapitel 5)
Consumer	Commit vor der Verarbeitung	Auto-Commit aus, Commit nach der Verarbeitung (Kapitel 8)
Verarbeitung	Duplikate nach Absturz	idempotent: Versionsprüfung, Upsert
Fehlerhafte Nachricht	Endlosschleife oder Stillstand	`DefaultErrorHandler` mit Backoff, dann anhalten oder Dead-Letter-Topic
Kafka → Kafka	Lesen, Verarbeiten, Schreiben doppelt	Kafka-Transaktionen, `read_committed`

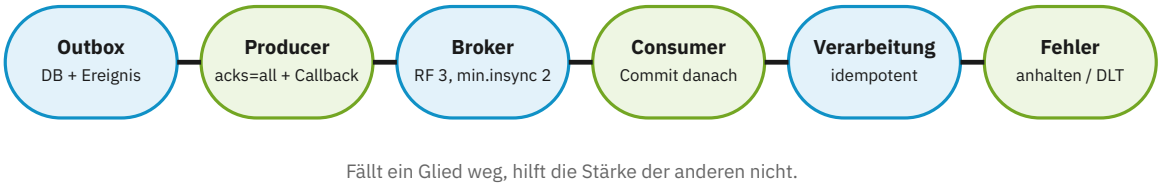


Abbildung 8: Die Glieder der Zustellsicherheit

Fällt ein Glied weg, hilft die Stärke der anderen nicht. `acks=all` nützt nichts, wenn die Anwendung den Fehler beim Senden ignoriert; ein idempotenter Producer nützt nichts, wenn die Datenbank-Änderung ohne Ereignis committet wurde.

9.2 Wenn eine Nachricht nicht verarbeitet werden kann

Eine Vertragsänderung verweist auf einen Tarif, den die Bestandsverwaltung nicht kennt. Jeder Versuch scheitert – eine **Poison Pill**. Spring Kafka übergibt die Ausnahme aus dem Listener an den Error Handler des Containers. Der Standard ist der `DefaultErrorHandler`:

1. Er setzt den Consumer auf die fehlerhafte Nachricht zurück (`seek`).
2. Er wartet nach einer Backoff-Strategie und versucht es erneut – **blockierend**: Die Partition wartet.
3. Sind die Versuche aufgebraucht, ruft er einen **Recoverer** auf, der entscheidet, was mit der Nachricht passiert.

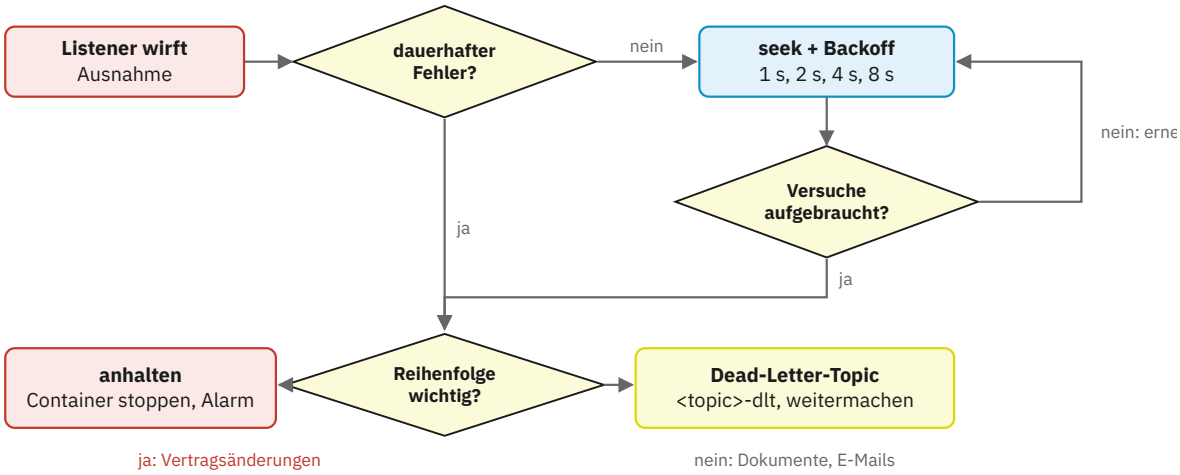


Abbildung 9: Wiederholen, dann anhalten oder ins Dead-Letter-Topic

Ohne eigenen Recoverer protokolliert er den Fehler und macht mit der nächsten Nachricht weiter – die fehlerhafte ist dann übersprungen.

```
@Bean
CommonErrorHandler errorHandler(ConsumerRecordRecoverer recoverer) {
    // Spring Framework 7: ExponentialBackOff mit maxAttempts
    // (ersetzt ExponentialBackOffWithMaxRetries aus Spring Kafka 3)
    var backoff = new ExponentialBackOff(1_000, 2.0);  // 1, 2, 4, 8 s
    backoff.setMaxAttempts(4);                          // 4 Wiederholungen

    var handler = new DefaultErrorHandler(recoverer, backoff);
    // dauerhafte Fehler: sofort zum Recoverer
    handler.addNotRetryableExceptions(UnbekannterTarifException.class);
    handler.setRetryListeners((datensatz, fehler, versuch) ->
            log.warn("Versuch {} fehlgeschlagen: {} @{}: {}", versuch,
                    datensatz.key(), datensatz.offset(),
                    fehler.getCause().getMessage()));
    return handler;
}
```

Spring Boot setzt eine einzelne `CommonErrorHandler`-Bean automatisch in die Listener-Container ein. Der `RetryListener` ist optional, macht aber auf der Konsole sichtbar, was sonst nur im Debug-Log steht.

Fehlerart	Beispiel	Wiederholen?
vorübergehend	Datenbank kurz nicht erreichbar, Timeout eines Partnersystems	ja, mit Backoff
dauerhaft	ungültige Vertragsnummer, unbekannter Tarif, kaputtes JSON	nein – `addNotRetryableExceptions`
Deserialisierung	Nachricht passt nicht zum Schema	nein – `ErrorHandlingDeserializer` meldet sie direkt

9.3 Anhalten oder Dead-Letter-Topic?

Nach den Wiederholungen gibt es zwei grundsätzlich verschiedene Wege.

9.3.1 Dead-Letter-Topic: aussortieren und weitermachen

```
@Bean
ConsumerRecordRecoverer deadLetter(KafkaTemplate<Object,
        Object> template) {
    // schreibt nach <topic>-dlt, gleiche Partition
    return new DeadLetterPublishingRecoverer(template);
}
```

Die fehlerhafte Nachricht landet in `bestand.vertragsaenderungen-dlt`, mit der Ausnahme und dem Original-Offset in Headern. Der Consumer macht mit der nächsten Nachricht weiter. Das DLT braucht mindestens so viele Partitionen wie das Quell-Topic.

Header im DLT	Inhalt
`kafka_dlt-original-topic`, `-partition`, `-offset`	woher die Nachricht kam
`kafka_dlt-exception-fqcn`	`ListenerExecutionFailedException` – Spring verpackt die Ausnahme des Listeners
`kafka_dlt-exception-cause-fqcn`	die eigentliche Ursache, z. B. `UnbekannterTarifException`
`kafka_dlt-exception-message`, `-stacktrace`	Text und Stacktrace

Der Test `DeadLetterIT` prüft genau diese Header – und dass die Folgeänderung v3 verbucht wird, obwohl v2 im DLT liegt.

9.3.2 Anhalten: stehen bleiben, Mensch entscheidet

```
@Bean
ConsumerRecordRecoverer anhalten(KafkaListenerEndpointRegistry registry) {
    return (datensatz, fehler) -> {
        log.error("ANHALTEN: Vertrag {} (P {}, Offset {}) scheitert",
                datensatz.key(), datensatz.partition(),
                        datensatz.offset());
```

```
            var container = registry.getListenerContainer("bestand");
            // nicht aus dem Consumer-Thread stoppen
            new Thread(() -> container.stop(), "anhalten").start();
            throw new IllegalStateException(
                    "Bestandsverwaltung angehalten bei " + datensatz.key());
        };
    }
```

Die Ausnahme sorgt dafür, dass der Offset der fehlerhaften Nachricht nicht committet wird. Nach der Korrektur startet der Container genau dort neu. Im Modul `09a-fehlerbehandlung` wählt `app.recoverer=anhalten|dlt` über `@ConditionalOnProperty` eine der beiden Beans; der Test `AnhaltenIT` prüft, dass der Container steht und v3 **nicht** verbucht wurde.

	Dead-Letter-Topic	Anhalten
Verarbeitung	läuft weiter	steht für diese Anwendung
Reihenfolge pro Vertrag	**verletzt**: spätere Änderungen an V-1001 überholen die fehlerhafte	bleibt erhalten
Lag	bleibt klein	wächst – Kafka puffert alles
Richtig für	unabhängige Nachrichten: Dokumente erzeugen, E-Mails, Statistiken	abhängige Änderungen: Bestandsverwaltung, Inkasso
Pflicht	DLT überwachen und abarbeiten	Alarm, Runbook, schnelle Reaktion

Für die Bestandsverwaltung: anhalten

Eine Adressänderung, die ins DLT wandert, während die folgende Kündigung verbucht wird, ergibt einen falschen Bestand. Für abhängige Änderungen gilt: blockierend wiederholen, dann anhalten und alarmieren. Die `bestand-demo` zeigt beide Varianten (`bestand.fehlerbehandlung=anhalten` bzw. `dlt`).

9.3.3 Nicht blockierende Wiederholung mit `@RetryableTopic`

Spring Kafka kann Wiederholungen auch über eigene Retry-Topics abwickeln:

```
@RetryableTopic(attempts = "4",
        backOff = @BackOff(delayString = "${app.retry.delay-ms}",
                multiplier = 2.0),
        replicationFactor = "${app.replikation}")
@KafkaListener(id = "dokumente", idIsGroup = false,
        topics = "${app.topic-dokumente}",
        properties = "spring.json.value.default.type="
                + "de.konitzer.beispiele.ereignisse.Dokumentauftrag")
void erzeugeDokument(Dokumentauftrag auftrag,
        @Header(KafkaHeaders.RECEIVED_TOPIC) String topic) {
    if ("KAPUTT".equals(auftrag.vorlage())) {
        throw new IllegalStateException("Vorlage kaputt");
    }
    log.info("Dokument {} für {} erzeugt", auftrag.vorlage(),
```

```
            auftrag.vertragsnummer());
}

@DltHandler
void nichtErzeugbar(Dokumentauftrag auftrag,
        @Header(KafkaHeaders.EXCEPTION_MESSAGE) String grund) {
    log.error("DLT: Dokument {} für {} nicht erzeugbar: {}",
            auftrag.vorlage(), auftrag.vertragsnummer(), grund);
}
```

`@BackOff` kommt in Spring Kafka 4 aus `org.springframework.kafka.annotation` (Spring Retry wird nicht mehr gebraucht). Die `properties` am Listener überschreiben den JSON-Zieltyp nur für diesen Consumer – so lesen zwei Listener in einer Anwendung zwei verschiedene Records.

Die fehlerhafte Nachricht wandert in `dokumente.erzeugen-retry-1000`, `-retry-2000`, `-retry-4000` (Spring benennt die Retry-Topics nach der Wartezeit in Millisekunden), die Haupt-Partition läuft sofort weiter. Das ist ideal für unabhängige Aufträge – und zerstört die Reihenfolge. Für Vertragsänderungen ist es tabu. Der Test `DeadLetterIT` zählt mit: vier Versuche (Haupt-Topic plus drei Retry-Topics), dann der `@DltHandler`. Die Topics legt Spring selbst an; `replicationFactor` gehört deshalb in die Annotation.

9.3.4 Wenn die Wiederholung auch scheitert

Dann ist die Nachricht eine Poison Pill, und die Anwendung muss entschieden haben, bevor Kafka es für sie tut. Drei Ausgänge:

1. **Endlos wiederholen.** Das Verhalten ohne Konfiguration: Backoff, nächster Versuch, wieder Fehler. Die Partition steht, der Lag wächst, nichts geht verloren, aber niemand merkt es außer dem Monitoring. Der schlechteste Ausgang, weil er still ist.
2. **Anhalten und alarmieren.** Nach n Versuchen stoppt der Container (`09a`, Recoverer `anhalten`): Offset bleibt stehen, Lag steigt sichtbar, Alert `ConsumerLagHigh` feuert, ein Mensch sieht sich die Nachricht an. Danach: Fehler in der Anwendung beheben und neu starten (die Nachricht geht dann durch), die Nachricht in der Quelle korrigieren und erneut senden (die alte per `seek` überspringen oder im Code als Duplikat behandeln), oder bewusst entscheiden, dass sie verworfen wird. In der Fallstudie sind das die Endpunkte `korrigieren` und `starten`.
3. **Nach n Versuchen weglegen.** Dead-Letter-Topic oder `@RetryableTopic` mit `@DltHandler`: Die Nachricht wandert in `…-dlt`, der Offset geht weiter. Richtig für unabhängige Aufträge, falsch für Versionen eines Vertrags.

Was nicht hilft: ein größeres Backoff. Ein fachlicher Fehler (unbekannter Tarif, kaputtes JSON) wird durch Warten nicht besser. Deshalb trennt `09a` mit `addNotRetryableExceptions` zwischen vorübergehend (wiederholen) und endgültig (sofort zum Recoverer).

9.3.5 Das Dead-Letter-Topic als Friedhof: drei Pflichten

Ein Topic als Ablage für gescheiterte Nachrichten ist ein guter Friedhof, wenn es drei Dinge hat, die ein Friedhof sonst nicht hat:

- **Eine eigene Retention.** Das Haupt-Topic hat vielleicht 14 Tage, das DLT sollte länger halten (90 Tage, oder `retention.ms=-1` mit `retention.bytes` als Obergrenze). Sonst ist die Nach-

richt weg, bevor jemand nachsieht. `09a` legt das DLT mit derselben Konfiguration wie das Quell-Topic an; für Produktion ist das die erste Zeile, die man ändert.

- **Die Todesursache als Header.** Spring schreibt sie mit: `kafka_dlt-exception-cause-fqcn`, `kafka_dlt-exception-message`, `kafka_dlt-original-topic`, `-partition`, `-offset`, `-timestamp`. Damit lässt sich jede Nachricht zum Ursprung zurückverfolgen; `18-fallstudie/shell/03-dlt-ansehen.sh` zeigt die Header.
- **Einen Alert, dass jemand dort liegt.** `kafka_log_log_size{topic=~".*-dlt"} > 0` oder der Lag einer Gruppe, die das DLT überwacht. Ohne Alarm wird aus dem Friedhof ein Massengrab, das niemand öffnet.

Was mit den Nachrichten passiert, braucht einen Prozess, keinen Automatismus: nachsehen, Ursache beheben, dann entweder die Nachricht in das Ursprungs-Topic zurückschreiben (Kafbat UI oder ein kleiner Producer, Key beibehalten) oder bewusst verwerfen. Ein automatischer Rückweg aus dem DLT erzeugt Schleifen, solange der Fehler noch da ist. Und der Haken aus dem Abschnitt davor bleibt: Für Nachrichten mit Reihenfolge je Key versteckt der Friedhof das Problem nur.

9.4 Das Outbox-Muster

Das Bestandssystem ändert einen Vertrag in seiner Datenbank und soll das Ereignis nach Kafka schreiben. Zwei Systeme, keine gemeinsame Transaktion:

- Erst Datenbank, dann Kafka: Stürzt die Anwendung dazwischen ab, ist der Vertrag geändert, aber niemand erfährt davon.
- Erst Kafka, dann Datenbank: Scheitert der Datenbank-Commit, haben alle anderen ein Ereignis zu einer Änderung, die es nicht gibt.

Die **Transactional Outbox** löst das, indem das Ereignis in derselben Datenbank-Transaktion gespeichert wird:

```
CREATE TABLE outbox (
    id            UUID PRIMARY KEY,
    -- Vertragsnummer = späterer Kafka-Key
    aggregat_id   VARCHAR(20)  NOT NULL,
    typ           VARCHAR(50)  NOT NULL,
    nutzdaten     TEXT         NOT NULL,   -- JSON des Ereignisses
    erstellt_am   TIMESTAMP    NOT NULL,
    gesendet_am   TIMESTAMP
);
```

Im Modul `09b-outbox` ist das eine JPA-Entity (`@Entity @Table(name = "outbox")`) neben der Entity `Vertrag`, beide in einer H2-Datenbank. Der Service schreibt beides in **einer** Transaktion; Kafka kommt hier nicht vor:

```
@Transactional
public int adresseAendern(String vertragsnummer, String neuePlz) {
    Vertrag vertrag = vertraege.findById(vertragsnummer)
            .orElseGet(() -> new Vertrag(vertragsnummer, null));
    vertrag.setPlz(neuePlz);
    vertrag.erhoeheVersion();
```

```
    vertraege.save(vertrag);

    var ereignis = new Vertragsaenderung(vertragsnummer,
            vertrag.getVersion(), "ADRESSE", Map.of("plz", neuePlz),
            Instant.now());
    outbox.save(new OutboxEintrag(vertragsnummer, "ADRESSE",
            json.writeValueAsString(ereignis)));

    if ("00000".equals(neuePlz)) {
        // simulierter Fehler: Rollback nimmt Vertrag UND Eintrag zurück
        throw new IllegalArgumentException("PLZ ungültig");
    }
    return vertrag.getVersion();
}   // ein Commit: Vertrag und Outbox-Eintrag – oder keins von beiden
```

`json` ist der `JsonMapper` von Jackson 3 (`tools.jackson.databind.json.JsonMapper`), den Spring Boot 4 bereitstellt. Der Test `OutboxIT` prüft beide Seiten: Die gültige Änderung kommt mit Key `V-1001` und `"version":1` in Kafka an, nach dem Rollback gibt es weder Vertrag noch Outbox-Eintrag für `V-1002`.

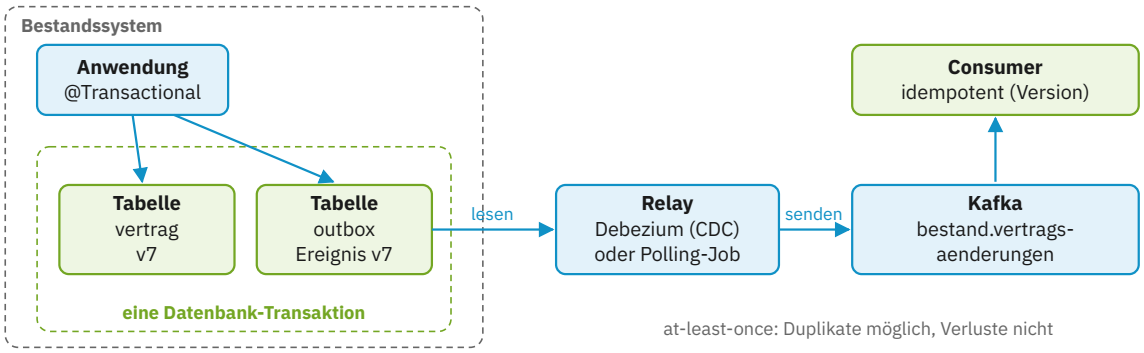


Abbildung 10: Transactional Outbox mit Relay

Ein Relay liest die Outbox und schreibt nach Kafka:

Relay	Wie	Eigenschaften
Polling in der Anwendung	geplanter Job liest ungesendete Einträge, sendet, markiert `gesendet_am`	einfach; Duplikate möglich, wenn nach dem Senden vor dem Markieren abgestürzt
Change Data Capture (Debezium)	liest das Transaktionslog der Datenbank, Outbox Event Router schreibt nach Kafka	keine Last durch Polling, Reihenfolge wie in der Datenbank; eigener Betrieb (Kapitel 12)

Ein einfaches Polling-Relay:

```
@Scheduled(fixedDelayString = "${app.relay-intervall-ms}")
public void senden() throws Exception {
    for (OutboxEintrag eintrag :
            outbox.findTop100ByGesendetAmIsNullOrderByErstelltAm()) {
```

```
            var meta = kafka.send(topic, eintrag.getAggregatId(),
                    eintrag.getNutzdaten())
                    // bewusst synchron: erst markieren, wenn Kafka bestätigt
                    // hat; Reihenfolge je Vertrag halten
                    .get(10, TimeUnit.SECONDS)
                    .getRecordMetadata();
            eintrag.setGesendetAm(Instant.now());
            outbox.save(eintrag);
            log.info("Outbox -> Kafka: {} -> Partition {}, Offset {}",
                    eintrag.getAggregatId(), meta.partition(), meta.offset());
        }
    }
```

Das Repository ist eine Spring-Data-Abfrage: `findTop100ByGesendetAmIsNullOrderByErstelltAm()` – die ältesten ungesendeten zuerst. Die Nutzdaten liegen in der Outbox schon als JSON-Text, deshalb reicht dem Relay ein `KafkaTemplate<String, String>` mit `StringSerializer`.

Beide Relays liefern **at-least-once**: Ein Ereignis kann zweimal ankommen. Deshalb trägt jedes Ereignis eine Version, und der Consumer verarbeitet idempotent (Kapitel 8).

Weitere Wege, Datenbank und Ereignisstrom zusammenzuhalten:

Weg	Wie	Vorsicht
CDC direkt auf den Fachtabellen	Debezium liest Änderungen der Vertragstabelle	kein Code-Umbau, aber das Tabellenmodell wird zum öffentlichen Vertrag
„Listen to yourself“	zuerst nach Kafka schreiben; die eigene Anwendung liest das Ereignis und aktualisiert erst dann die Datenbank	die Anwendung sieht ihre eigene Änderung verzögert
Outbox mit Polling	Job liest die Outbox regelmäßig	einfacher, mehr Latenz und Datenbank-Last
Outbox mit CDC	Debezium liest die Outbox aus dem Transaktionslog	zusätzlicher Betrieb von Connect und Debezium

Läuft das Relay auf mehreren Instanzen?

Dann dürfen nicht zwei Instanzen dieselben Einträge senden. Lösungen: Sperre auf Datenbankebene (`SELECT … FOR UPDATE SKIP LOCKED`), nur eine aktive Instanz (Leader Election, z. B. ShedLock) oder CDC statt Polling.

9.5 Kafka-Transaktionen

Liest eine Anwendung aus Kafka und schreibt das Ergebnis wieder nach Kafka – etwa: Schadenmeldung lesen, Reserve berechnen, Reserve-Ereignis schreiben –, dann können Lesen, Schreiben und Commit in **einer** Kafka-Transaktion stehen:

```
spring.kafka.producer.transaction-id-prefix=reserve-
spring.kafka.consumer.isolation-level=read_committed
```

Mit `transaction-id-prefix` legt Spring Boot einen `KafkaTransactionManager` an, und der Listener-Container führt jeden Datensatz (bzw. Batch) in einer Transaktion aus. Was der Listener mit dem `KafkaTemplate` sendet, und der Offset-Commit werden gemeinsam festgeschrieben – oder gemeinsam verworfen.

```
@KafkaListener(id = "reserven", idIsGroup = false,
        topics = "${app.topic-meldungen}")
void reserveBerechnen(Schadenmeldung meldung) {
    // schon gesendet, bevor die Prüfung wirft – das Ereignis gehört zur
    // Transaktion und wird mit ihr verworfen
    var reserve = new Reserve(meldung.schadennummer(),
            berechne(meldung.betrag()), "120 % der Meldung");
    kafka.send(reservenTopic, meldung.schadennummer(), reserve);

    if (meldung.betrag().signum() <= 0) {
        throw new IllegalArgumentException("Schadensumme nicht positiv");
    }
}   // Commit der Transaktion: Reserve-Ereignis und Offset der Meldung
```

Wirft der Listener, bricht Spring die Transaktion ab. Danach übernimmt nicht der Error Handler, sondern der `AfterRollbackProcessor`: Er setzt den Consumer zurück, wiederholt nach Backoff und ruft am Ende seinen Recoverer:

```
@Bean
AfterRollbackProcessor<Object, Object> nachRollback() {
    return new DefaultAfterRollbackProcessor<>((datensatz, fehler) ->
            log.error("ÜBERSPRUNGEN: {} @{}: {}", datensatz.key(),
                    datensatz.offset(), fehler.getCause().getMessage()),
            new FixedBackOff(500L, 2L));   // 2 Wiederholungen
}
```

Auch ein transaktionaler Producer außerhalb eines Listeners sendet nur innerhalb einer Transaktion – sonst wirft `send()`: `kafka.executeInTransaction(t -> { t.send(…); return null; })`. Consumer der Ergebnisse müssen `isolation.level=read_committed` setzen, sonst sehen sie auch Nachrichten abgebrochener Transaktionen. Der Test `TransaktionIT` macht den Unterschied sichtbar: Nach einer gültigen und einer ungültigen Meldung sieht ein `read_committed`-Consumer **eine** Reserve, ein `read_uncommitted`-Consumer **vier** – die gültige und die drei verworfenen Versuche.

Warum das Beispiel flush() aufruft

Bricht der Producer eine Transaktion ab, verwirft er alles, was noch in seinem Puffer liegt und den Broker nie erreicht hat. Wirft der Listener direkt nach `send()`, kommt die Reserve also gar nicht erst im Topic an – und `read_uncommitted` hätte nichts zu zeigen. `ReserveBerechnung` ruft deshalb vor der Prüfung `kafka.flush()` auf. In Produktion lässt man das weg; es dient nur der Vorführung.

Was Kafka-Transaktionen leisten	Was nicht
Lesen → Verarbeiten → Schreiben innerhalb Kafka genau einmal	eine Datenbank-Änderung in dieselbe Transaktion ziehen
mehrere Topics atomar beschreiben	Duplikate aus der Quelle verhindern
Zombies (alte Instanzen nach Rebalance) abwehren	Fehler im fachlichen Code ungeschehen machen

Für Datenbank + Kafka bleibt die Outbox das Mittel der Wahl. Kafka Streams nutzt Transaktionen automatisch mit `processing.guarantee=exactly_once_v2` (Kapitel 11).

9.5.1 Unter der Haube: Wie eine Kafka-Transaktion abläuft

1. **Anmelden:** Der Producer hat eine feste `transactional.id`. Beim Start fragt er den **Transaction Coordinator** – einen Broker, bestimmt über den Hash der ID auf `__transaction_state` – nach Producer-ID und Epoche. Dabei wird die Epoche erhöht: Eine ältere Instanz mit derselben ID (ein „Zombie“ nach einem Rebalance) wird ab jetzt abgewiesen.
2. **Beginnen und schreiben:** Der Producer schreibt in beliebige Partitionen. Der Coordinator merkt sich, welche Partitionen zur Transaktion gehören.
3. **Offsets mitschreiben:** Bei Lesen-Verarbeiten-Schreiben übergibt der Producer auch die Offsets der gelesenen Nachrichten an die Transaktion – sie werden Teil desselben Commits.
4. **Abschließen:** Beim Commit schreibt der Coordinator zuerst „PREPARE_COMMIT“ in sein Log, dann einen **Kontroll-Marker** (COMMIT bzw. ABORT) in jede beteiligte Partition, zuletzt „COMPLETE_COMMIT“.
5. **Lesen:** Ein Consumer mit `read_committed` liest nur bis zum **Last Stable Offset** – dem ersten Offset einer noch offenen Transaktion. Nachrichten abgebrochener Transaktionen filtert er über einen Index heraus.

Die Kontroll-Marker belegen eigene Offsets – deshalb gibt es in transaktionalen Topics Lücken in der Offset-Folge.

9.5.2 Hintergrund und Konsequenzen

- **Transaktionen kosten:** zusätzliche Requests an den Transaction Coordinator, und `read_committed`-Consumer lesen nur bis zur letzten abgeschlossenen Transaktion (Last Stable Offset). Eine hängende Transaktion lässt den Lag aller dieser Consumer wachsen.
- `transaction.timeout.ms` (Standard 60 s) begrenzt, wie lange eine Transaktion offen sein darf; danach bricht der Broker sie ab.
- **Interne Infrastruktur:** `__transaction_state` braucht wie `__consumer_offsets` RF 3 und `min.insync.replicas=2`.

9.6 Fehler in der Deserialisierung

Eine Nachricht, die nicht zum erwarteten Format passt, scheitert schon vor dem Listener. Mit dem `ErrorHandlingDeserializer` (Kapitel 8) wird daraus ein normaler Fehler, den der Error Handler sofort – ohne Wiederholung – an den Recoverer gibt. Im DLT landet die Nachricht dann mit den Original-Bytes; im Anhalten-Fall steht die Anwendung, bis die Nachricht geklärt ist.

9.7 Checkliste Zustellsicherheit

- ☐ Producer: `acks=all`, Idempotenz an, Ergebnis von `send()` ausgewertet
- ☐ Datenbank und Ereignis über Outbox oder CDC gekoppelt
- ☐ Topic: RF 3, `min.insync.replicas=2`, Unclean Election aus
- ☐ Consumer: Auto-Commit aus, Commit nach der Verarbeitung
- ☐ Verarbeitung idempotent (Version, Upsert oder Dedupe)
- ☐ Error Handler mit Backoff und Liste dauerhafter Fehler
- ☐ Entscheidung „anhalten oder DLT“ je Topic getroffen und dokumentiert
- ☐ DLT bzw. angehaltene Listener werden überwacht und alarmiert
- ☐ Kafka-zu-Kafka-Verarbeitung mit Transaktionen oder Kafka Streams `exactly_once_v2`

9.8 Weiterführende Links

- Spring for Apache Kafka, Error Handling: https://docs.spring.io/spring-kafka/reference/kafka/annotation-error-handling.html
- Spring for Apache Kafka, Transactions: https://docs.spring.io/spring-kafka/reference/kafka/transactions.html
- Debezium Outbox Event Router: https://debezium.io/documentation/reference/stable/transformations/outbox-event-router.html

10 Datenformate und Schemas

Für den Broker sind Nachrichten nur Bytes. Was darin steht, ist ein Vertrag zwischen dem Team, das schreibt, und allen Teams, die lesen – oft über Jahre. Dieses Kapitel zeigt, wie man Ereignisse entwirft, welches Format passt und wie Schemas verhindern, dass eine Änderung im Bestandssystem das Inkasso lahmlegt.

10.1 Ereignisse entwerfen

10.1.1 Ereignis oder Befehl?

	Ereignis	Befehl
Bedeutung	etwas ist passiert	etwas soll passieren
Name	Vergangenheit: `VertragGekuendigt`, `SchadenGemeldet`	Imperativ: `PoliceErstellen`, `DokumentDrucken`
Empfänger	beliebig viele, dem Sender unbekannt	ein bestimmter Dienst
Typisches Topic	`bestand.vertragsaenderungen`	`dokumente.erzeugen`

Kafka eignet sich für beides. Ereignisse sind der Normalfall; Befehle über Kafka sind sinnvoll, wenn Entkopplung und Pufferung wichtiger sind als eine direkte Antwort.

10.1.2 Wie viel gehört in ein Ereignis?

Stil	Inhalt	Vorteil	Nachteil
Benachrichtigung (*thin*)	nur Vertragsnummer und Art: „V-1001 geändert“	klein, keine Datenhoheit verletzt	jeder Leser fragt beim Bestandssystem nach – Last und Kopplung
Zustandsübertragung (*fat*)	vollständiger neuer Stand des Vertrags	Leser brauchen keine Rückfrage	groß; personenbezogene Daten verteilen sich
Delta	was sich geändert hat, mit Version	kompakt, nachvollziehbar	Leser brauchen den vorherigen Stand

Für die Bestandsverwaltung hat sich das Delta mit Versionsnummer bewährt: `{"vertragsnummer":"V-1001","version":7,"art":"ADRESSE","daten":{"plz":"50667"}}`. Für ein kompaktiertes Topic mit dem aktuellen Stand je Kunde (`kunden.stammdaten`) passt die Zustandsübertragung.

10.1.3 Pflichtfelder, die sich bewähren

- **Fachlicher Schlüssel** – auch im Wert, nicht nur als Key (Vertragsnummer, Schadennummer).
- **Version** je Fachobjekt – für Idempotenz und Lückenerkennung.
- **Ereignis-ID** (UUID) – für Dedupe und Nachverfolgung.
- **Zeitpunkt des fachlichen Ereignisses** – nicht nur der Kafka-Timestamp.
- **Schema- oder Ereignistyp** – damit Leser verschiedene Ereignisse in einem Topic unterscheiden können.

10.2 Formate im Vergleich

Format	Schema	Größe	Werkzeuge	Passt für
Text / CSV	keines	klein	überall lesbar	Demos, einfache Kennzahlen
JSON	optional (JSON Schema)	mittel	überall lesbar, Spring-Standard	Einstieg, Teams ohne Schema Registry
Avro	Pflicht	klein (binär)	Schema Registry, Code-Generierung	Plattformen mit vielen Teams, langlebige Topics
Protobuf	Pflicht	klein (binär)	Schema Registry, gRPC-Umfeld	Teams, die Protobuf ohnehin nutzen

10.3 JSON mit Spring Kafka

Kapitel 7 und 8 haben JSON mit Jackson bereits verwendet:

```
# Producer
spring.kafka.producer.value-serializer=\
    org.springframework.kafka.support.serializer.JacksonJsonSerializer
spring.kafka.producer.properties.spring.json.add.type.headers=false

# Consumer
spring.kafka.consumer.value-deserializer=\
    org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
spring.kafka.consumer.properties.spring.deserializer.value.delegate.class=\
    org.springframework.kafka.support.serializer.JacksonJsonDeserializer
spring.kafka.consumer.properties.spring.json.value.default.type=\
    de.beispiel.bestand.Vertragsaenderung
spring.kafka.consumer.properties.spring.json.use.type.headers=false
```

10.3.1 Typ-Header: praktisch, aber gefährlich

Standardmäßig schreibt der Spring-Serializer den Java-Klassennamen in einen Header (`__TypeId__`), und der Deserializer erzeugt diese Klasse. Das koppelt Producer und Consumer an denselben Klassennamen – benennt das Bestandssystem sein Paket um, scheitern alle Leser. Außerdem darf ein Deserializer nur Klassen aus vertrauenswürdigen Paketen erzeugen (`spring.json.trusted.packages`); `*` öffnet die Tür für fremde Klassen.

Empfehlung: Typ-Header beim Producer abschalten, beim Consumer einen festen Zieltyp setzen – wie oben.

10.3.2 Schema-Brüche mit JSON

JSON prüft nichts. Typische Brüche:

- Feld umbenannt (`plz` → `postleitzahl`): Leser sehen `null`.
- Typ geändert (`"version":"7"` statt `7`): Deserialisierung scheitert – Poison Pill.
- Pflichtfeld entfernt: Fachlogik scheitert erst Tage später.

Gegenmittel ohne Schema Registry: unbekannte Felder tolerieren (prüfen, wie der verwendete Mapper `FAIL_ON_UNKNOWN_PROPERTIES` setzt; im Zweifel `@JsonIgnoreProperties(ignoreUnknown = true)` an den Ereignis-Klassen), Felder nur hinzufügen, nie umbenennen, und Verträge in Tests prüfen (Kapitel 13).

10.4 Schemas und Schema Registry

Eine Schema Registry speichert die Schemas aller Topics und prüft beim Schreiben, ob eine neue Schema-Version zu den alten passt. Producer schreiben nur noch eine kleine Schema-ID in die Nachricht, Consumer holen das Schema zur ID.

Kafka selbst bringt keine Schema Registry mit. Verbreitet sind:

Produkt	Lizenz	Anmerkung
Confluent Schema Registry	Confluent Community License	weit verbreitet; Nutzung im eigenen Betrieb erlaubt, nicht als konkurrierender SaaS-Dienst
Apicurio Registry	Apache 2.0	Red-Hat-Umfeld, kompatible API
Karapace	Apache 2.0	kompatible API zur Confluent Schema Registry

10.4.1 Ein Avro-Schema für Vertragsänderungen

```
{
  "type": "record",
  "name": "Vertragsaenderung",
  "namespace": "de.beispiel.bestand",
  "fields": [
    { "name": "vertragsnummer", "type": "string" },
    { "name": "version",        "type": "int" },
    { "name": "art",            "type": { "type": "enum",
            "name": "Aenderungsart",
                                    "symbols": ["ADRESSE",
                                            "BEITRAG",
                                            "BANKVERBINDUNG",
                                                    "KUENDIGUNG"] } },
    { "name": "plz",            "type": ["null", "string"],
            "default": null },
    { "name": "zeitpunkt",      "type": { "type": "long",
            "logicalType": "timestamp-millis" } }
  ]
}
```

In Spring Boot (mit der Serializer-Bibliothek der jeweiligen Registry):

```
spring.kafka.producer.value-serializer=\
    io.confluent.kafka.serializers.KafkaAvroSerializer
spring.kafka.consumer.value-deserializer=\
    io.confluent.kafka.serializers.KafkaAvroDeserializer
spring.kafka.properties.schema.registry.url=\
    https://schema-registry.intern:8081
```

```
spring.kafka.properties.specific.avro.reader=true
# in Produktion: Schemas über die Pipeline registrieren, nicht beim Senden
spring.kafka.producer.properties.auto.register.schemas=false
```

10.4.2 Kompatibilitätsregeln

Modus	Erlaubt	Wer zuerst aktualisiert
`BACKWARD` (Standard)	Felder löschen, optionale Felder (mit Default) hinzufügen	Consumer zuerst – neue Leser verstehen alte Nachrichten
`FORWARD`	Felder hinzufügen, optionale Felder löschen	Producer zuerst – alte Leser verstehen neue Nachrichten
`FULL`	nur optionale Felder hinzufügen oder löschen	egal
`*_TRANSITIVE`	wie oben, gegen **alle** früheren Versionen	wichtig bei langer Retention oder Replay
`NONE`	alles	niemand ist geschützt

Für Topics, die Jahre gelesen und erneut abgespielt werden, ist `FULL_TRANSITIVE` die sicherste Wahl: Jede Version kann jede andere lesen.

10.5 Namenskonventionen für Topics

Ein Topic-Name lebt so lange wie das Topic – Umbenennen geht nicht. Bewährt hat sich `<domäne>.<inhalt>[.<variante>]`:

Topic	Inhalt
`bestand.vertragsaenderungen`	Änderungen an Verträgen (Ereignisse)
`schaden.meldungen`	neu gemeldete Schäden
`inkasso.zahlungen`	Zahlungseingänge
`kunden.stammdaten`	letzter Stand je Kunde (kompaktiert)
`dokumente.erzeugen`	Aufträge an den Dokumentendienst (Befehle)
`bestand.vertragsaenderungen-dlt`	Dead-Letter-Topic

Regeln: Kleinbuchstaben, keine Umgebung im Namen (die gehört in den Cluster), keine Teamnamen (Teams ändern sich), Punkte **oder** Unterstriche – nicht beides, weil Kafka sie in Metriknamen gleich behandelt.

10.6 Kompaktierte Topics

Für Stammdaten will man nicht die ganze Historie, sondern den letzten Stand je Schlüssel:

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic
kunden.stammdaten --partitions 6 --replication-factor 3 --config
cleanup.policy=compact --config min.compaction.lag.ms=3600000
```

Kafka entfernt im Hintergrund ältere Nachrichten mit gleichem Key. Ein **Tombstone** – Key mit Wert `null` – löscht den Kunden ganz; nach `delete.retention.ms` (Standard 24 h) verschwindet auch

der Tombstone. Das ist der Weg für Löschpflichten nach DSGVO in kompaktierten Topics. Kafka Streams liest solche Topics als `KTable` (Kapitel 11).

10.7 Zusammenfassung

- Ereignisse als Delta mit fachlichem Schlüssel, Version, Ereignis-ID und Zeitpunkt.
- JSON ist ein guter Einstieg; Typ-Header abschalten, `ErrorHandlingDeserializer` davor.
- Ab mehreren Teams und langer Retention: Schemas mit Registry und einer bewusst gewählten Kompatibilitätsregel.
- Topic-Namen nach Domäne und Inhalt; Stammdaten als kompaktiertes Topic.

10.8 Weiterführende Links

- Apache Avro: https://avro.apache.org/docs/
- Apicurio Registry: https://www.apicur.io/registry/
- Kafka-Dokumentation, Log Compaction: https://kafka.apache.org/documentation/#compaction

11 Kafka Streams mit Spring Boot

Ein Antrag und die zugehörige Risikoprüfung sollen zu einer Police zusammengeführt werden. Die Zahl der offenen Schäden je Vertrag soll laufend bekannt sein. Vertragsänderungen sollen um die Kundendaten ergänzt werden. Solche Aufgaben – Ereignisse verbinden, zählen, anreichern – löst Kafka Streams: eine Java-Bibliothek, die in der eigenen Anwendung läuft und Zustand, Fehlertoleranz und Skalierung übernimmt.

11.1 Grundbegriffe

Begriff	Bedeutung	Beispiel
`KStream`	Strom einzelner Ereignisse	jede Schadenmeldung
`KTable`	aktueller Stand je Key, aus einem Topic aufgebaut	Kundenstammdaten je Kundennummer
`GlobalKTable`	wie `KTable`, aber vollständig auf jeder Instanz	Tariftabelle
Topologie	der Verarbeitungsgraph aus Quellen, Operationen und Zielen	Anträge + Prüfungen → Join → Policen
State Store	lokaler Zustand (RocksDB), gesichert in einem Changelog-Topic	offene Anträge im Zeitfenster
Task	Verarbeitungseinheit je Eingangspartition	Partition 2 von `antraege` und `pruefungen`
`application.id`	Name der Anwendung, zugleich Consumer Group und Präfix interner Topics	`join-app`

Kafka Streams ist keine eigene Plattform: kein Cluster, kein Server. Die Anwendung startet mehrere Instanzen, und Kafka verteilt die Tasks über die Consumer Group – genau wie bei einem normalen Consumer.

Die Beispiele zu diesem Kapitel

`kafka-einfach/join-app` für die Vorführung des Joins im Lab; `labs/11-streams` mit allen vier Topologien dieses Kapitels in der Klasse `Topologien` – als statische Methoden auf dem `StreamsBuilder`, ohne Spring, damit der `TopologyTestDriver` sie prüfen kann (Kapitel 13).

11.2 Einrichten in Spring Boot

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-kafka</artifactId>
</dependency>
<dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-streams</artifactId>
</dependency>
```

```
spring.kafka.bootstrap-servers=\
    localhost:19092,localhost:19093,localhost:19094
spring.kafka.streams.application-id=join-app
# interne Topics (Changelog, Repartition) mit 3 Kopien
spring.kafka.streams.replication-factor=3
spring.kafka.streams.properties.default.key.serde=\
    org.apache.kafka.common.serialization.Serdes$StringSerde
spring.kafka.streams.properties.default.value.serde=\
    org.apache.kafka.common.serialization.Serdes$StringSerde
# Lesen, Zustand und Schreiben in einer Kafka-Transaktion
spring.kafka.streams.properties.processing.guarantee=exactly_once_v2
# lokaler Zustand (RocksDB)
spring.kafka.streams.state-dir=zustand
```

```
@SpringBootApplication
@EnableKafkaStreams
public class JoinApp {
    public static void main(String[] args) {
        SpringApplication.run(JoinApp.class, args);
    }
}
```

`@EnableKafkaStreams` legt einen `StreamsBuilder` an und startet die Topologie mit dem Spring-Kontext. Jede `@Bean`-Methode, die den `StreamsBuilder` bekommt, trägt einen Teil zur Topologie bei.

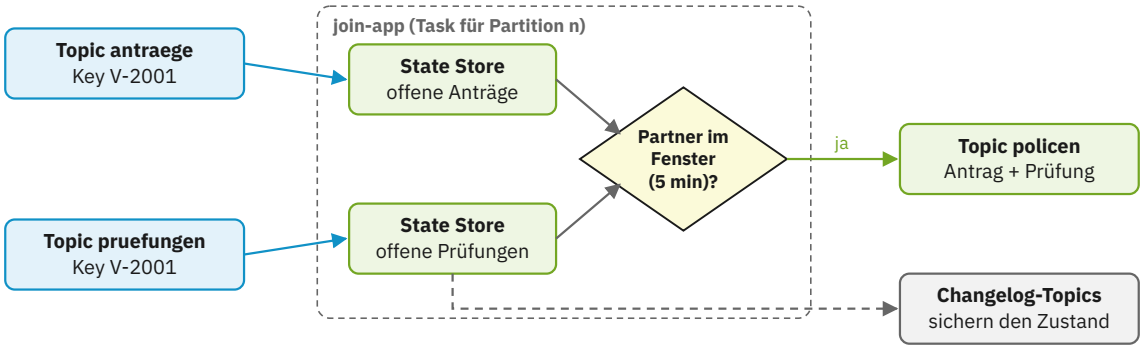
11.3 Zwei Ereignisse zusammenführen: Antrag und Prüfung

Die `join-app` aus `kafka-einfach.zip` führt einen Antrag (Topic `antraege`) und die Risikoprüfung (Topic `pruefungen`) zum selben Vertrag zusammen, wenn beide höchstens fünf Minuten auseinanderliegen. Das Ergebnis geht als Police in das Topic `policen`. Die Reihenfolge der beiden Ereignisse ist egal.

```
@Bean
KStream<String, String> zusammenfuehren(StreamsBuilder builder) {
    KStream<String, String> antraege = builder.stream("antraege");
    KStream<String, String> pruefungen = builder.stream("pruefungen");

    KStream<String, String> policen = antraege.join(
            pruefungen,
            (antrag, pruefung) ->
                    "Police aus Antrag \"" + antrag
                    + "\" und Prüfung \"" + pruefung + "\"",
            JoinWindows.ofTimeDifferenceWithNoGrace(Duration.ofMinutes(5)),
            StreamJoined.with(Serdes.String(), Serdes.String(),
                    Serdes.String())
                    .withStoreName("antrag-pruefung"));
```

```
    policen.to("policen");
    return policen;
}
```



Voraussetzung: gleicher Key und gleiche Partitionsanzahl in beiden Topics (Co-Partitioning).

Abbildung 11: Stream-Stream-Join von Antrag und Prüfung

Im Lab, drei Fenster – die Eingänge liefert die `producer-app` mit anderem Topic:

```
cd $HOME\Desktop\kafka-einfach\join-app
java -jar join-app.jar
```

```
cd $HOME\Desktop\kafka-einfach\producer-app
java -jar producer-app.jar --app.topic=antraege
```

```
cd $HOME\Desktop\kafka-einfach\producer-app
java -jar producer-app.jar --app.topic=pruefungen
```

```
17:12:08  Antrag     key=V-2001   "Antrag KFZ Müller" – wartet auf die Prüfung
17:12:12  Prüfung    key=V-2001   "Risiko geprüft, ok" – wartet auf den Antrag
17:12:12  ZUSAMMEN   key=V-2001   Police aus Antrag "Antrag KFZ Müller" und
Prüfung "Risiko geprüft, ok"
17:13:24  Prüfung    key=V-2004   "Risiko geprüft, ok" – wartet auf den Antrag
17:13:27  Antrag     key=V-2004   "Antrag Leben Weber" – wartet auf die Prüfung
17:13:27  ZUSAMMEN   key=V-2004   Police aus Antrag "Antrag Leben Weber" und
Prüfung "Risiko geprüft, ok"
```

Eine Prüfung ohne passenden Antrag bleibt ohne Ergebnis. Der Start dauert etwa 20 Sekunden: Kafka Streams legt seine internen Topics an und initialisiert die Transaktionen.

11.3.1 Was dabei im Hintergrund passiert

Was	Warum es wichtig ist
gleicher Key in beiden Topics	nur Ereignisse mit gleichem Key werden zusammengeführt
gleiche Partitionsanzahl (Co-Partitioning)	Partition *n* beider Topics bildet eine Task; sonst bricht Kafka Streams beim Start ab
Zeitfenster (`JoinWindows`)	gemessen an den Zeitstempeln der Nachrichten, nicht an der Uhr der Anwendung
State Stores `antrag-pruefung-this/other`	halten die noch offenen Ereignisse beider Seiten
Changelog-Topics `join-app-antrag-pruefung-…-changelog`	sichern die State Stores; nach Verlust wird der Zustand daraus wiederhergestellt
`exactly_once_v2`	Lesen, Zustand und Ergebnis werden in einer Transaktion festgeschrieben

11.4 Weitere Join-Arten

Join	Links	Rechts	Beispiel
Stream–Stream	`KStream`	`KStream` mit Zeitfenster	Antrag + Risikoprüfung
Stream–Table	`KStream`	`KTable`	Vertragsänderung + aktuelle Kundendaten
Stream–GlobalTable	`KStream`	`GlobalKTable`	Schadenmeldung + Tarif (kleine Tabelle, überall gebraucht)
Table–Table	`KTable`	`KTable`	Vertragsstand + Kundenstand zu einer Gesamtsicht

Anreichern einer Vertragsänderung mit Kundendaten aus dem kompaktierten Topic `kunden.stammdaten`:

```
public static KStream<String, String> anreichern(StreamsBuilder builder) {
    // Key: Kundennummer, Wert: Name – je Key der letzte Stand
    KTable<String, String> kunden = builder.table(KUNDEN,
            Materialized.as("kunden"));
    KStream<String, String> aenderungen = builder.stream(AENDERUNGEN);

    KStream<String, String> angereichert = aenderungen
            // Key wechseln: Kafka Streams partitioniert intern um
            .selectKey((vertrag, aenderung) -> kundennummerAus(aenderung))
            .join(kunden, (aenderung, kunde) ->
                    textAus(aenderung) + " | Kunde: " + kunde);

    angereichert.to(ANGEREICHERT);
```

```
    return angereichert;
}
```

Eine Änderung zu einem unbekannten Kunden fällt beim inneren Join weg (`leftJoin` würde sie mit `null` durchlassen). Und weil die `KTable` nur den Stand kennt, der **vor** der Änderung eingetroffen ist, zählt die Reihenfolge der Zeitstempel: Der Test `aenderungWirdUmKundenAngereichert` schickt den Kunden eine Sekunde vor der Änderung.

`selectKey` wechselt den Key von der Vertrags- auf die Kundennummer. Kafka Streams schreibt die Daten dafür über ein internes **Repartition-Topic** neu – zusätzlicher Datenverkehr und eine neue Partitionierung nach Kunde. Die Reihenfolge pro Vertrag gilt danach nicht mehr, nur noch pro Kunde.

Variante	Liefert
`join`	nur Paare
`leftJoin`	auch linke Ereignisse ohne Partner (mit `null`); bei Stream–Stream erst nach Ablauf des Fensters
`outerJoin`	beide Seiten auch ohne Partner

11.5 Zählen und Summieren

Wie viele offene Schäden hat jeder Vertrag?

```
public static KTable<String, Long> schaedenJeVertrag(StreamsBuilder b) {
    // Key der Meldungen: Vertragsnummer
    KTable<String, Long> anzahl = b.<String, String>stream(SCHAEDEN)
            .groupByKey()
            .count(Materialized.as("schaeden-je-vertrag"));

    anzahl.toStream().to(SCHAEDEN_JE_VERTRAG,
            Produced.with(Serdes.String(), Serdes.Long()));
    return anzahl;
}
```

Warum kommt das Ergebnis erst nach 30 Sekunden?

Kafka Streams puffert Aggregationsergebnisse im Cache und schreibt sie erst beim Commit weiter (`commit.interval.ms`, bei `exactly_once_v2` standardmäßig 100 ms, sonst 30 s). Für die Vorführung setzt `11-streams` `statestore.cache.max.bytes=0`, damit jeder Zählschritt sofort erscheint. In Produktion lässt man den Cache an: Er fasst viele Änderungen desselben Keys zu einer zusammen.

Mit Zeitfenstern – Schadenmeldungen je Vertrag in den letzten 24 Stunden, etwa zur Betrugserkennung:

```
builder.<String, String>stream(SCHAEDEN)
        .groupByKey()
        .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofHours(24)))
```

```
        .count(Materialized.as("schaeden-24h"))
        .toStream()
        .filter((fenster, anzahl) -> anzahl >= 3)
        .map((fenster, anzahl) -> KeyValue.pair(fenster.key(),
                "auffällig: " + anzahl + " Schäden in 24 h"))
        .to(AUFFAELLIG);
```

`TimeWindows.ofSize` bildet feste, aneinandergrenzende Fenster (0–24 h, 24–48 h …), gemessen an den Zeitstempeln der Nachrichten. Drei Schäden um 9, 15 und 21 Uhr desselben Tages fallen in ein Fenster; der Test `dreiSchaedenInZwoelfStundenSindAuffaellig` schickt genau das. Für „drei Schäden innerhalb von 24 Stunden, egal wann“ wären `SlidingWindows` das richtige Werkzeug.

11.6 Unter der Haube: Tasks, Threads und Zustand

- **Sub-Topologien und Tasks:** Kafka Streams zerlegt die Topologie in unabhängige Teile. Für jeden Teil gibt es so viele **Tasks**, wie die Eingangs-Topics Partitionen haben. Bei der `join-app` mit je 3 Partitionen sind das 3 Tasks; Task 1 liest Partition 1 von `antraege` und Partition 1 von `pruefungen`.
- **Threads:** Eine Instanz führt ihre Tasks mit `num.stream.threads` Threads aus (Standard 1). Tasks sind die Einheit der Verteilung – sie wandern beim Rebalance zwischen Instanzen und Threads.
- **Zustand je Task:** Jede Task hat ihre eigenen State Stores in einem Unterordner des Zustandsverzeichnisses. Jede Änderung am Store wird zugleich in das Changelog-Topic geschrieben (eine Partition je Task).
- **Wiederherstellung:** Übernimmt eine Instanz eine Task, deren Store sie nicht hat, liest sie das Changelog-Topic von vorn ein, bevor sie neue Daten verarbeitet. Standby-Replikate (`num.standby.replicas`) halten auf anderen Instanzen eine Kopie warm, damit dieser Schritt kurz bleibt.
- **Exactly-once:** Mit `exactly_once_v2` schreibt jede Task Ausgabe, Changelog und Eingangs-Offsets in einer Transaktion. Nach einem Absturz wird die unvollständige Transaktion verworfen und der Store aus dem Changelog wieder auf den festgeschriebenen Stand gebracht.

11.7 Betrieb einer Streams-Anwendung

- `application.id` **ist die Consumer Group:** Lag überwachen wie bei jedem Consumer – `kafka-consumer-groups.sh --describe --group join-app`.
- **Interne Topics** (`<application.id>-…-changelog`, `…-repartition`) gehören zur Anwendung. Sie brauchen RF 3 (`spring.kafka.streams.replication-factor`) und verursachen Speicher und Netzverkehr. State Stores und Operatoren benennen (`withStoreName`, `Materialized.as`, `Named.as`), sonst entstehen Namen wie `KSTREAM-JOINTHIS-0000000006`, die sich bei Änderungen der Topologie verschieben.
- **Wiederherstellung:** Startet eine Instanz auf einem neuen Rechner, baut sie den Zustand aus dem Changelog neu auf. Bei großem Zustand dauert das; `num.standby.replicas=1` hält eine warme Kopie auf einer anderen Instanz.
- **Skalierung:** Höchstens so viele aktive Tasks wie Eingangspartitionen. Mehr Instanzen bleiben Reserve.

- **Reset:** Zustand und Offsets zurücksetzen mit `kafka-streams-application-reset.sh --bootstrap-server kafka1:9092 --application-id join-app` bei gestoppter Anwendung, danach den lokalen Zustandsordner löschen.
- **Zeitstempel:** Fenster richten sich nach den Zeitstempeln der Nachrichten. Falsche Producer-Uhren führen zu verpassten Joins.
- **Topologie ändern:** Neue oder umbenannte Operatoren können neue interne Topics erzeugen. Größere Änderungen erfordern einen Reset oder eine neue `application.id`.

11.8 Kafka Streams oder selbst korrelieren?

Statt Kafka Streams kann eine normale Spring-Anwendung beide Ereignisse selbst zusammenführen – das erste Ereignis in ihrer Datenbank zwischenspeichern und beim zweiten nachschlagen. Als **Skizze** (nicht Teil der Beispielprojekte):

```
@KafkaListener(topics = {"antraege", "pruefungen"})
@Transactional
void verarbeite(ConsumerRecord<String, String> ereignis,
        Acknowledgment commit) {
    Offen offen = repo.findById(ereignis.key())
            .orElse(new Offen(ereignis.key()));
    // idempotent: überschreibt bei Wiederholung
    offen.merke(ereignis.topic(), ereignis.value());
    if (offen.istVollstaendig()) {
        policen.anlegen(offen);
        repo.delete(offen);
    } else {
        repo.save(offen);
    }
    // erst nach der DB-Transaktion
    commit.acknowledge();
}
```

	Kafka Streams	selbst korrelieren
Zustand	lokal (RocksDB) + Changelog-Topics, automatisch wiederhergestellt	in der eigenen Datenbank
„Partner kommt nie“	über das Zeitfenster eingebaut	selbst bauen: Ablaufdatum, Aufräumjob, Alarm
Voraussetzung	gleicher Key, gleiche Partitionsanzahl	ein gemeinsamer Schlüssel
Zustellsicherheit	`exactly_once_v2` innerhalb Kafka	idempotent speichern, Commit nach DB-Transaktion
Betrieb	interne Topics, Zustandsordner, Reset-Werkzeug	bekannt: Datenbank und Listener
Gut für	hohe Volumen, viele Joins und Aggregationen	wenige Fälle, Spring-Team mit Datenbank

11.9 Zusammenfassung

- Kafka Streams ist eine Bibliothek in der Anwendung; Skalierung über Partitionen und Consumer Group.
- Joins brauchen gleichen Key und gleiche Partitionsanzahl; Zeitfenster richten sich nach Nachrichten-Zeitstempeln.
- Zustand liegt lokal und in Changelog-Topics; Stores benennen, RF 3, `exactly_once_v2`.
- Für wenige, einfache Korrelationen ist die eigene Datenbank oft die pragmatischere Wahl.

11.10 Weiterführende Links

- Kafka Streams, Dokumentation: https://kafka.apache.org/documentation/streams/
- Spring for Apache Kafka, Kafka Streams: https://docs.spring.io/spring-kafka/reference/streams.html

12 Kafka Connect

Nicht jedes System soll eigenen Kafka-Code bekommen. Das Bestandssystem auf dem Host schreibt in seine Datenbank, das Reporting erwartet eine Tabelle, die bestehende Queue soll während der Umstellung weiterlaufen. Kafka Connect verbindet solche Systeme mit Kafka über Konfiguration statt Code.

12.1 Konzept

Begriff	Bedeutung
Worker	ein Connect-Prozess; mehrere Worker bilden einen Connect-Cluster
Connector	Konfiguration für eine Verbindung, z. B. „Outbox-Tabelle des Bestandssystems → Kafka“
Source-Connector	liest aus einem Fremdsystem und schreibt nach Kafka
Sink-Connector	liest aus Kafka und schreibt in ein Fremdsystem
Task	Arbeitseinheit eines Connectors; Connect verteilt Tasks auf die Worker
Converter	wandelt zwischen Kafka-Bytes und dem internen Datenmodell (JSON, Avro, String)
SMT (Single Message Transform)	kleine Umformung je Nachricht: Feld umbenennen, Key setzen, filtern

Connect gehört zu Apache Kafka (`connect-distributed.sh`), die meisten Connectoren sind eigene Projekte mit eigenen Lizenzen. MirrorMaker 2 (Kapitel 19) ist selbst ein Satz von Connectoren.

12.1.1 Standalone oder verteilt?

Modus	Konfiguration und Zustand	Einsatz
Standalone	Properties-Datei, Offsets in einer lokalen Datei	Entwicklung, einzelne Edge-Fälle
Distributed	REST-API; Konfiguration, Offsets und Status in Kafka-Topics	Produktion: Ausfallsicherheit und Skalierung über mehrere Worker

Ein verteilter Worker legt drei interne Topics an – `config.storage.topic`, `offset.storage.topic`, `status.storage.topic`. Sie brauchen RF 3 und `cleanup.policy=compact`.

12.2 Typische Connectoren im Versicherungsumfeld

Aufgabe	Connector	Richtung
Outbox-Tabelle des Bestandssystems nach Kafka	Debezium (PostgreSQL, Oracle, Db2, SQL Server) mit Outbox Event Router	Source
Schadenmeldungen in das Reporting-DWH	JDBC Sink	Sink
bestehende IBM-MQ-Queue während der Migration spiegeln	MQ Source	Source
Dokumente-Ereignisse ins Archiv	S3 / Objektspeicher Sink	Sink
Topics zwischen Rechenzentren spiegeln	MirrorMaker 2	Source

12.3 Change Data Capture mit Debezium

Debezium liest das Transaktionslog der Datenbank und erzeugt aus jeder Zeilenänderung ein Ereignis – ohne die Datenbank abzufragen. Zusammen mit der Outbox-Tabelle aus Kapitel 9 entsteht ein zuverlässiges Relay:

```
{
  "name": "bestand-outbox",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "bestand-db.intern",
    "database.port": "5432",
    "database.user": "debezium",
    "database.password": "${file:/opt/connect/geheim.properties:bestand}",
    "database.dbname": "bestand",
    "topic.prefix": "bestand",
    "table.include.list": "public.outbox",
    "transforms": "outbox",
    "transforms.outbox.type": "io.debezium.transforms.outbox.EventRouter",
    "transforms.outbox.table.field.event.key": "aggregat_id",
    "transforms.outbox.route.by.field": "typ",
    "transforms.outbox.route.topic.replacement": "bestand.${routedByValue}",
    "key.converter": "org.apache.kafka.connect.storage.StringConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter.schemas.enable": "false"
  }
}
```

Der Event Router nimmt die Spalte `aggregat_id` (die Vertragsnummer) als Kafka-Key und leitet nach der Spalte `typ` auf Topics wie `bestand.vertragsaenderungen`. Passwörter stehen nicht in der Konfiguration, sondern kommen über einen Config Provider aus einer Datei oder einem Secret-Store.

12.3.1 Hintergrund und Konsequenzen

- **Reihenfolge:** Debezium liefert Änderungen in Commit-Reihenfolge der Datenbank; mit der Vertragsnummer als Key bleibt die Reihenfolge je Vertrag in Kafka erhalten.
- **Zustellung:** at-least-once. Nach einem Neustart kann Debezium Änderungen erneut liefern – Consumer verarbeiten idempotent.
- **Datenbank-Seite:** CDC braucht Rechte und Einstellungen in der Datenbank (logische Replikation, Supplemental Logging). Das ist mit dem Datenbank-Betrieb abzustimmen.
- **Outbox aufräumen:** Debezium liest das Log, nicht die Tabelle. Alte Outbox-Zeilen können sofort oder per Job gelöscht werden.

12.4 Ins Reporting schreiben: JDBC Sink

```
{
  "name": "reporting-schaeden",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSinkConnector",
    "topics": "schaden.meldungen",
    "connection.url": "jdbc:postgresql://reporting-db:5432/reporting",
    "connection.user": "connect",
    "connection.password": "${file:/opt/connect/geheim.properties:reporting}",
    "insert.mode": "upsert",
    "pk.mode": "record_key",
    "pk.fields": "schadennummer",
    "auto.create": "false",
    "tasks.max": "3",
    "errors.tolerance": "all",
    "errors.deadletterqueue.topic.name": "connect.reporting-schaeden-dlq",
    "errors.deadletterqueue.context.headers.enable": "true"
  }
}
```

`upsert` mit dem Key als Primärschlüssel macht den Sink idempotent: Eine doppelt gelieferte Schadenmeldung überschreibt nur dieselbe Zeile. `errors.tolerance=all` mit Dead-Letter-Queue lässt den Connector bei kaputten Nachrichten weiterlaufen – dieselbe Abwägung wie in Kapitel 9: Für unabhängige Datensätze passend, für abhängige Änderungen nicht.

12.5 Connect über die REST-API bedienen

Aufruf	Wirkung
`GET /connectors`	alle Connectoren
`POST /connectors` (JSON wie oben)	Connector anlegen
`PUT /connectors/<name>/config`	Konfiguration anlegen oder ändern (idempotent – gut für Pipelines)
`GET /connectors/<name>/status`	Zustand des Connectors und jeder Task
`POST /connectors/<name>/restart?includeTasks=true&onlyFailed=true`	fehlgeschlagene Tasks neu starten
`PUT /connectors/<name>/pause`, `/resume`	anhalten, fortsetzen
`DELETE /connectors/<name>`	entfernen

```
curl.exe -s http://connect.intern:8083/connectors/bestand-outbox/status
```

12.6 Woran sehe ich das im Betrieb?

Frage	Wo
Läuft jede Task?	REST …/`status`: `RUNNING` vs. `FAILED`; JMX `connector-status`, `task-status`
Wie weit hängt ein Sink hinterher?	Consumer Group `connect-<name>`: Lag wie bei jedem Consumer
Wie viele Fehler?	JMX `task-error-metrics` (`total-record-errors`, `deadletterqueue-produce-requests`), Größe der DLQ
Wie aktuell ist CDC?	Debezium-Metrik `MilliSecondsBehindSource`

Eine Task im Zustand `FAILED` startet nicht von selbst neu. Ein Alarm darauf gehört in jede Connect-Installation.

12.7 Zusammenfassung

- Connect verbindet Datenbanken, Queues und Speicher mit Kafka über Konfiguration.
- Distributed-Modus für Produktion; interne Topics mit RF 3 und Compaction.
- Debezium + Outbox ist das zuverlässigste Relay vom Bestandssystem nach Kafka.
- Sinks idempotent konfigurieren (`upsert`); Fehlerbehandlung bewusst wählen; `FAILED`-Tasks alarmieren.

12.8 Weiterführende Links

- Kafka Connect, Dokumentation: https://kafka.apache.org/documentation/#connect
- Debezium: https://debezium.io/documentation/

13 Kafka-Anwendungen testen

Ob die Bestandsverwaltung eine doppelt gelieferte Änderung erkennt oder die Police erst nach Antrag und Prüfung entsteht, lässt sich automatisiert prüfen – auf drei Ebenen: ohne Kafka, mit einem eingebetteten Broker und mit einem echten Broker im Container.

13.1 Die Testpyramide für Kafka-Anwendungen

Ebene	Werkzeug	Prüft	Dauer
Unit-Test	JUnit, Mockito	Fachlogik ohne Kafka: Versionsprüfung, Berechnung	Millisekunden
Topologie-Test	`TopologyTestDriver`	Kafka-Streams-Logik ohne Broker	Millisekunden
Integrationstest	`@EmbeddedKafka` (spring-kafka-test)	Listener, Serialisierung, Commits gegen einen Broker in der JVM	Sekunden
Integrationstest	Testcontainers mit `apache/kafka`	wie oben, gegen das echte Broker-Image der Produktion	Sekunden bis eine halbe Minute

Die Fachlogik gehört in Klassen, die man ohne Kafka testen kann. Der Listener ruft sie nur auf.

Die Beispiele zu diesem Kapitel

Das Modul `labs/13-tests` enthält genau die Tests dieses Kapitels – gegen die Verbuchung aus `08-consumer-json` und die Topologien aus `11-streams`. Die Fehlerfälle liegen bei ihren Modulen: `AnhaltenIT` und `DeadLetterIT` in `09a-fehlerbehandlung`, `OutboxIT` in `09b-outbox`, `TransaktionIT` in `09c-transaktionen`, `BrokerNichtErreichbarIT` in `07-producer-json`.

13.2 Abhängigkeiten

```
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka-test</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-testcontainers</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-kafka</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-junit-jupiter</artifactId>
```

```
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-streams-test-utils</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.awaitility</groupId>
    <artifactId>awaitility</artifactId>
    <scope>test</scope>
</dependency>
```

Die Versionen verwaltet Spring Boot 4.1 (Testcontainers 2). In Testcontainers 1 hießen die Artefakte `kafka` und `junit-jupiter`, und die Klasse `KafkaContainer` lag in `org.testcontainers.containers` und nutzte das Confluent-Image; in Testcontainers 2 liegt sie in `org.testcontainers.kafka` und nutzt `apache/kafka`.

test oder verify?

Maven führt `*Test.java` in der Phase `test` aus (Surefire) und `*IT.java` erst in `verify` (Failsafe, muss im POM eingetragen sein). So laufen die schnellen Tests bei jedem Build, die Integrationstests mit Broker nur auf Wunsch: `./mvnw test` gegenüber `./mvnw verify`.

13.3 Fachlogik ohne Kafka

```
class VersionspruefungTest {

    private final Bestand bestand = new Bestand();   // im Speicher
    private final Verbuchung verbuchung = new Verbuchung(bestand);

    @Test
    void duplikatWirdUebersprungen() {
        verbuchung.verarbeite(aenderung("V-1001", 1));
        // zweite Zustellung derselben Änderung (at-least-once)
        Verbuchung.Ergebnis ergebnis =
                verbuchung.verarbeite(aenderung("V-1001", 1));

        assertThat(ergebnis).isEqualTo(Verbuchung.Ergebnis.DUPLIKAT);
        assertThat(bestand.letzteVersion("V-1001")).isEqualTo(1);
        assertThat(verbuchung.duplikate()).isEqualTo(1);
    }

    @Test
    void lueckeWirdErkannt() {
        verbuchung.verarbeite(aenderung("V-1001", 1));
        // v2 fehlt
        Verbuchung.Ergebnis ergebnis =
```

```
                verbuchung.verarbeite(aenderung("V-1001", 3));

        assertThat(ergebnis).isEqualTo(Verbuchung.Ergebnis.LUECKE);
        assertThat(bestand.letzteVersion("V-1001")).isEqualTo(3);
    }

    private static Vertragsaenderung aenderung(String vertrag, int v) {
        return new Vertragsaenderung(vertrag, v, "ADRESSE",
                Map.of("plz", "50667"), Instant.now());
    }
}
```

13.4 Integrationstest mit eingebettetem Broker

```
@SpringBootTest(classes = ConsumerApp.class, properties = {
        "spring.kafka.bootstrap-servers=${spring.embedded.kafka.brokers}",
        "spring.kafka.consumer.auto-offset-reset=earliest",
        "spring.kafka.producer.value-serializer="
                + "org.springframework.kafka.support.serializer"
                + ".JacksonJsonSerializer",
        "spring.kafka.producer.properties.spring.json.add.type.headers"
                + "=false"
})
@EmbeddedKafka(partitions = 3, topics = "bestand.vertragsaenderungen")
class BestandsverwaltungIT {

    @Autowired KafkaTemplate<String, Object> kafka;
    @Autowired Bestand bestand;
    @Autowired Verbuchung verbuchung;

    @Test
    void aenderungenWerdenInReihenfolgeVerbuchtUndDuplikatErkannt() {
        kafka.send("bestand.vertragsaenderungen", "V-1001",
                aenderung(1, "ADRESSE"));
        kafka.send("bestand.vertragsaenderungen", "V-1001",
                aenderung(2, "BANKVERBINDUNG"));
        kafka.send("bestand.vertragsaenderungen", "V-1001",
                aenderung(2, "BANKVERBINDUNG"));   // Duplikat

        await().atMost(Duration.ofSeconds(15)).untilAsserted(() -> {
            assertThat(bestand.letzteVersion("V-1001")).isEqualTo(2);
            assertThat(verbuchung.duplikate()).isEqualTo(1);
        });
    }

    @Test
    void kaputtesJsonWirdUebersprungen() {
```

```
        kafka.send("bestand.vertragsaenderungen", "V-2001",
                "das ist kein JSON");
        kafka.send("bestand.vertragsaenderungen", "V-2001",
                aenderung("V-2001", 1, "ADRESSE"));

        await().atMost(Duration.ofSeconds(15)).untilAsserted(() ->
                assertThat(bestand.letzteVersion("V-2001")).isEqualTo(1));
    }
}
```

`@EmbeddedKafka` startet einen KRaft-Broker in der Test-JVM; `spring.embedded.kafka.brokers` enthält seine Adresse. Der Test überschreibt den Producer-Serializer, weil die Consumer-Anwendung selbst keinen konfiguriert. Asynchrone Verarbeitung wird mit Awaitility abgewartet, nie mit `Thread.sleep`.

13.5 Integrationstest mit Testcontainers

Ein Container mit demselben Image wie im Lab und in Produktion:

```
@SpringBootTest(classes = ConsumerApp.class,
        properties = { /* Serializer wie oben */ })
@Testcontainers
class BestandsverwaltungContainerIT {

    @Container
    @ServiceConnection
    static KafkaContainer kafka =
            new KafkaContainer("apache/kafka:4.4.0");

    @Autowired KafkaTemplate<String, Object> template;
    @Autowired Bestand bestand;

    @Test
    void verbuchtAenderung() {
        template.send("bestand.vertragsaenderungen", "V-1001",
                new Vertragsaenderung("V-1001", 1, "ADRESSE",
                        Map.of("plz", "50667"), Instant.now()));

        await().atMost(Duration.ofSeconds(30)).untilAsserted(() ->
                assertThat(bestand.letzteVersion("V-1001")).isEqualTo(1));
    }
}
```

`@ServiceConnection` setzt `spring.kafka.bootstrap-servers` automatisch auf den Container. Testcontainers braucht ein laufendes Docker – auf den Seminarrechnern ist das vorhanden.

	`@EmbeddedKafka`	Testcontainers
Voraussetzung	nichts	Docker
Start	schnell	einige Sekunden (Image einmalig laden)
Nähe zur Produktion	Broker-Code derselben Client-Version	genau das Broker-Image der Produktion
Gut für	viele schnelle Tests	wenige, realistische Tests; Broker-Verhalten wie Transaktionen, Security

13.6 Fehlerfälle testen

Die interessanten Tests prüfen, was bei Fehlern passiert:

```
@Test
void poisonPillHaeltDieVerarbeitungAn() {
    kafka.send(topic, "V-1001", new Vertragsaenderung("V-1001", 1,
            "ADRESSE", Map.of("plz", "50667"), Instant.now()));
    kafka.send(topic, "V-1001", new Vertragsaenderung("V-1001", 2,
            "TARIF", Map.of("tarif", "UNBEKANNT"), Instant.now()));
    kafka.send(topic, "V-1001", new Vertragsaenderung("V-1001", 3,
            "KUENDIGUNG", Map.of(), Instant.now()));

    await().atMost(Duration.ofSeconds(20)).untilAsserted(() ->
            assertThat(registry.getListenerContainer("bestand")
                    .isRunning()).isFalse());

    // v1 verbucht, v2 scheitert, v3 NICHT verbucht: Reihenfolge gewahrt
    assertThat(bestand.letzteVersion("V-1001")).isEqualTo(1);
}
```

Die Backoff-Zeiten kommen aus Properties (`app.backoff.start-ms`), damit der Test sie auf 100 ms setzen kann – in der Anwendung bleiben es Sekunden. Weitere Fälle, alle in `labs` umgesetzt:

Fall	Test	Modul
Poison Pill mit DLT: Folgeänderung überholt, Header im DLT stimmen	`DeadLetterIT`	09a
vorübergehender Fehler: dritter Versuch klappt	`DeadLetterIT`	09a
`@RetryableTopic`: vier Versuche, dann `@DltHandler`	`DeadLetterIT`	09a
kaputtes JSON wird übersprungen, nächste Nachricht verarbeitet	`BestandsverwaltungIT`	13
Rollback nimmt Vertrag und Outbox-Eintrag zurück	`OutboxIT`	09b
abgebrochene Transaktion für `read_committed` unsichtbar	`TransaktionIT`	09c
Broker nicht erreichbar: Fehler wird gemeldet, nicht verschluckt	`BrokerNichtErreichbarIT`	07
Drahtformat: JSON ohne Typ-Header, gleicher Key → gleiche Partition	`BestandssystemIT`	07
verschlüsseltes Feld, Crypto-Shredding	`FeldverschluesselungTest`	16

Noch offen und lohnend: ein Ereignis mit unbekanntem Zusatzfeld (Schema-Evolution, Kapitel 10) und ein Absturz zwischen Verarbeitung und Commit (Listener werfen lassen, Container neu starten, Duplikat erwarten).

13.7 Kafka Streams ohne Broker testen

```
@Test
void antragUndPruefungErgebenPolice() {
    StreamsBuilder builder = new StreamsBuilder();
    Topologien.antragUndPruefung(builder, Duration.ofMinutes(5));

    Properties config = new Properties();
    config.put(StreamsConfig.APPLICATION_ID_CONFIG, "test");
    config.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG,
            Serdes.StringSerde.class);
    config.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG,
            Serdes.StringSerde.class);

    try (var treiber = new TopologyTestDriver(builder.build(), config)) {
        var antraege = treiber.createInputTopic("antraege",
                new StringSerializer(), new StringSerializer());
        var pruefungen = treiber.createInputTopic("pruefungen",
                new StringSerializer(), new StringSerializer());
        var policen = treiber.createOutputTopic("policen",
                new StringDeserializer(), new StringDeserializer());

        antraege.pipeInput("V-2001", "Antrag KFZ",
                Instant.parse("2026-11-03T09:00:00Z"));
        pruefungen.pipeInput("V-2001", "geprüft",
                Instant.parse("2026-11-03T09:03:00Z"));
```

```
        pruefungen.pipeInput("V-2002", "geprüft",
                Instant.parse("2026-11-03T09:04:00Z"));   // kein Antrag

        assertThat(policen.readKeyValuesToList())
                .containsExactly(KeyValue.pair("V-2001",
                        "Police aus Antrag \"Antrag KFZ\""
                        + " und Prüfung \"geprüft\""));
    }
}
```

Der `TopologyTestDriver` verarbeitet synchron und mit kontrollierter Zeit: Zeitstempel werden beim `pipeInput` mitgegeben, Zeitfenster lassen sich damit exakt testen. `TopologienTest` in `11-streams` prüft so auch, dass eine Prüfung nach sechs Minuten keine Police mehr ergibt, dass die Anreicherung einen unbekannten Kunden verwirft und dass drei Schäden in einem 24-Stunden-Fenster eine Auffälligkeit auslösen. Für Aggregationen gehört `statestore.cache.max.bytes=0` in die Testkonfiguration, sonst hält der Cache Zwischenergebnisse zurück.

13.8 Verträge zwischen Teams prüfen

Wenn das Bestandssystem ein Feld umbenennt, merkt das Inkasso es erst in Produktion – es sei denn, ein Test prüft den Vertrag:

- **Mit Schema Registry:** Kompatibilitätsprüfung in der Pipeline, bevor ein neues Schema registriert wird.
- **Ohne Registry:** Beispielnachrichten des Producers als Testdaten im Consumer-Projekt; der Consumer-Test liest sie. Änderungen am Producer laufen erst durch, wenn die Beispiele aktualisiert und die Consumer-Tests grün sind.

13.9 Zusammenfassung

- Fachlogik ohne Kafka testen; Listener bleiben dünn.
- `@EmbeddedKafka` für schnelle Integrationstests, Testcontainers mit `apache/kafka:4.4.0` für realistische.
- Fehlerfälle gezielt testen: Poison Pill, Duplikat, Deserialisierung, Broker weg.
- Kafka Streams mit `TopologyTestDriver` und kontrollierter Zeit.

13.10 Weiterführende Links

- Spring for Apache Kafka, Testing: https://docs.spring.io/spring-kafka/reference/testing.html
- Testcontainers Kafka: https://java.testcontainers.org/modules/kafka/
- Kafka Streams, Testing: https://kafka.apache.org/documentation/streams/developer-guide/testing.html

14 Monitoring und Alerting

Kafka gleicht Ausfälle aus. Das ist gut – und gefährlich: Mit jedem ausgefallenen Broker sinkt die Reserve, ohne dass eine Anwendung es merkt. Monitoring zeigt, wie viel Reserve noch da ist, hilft bei der Ursachensuche und weckt jemanden, wenn Vertragsänderungen nicht mehr ankommen.

Die Beispiele zu diesem Kapitel

`14-monitoring` ist der Consumer aus Kapitel 8 mit Actuator und Micrometer: Kafka-Client-Metriken und die eigenen Zähler `Verbuchungsmetriken` unter `/actuator/prometheus`; das Lab-Prometheus holt sie ab. Die Monitoring-Befehle des Seminars (Gesundheitscheck, JMX, PromQL, Alerts, Lag erzeugen, alle Kennzahlen aus Kapitel 6 in einem Lauf: `08-metriken-ansehen`) liegen als Skripte in `14-monitoring/shell`.

Grundlage: Buch Zelenin/Kropp Kap. 15 und Anhang B; Kafka-Dokumentation 4.4

14.1 Die Metrik-Landkarte

Ebene	Was	Woher
Von außen	Lag der Consumer Groups, Ende-zu-Ende-Zeit	`kafka-consumer-groups.sh`, Exporter, Kafbat UI, Synthetik-Test
Clients	Producer-Fehler, Puffer, Consumer-Lag, Poll-Abstände	Micrometer in Spring Boot, JMX der Clients
Broker und KRaft	Replikation, Requests, Controller, Speicher	JMX der Broker
Infrastruktur	CPU, Arbeitsspeicher und Page Cache, Platte, Netz	Betriebssystem; Platten-I/O seit Kafka 4.4 auch per JMX

Bei einer Störung von unten nach oben prüfen. Viele Kafka-Symptome sind in Wahrheit eine volle Platte oder ein Netzwerkproblem. Typische Kette: Platte langsam → hohe lokale Zeit bei Produce → Follower kommen nicht hinterher → ISR schrumpft → `acks=all`-Latenz steigt → Producer-Puffer voll → das Bestandssystem meldet Timeouts.

14.2 Woher kommen die Metriken?

Kafka veröffentlicht seine Metriken über JMX. Im Lab läuft – wie im Buch, Anhang B – der Prometheus-JMX-Exporter als Java-Agent im Broker und stellt sie als HTTP-Seite bereit. Prometheus holt sie alle 15 Sekunden ab, Grafana zeigt sie an, der Alertmanager verschickt Alarme.

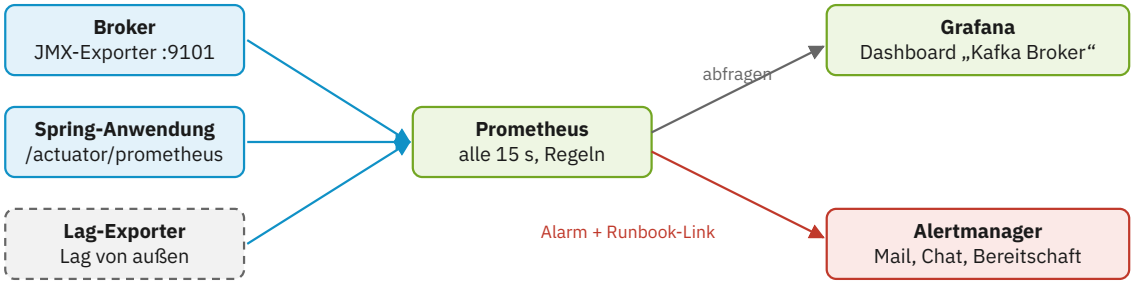


Abbildung 12: Von der Metrik zum Alarm

Ohne Prometheus geht es mit dem Bordmittel `kafka-jmx.sh`, praktisch wenn das Monitoring selbst ausgefallen ist:

```
./kafka-jmx.sh --jmx-url service:jmx:rmi:///jndi/rmi://kafka1:9999/jmxrmi --
object-name kafka.server:type=ReplicaManager,name=UnderReplicatedPartitions --one-
time true
```

14.2.1 Hintergrund und Konsequenzen

- **Java-Agent statt Remote-JMX:** Der Exporter liest die MBeans lokal; es muss kein JMX-Port nach außen offen sein. Offener Remote-JMX ohne Authentifizierung ist ein Sicherheitsrisiko.
- **Kardinalität:** Viele Metriken gibt es pro Topic und Partition. Bei Tausenden Partitionen wird Prometheus langsam – in der Exporter-Konfiguration gezielt filtern.
- **Fehlende Daten lösen keine Alarme aus.** Fällt der Exporter aus, schweigt jede Regel. Deshalb gibt es immer einen Alarm auf die Erreichbarkeit (`up == 0`).

14.3 Metriknamen lesen

Aus dem MBean-Namen wird in Prometheus ein Name in Kleinbuchstaben mit Unterstrichen: Domäne, Typ, Name.

```
kafka.server:type=ReplicaManager,name=UnderReplicatedPartitions
→ kafka_server_replicamanager_underreplicatedpartitions
```

Zusätzliche Teile des MBean-Namens (`topic=…`, `partition=…`) werden zu Labels. Die häufigsten Domänen: `kafka_server`, `kafka_controller`, `kafka_network`, `kafka_log`. Zähler (steigen nur) immer mit `rate()` oder `increase()` auswerten, Gauges direkt vergleichen.

14.4 Zustand: diese Werte müssen 0 bzw. 1 sein

Metrik	Soll	Bedeutung bei Abweichung
`UnderReplicatedPartitions` (summiert)	0	konstant: ein Broker fehlt; schwankend: ein Broker hat ein Ressourcenproblem
`UnderMinIsrPartitionCount`	0	Producer mit `acks=all` werden abgewiesen – das Bestandssystem steht
`AtMinIsrPartitionCount`	0	es geht noch, aber ohne Reserve
`OfflinePartitionsCount`	0	Partitionen ohne Leader – dort geht nichts
`ActiveControllerCount` (summiert)	1	0: Metadaten eingefroren
`PreferredReplicaImbalanceCount`	0	Last schief verteilt; `kafka-leader-election.sh`
`OfflineLogDirectoryCount`	0	eine Platte ist ausgefallen

14.4.1 Hintergrund und Konsequenzen

- **Wer meldet was:** UnderReplicated und UnderMinIsr meldet jeder Broker für seine Leader-Partitionen. Ein gestoppter Broker meldet nichts – die anderen melden seine fehlenden Kopien. Offline-Partitionen meldet nur der aktive Controller.
- **Schuldigen finden:** `kafka-topics.sh --describe --under-replicated-partitions` zeigt, welche Replik fehlt. Taucht immer dieselbe Broker-ID auf, ist es dieser Broker.
- **Ausgesperrt (fenced):** Ein Broker, der dem Controller keine Heartbeats mehr schickt (`broker.session.timeout.ms`, 9 s), bekommt keine Leader mehr – er kann trotzdem noch laufen.

14.5 Auslastung und Requests

Metrik	Bedeutung	Faustregel
`NetworkProcessorAvgIdlePercent`	Anteil freier Netzwerk-Threads (0–1)	unter 0,3 wird es eng
`RequestHandlerAvgIdlePercent`	Anteil freier Request-Handler (0–1)	unter 0,3 wird es eng
Produce-Purgatory	Requests mit `acks=all`, die auf Follower warten	steigt mit langsamen Followern
Fetch-Purgatory	Consumer im Long Polling	groß ist normal

14.5.1 Produce-Requests nach Zeitanteil zerlegen

Klagt das Bestandssystem über Latenz, zeigt die Zerlegung eines Produce-Requests die Ursache:

Größter Anteil	Ursache
Warten in der Request-Queue	Request-Handler überlastet
lokale Zeit	Platte langsam
Remote-Zeit	Follower langsam; `acks=all` wartet auf sie
Throttle-Zeit	eine Quota bremst
Antwort-Queue und Senden	Netzwerk-Threads oder der Client

Perzentile statt Mittelwert: Das 99. Perzentil zeigt, was die langsamsten Requests erleben – und genau die führen zu Timeouts.

14.6 Replikation, KRaft und Speicher

- `MaxLag` (in Nachrichten, je Broker): dauerhaft wachsend heißt, der Broker kann nicht so schnell replizieren, wie geschrieben wird.
- `IsrShrinksPerSec` / `IsrExpandsPerSec`: außerhalb von Wartung sollten sie null sein.
- **KRaft:** die Commit-Latenz des Quorums und wie weit ein Broker beim Anwenden der Metadaten hinterherhängt. `kafka-metadata-quorum.sh describe --replication` zeigt den Lag jedes Knotens.
- **Größe je Partition** und `RetentionSizeInPercent` (neu in Kafka 4.3): wie weit eine Partition ihr `retention.bytes` ausschöpft. Wichtig für Topics, die als Puffer dienen (Kapitel 18).
- **Platten-I/O** meldet Kafka seit 4.4 pro Broker selbst über JMX.

retention.bytes gilt pro Partition

Speicherbedarf eines Topics = `retention.bytes` × Partitionen × Replication Factor. 10 GB × 6 × 3 = 180 GB. Ein häufiger Planungsfehler.

14.7 Client-Metriken

14.7.1 Producer

Metrik	Bedeutung
`record-error-rate`	muss 0 sein – sonst wurden Nachrichten nach allen Wiederholungen aufgegeben
`record-retry-rate`	Wiederholungen; bei Leader-Wechseln normal
`request-latency-avg`, `record-queue-time-avg`	wo Zeit vergeht
`buffer-available-bytes`, `bufferpool-wait-time`	Puffer voll ⟶ `send()` blockiert gleich
`batch-size-avg`, `records-per-request-avg`, `compression-rate-avg`	wirkt das Tuning? (Kapitel 15)
`produce-throttle-time-avg`	eine Quota bremst

14.7.2 Consumer

Metrik	Bedeutung
`records-lag-max`	größter Lag über die Partitionen dieses Consumers
`records-lead-min`	Abstand zum Anfang des Logs; nahe 0 heißt: die Retention löscht gleich Ungelesenes
`time-between-poll-max`	nähert es sich `max.poll.interval.ms`, droht der Rauswurf
`poll-idle-ratio-avg`	nahe 0: der Consumer ist voll ausgelastet
`rebalance-rate-per-hour`	häufige Rebalances deuten auf Instabilität

Steht ein Consumer, verschwindet seine Metrik. Dann hilft nur der Blick von außen.

14.8 Spring Boot: Metriken mit Micrometer

Spring Boot veröffentlicht die Kafka-Client-Metriken über Micrometer, sobald Actuator und eine Registry da sind:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

```
management.endpoints.web.exposure.include=health,info,prometheus
management.metrics.tags.application=bestandsverwaltung
# Spans und Timer je gesendeter bzw. verarbeiteter Nachricht
spring.kafka.template.observation-enabled=true
spring.kafka.listener.observation-enabled=true
```

Unter `/actuator/prometheus` erscheinen dann Metriken wie `kafka_consumer_fetch_manager_records_lag_max` und `kafka_producer_record_error_rate`, jeweils mit `client_id`. `14-monitoring` und `18-fallstudie` sind so konfiguriert; Prometheus holt ihre Metriken im Lab unter `host.docker.internal:8085` bzw. `:8081`.

Fachliche Kennzahlen gehören ebenfalls dazu. Die Kafka-Metriken sagen, wie viel gelesen wird, nicht ob die Verarbeitung fachlich gut ging. `Verbuchungsmetriken` aus `14-monitoring` zählt deshalb selbst – gleicher Name, verschiedener Tag `ergebnis`, damit PromQL summieren und filtern kann:

```
@Component
public class Verbuchungsmetriken {

    private final Counter verbucht;
    private final Counter duplikate;
    private final Timer dauer;

    public Verbuchungsmetriken(MeterRegistry registry) {
        verbucht = registry.counter("beitraege.verbuchungen",
                "ergebnis", "verbucht");
        duplikate = registry.counter("beitraege.verbuchungen",
                "ergebnis", "duplikat");
        dauer = registry.timer("beitraege.verbuchung.dauer");
    }

    public void verbucht() { verbucht.increment(); }
    public void duplikat() { duplikate.increment(); }
    public Timer dauer() { return dauer; }
}
```

Der Listener meldet jedes Ergebnis und misst die Dauer mit `metriken.dauer().record(() -> …)`. In Prometheus heißen die Zähler `beitraege_verbuchungen_total{ergebnis="verbucht"}` und der Timer `beitraege_verbuchung_dauer_seconds_count`, `_sum` und `_max`. Der Test `MetrikenIT` schreibt drei Nachrichten (eine doppelt) und prüft beide Zähler und den Endpunkt. Ein Alert `rate(beitraege_verbuchungen_total[5m]) == 0` erkennt einen stehenden Consumer auch dann, wenn die Lag-Metrik mit ihm verschwunden ist.

14.9 Lag von außen messen

Damit der Lag auch sichtbar bleibt, wenn ein Consumer steht, fragt ein eigener Dienst die Offsets der Gruppen beim Broker ab und stellt sie als Metrik bereit:

Werkzeug	Bemerkung
Kafka Exporter	Prometheus-Exporter für Lag je Gruppe und Partition; in Strimzi direkt einschaltbar
KMinion	Exporter mit Lag in Nachrichten und als Zeit, Ende-zu-Ende-Messung
Burrow	bewertet den Lag-Verlauf (wachsend, stehend) statt fester Schwellen
`kafka-consumer-groups.sh`	für Skripte und die Fehlersuche

14.10 Verarbeitung nachverfolgen

Wo ist die Vertragsänderung V-1001 v7 gerade? Drei Mittel:

- **Trace-ID im Header:** Die Quelle schreibt eine Korrelations- bzw. Trace-ID in den Header; jeder Verarbeitungsschritt gibt sie weiter und schreibt sie ins Log.
- **OpenTelemetry:** Mit `spring.kafka.template.observation-enabled` und `spring.kafka.listener.observation-enabled` (plus Micrometer Tracing) setzt Spring den Header `traceparent` automatisch und erzeugt Spans über Producer, Topic und Consumer – ein durchgängiger Trace, sichtbar in Tempo, Jaeger oder einem kommerziellen APM.
- **Fachliche Status-Ereignisse:** Für die fachliche Nachverfolgung schreibt jeder Schritt ein Status-Ereignis in ein eigenes Topic („empfangen“, „geprüft“, „verbucht“, „abgelehnt“). Daraus lässt sich für jeden Vertrag die Bearbeitungskette lesen.

14.11 Monitoring mit Dynatrace und Splunk

Beim Kunden laufen Dynatrace und Splunk. Beide können den Stack aus dem Lab (JMX-Exporter, Prometheus, Alertmanager, Grafana) ersetzen, aber an verschiedenen Stellen: Dynatrace ersetzt das Monitoring samt Alerting und Traces, Splunk sammelt Logs, Metriken und Audit-Ereignisse und lässt darin suchen. Die Regeln dieses Kapitels bleiben unverändert: dieselben Metriken, dieselben Schwellen, Lag von außen, Alarm auf Symptome, Runbook je Alert.

Rolle im Lab	Dynatrace	Splunk
Broker-Metriken sammeln	OneAgent auf den Broker-Knoten, Extension „Apache Kafka“ (JMX)	Splunk OpenTelemetry Collector mit `kafkametrics`- und `jmx`-Receiver oder Prometheus-Scraping des Exporters
Client-Metriken der Spring-Apps	OneAgent im Pod instrumentiert den Kafka-Client automatisch; Micrometer über `micrometer-registry-dynatrace`	Collector holt `/actuator/prometheus` ab
Lag von außen	Kafka-Extension (Gruppen-Lag über die Admin-API); wo die Extension-Version das nicht liefert, Kafka Exporter	`kafkametrics`-Receiver (Consumer-Group-Lag) oder Kafka Exporter
Logs	OneAgent (Log Monitoring)	HEC oder Collector, strukturierte Logs
Traces	PurePath, Spring Kafka wird automatisch erkannt	Splunk APM über OpenTelemetry
Alerting	Davis (Baseline) plus feste Regeln	Detectors oder gespeicherte Suchen
Dashboards	Dynatrace-Dashboards, fertige Kafka-Ansichten der Extension	Splunk Observability oder klassische Dashboards
Topic-Inhalte auswerten	nicht vorgesehen	Splunk Connect for Kafka (Connect-Sink zum HEC)

14.11.1 Dynatrace: empfohlene Konfiguration

Bereich	Einstellung	Warum
Broker	OneAgent auf jedem Broker- und Controller-Knoten; bei OpenShift-Betrieb den Dynatrace Operator mit `cloudNativeFullStack`	erkennt den Kafka-Prozess, liest JMX ohne Exporter
Extension	„Apache Kafka“ aktivieren, Feature-Sets Broker, Controller, Replication, Log, Consumer Group Lag; Abfrageintervall 1 Minute	liefert die Metriken aus Tabelle 15.1 bis 15.4 unter `kafka.server.*`, `kafka.controller.*`, `kafka.consumer.*`
Confluent Cloud	statt OneAgent die Integration über die Confluent Metrics API	dort gibt es keinen Broker-Host
Spring-Boot-Pods	OneAgent über den Operator injizieren; `spring.kafka.*.client-id` je Instanz setzen	Kafka-Client-Metriken und PurePaths ohne Code; `client-id` wird zur Dimension
Eigene Zähler	`micrometer-registry-dynatrace` (OneAgent-Endpunkt, kein Token im Code) oder OpenTelemetry Metrics	`beitraege.verbuchungen` erscheint neben den Kafka-Metriken
Traces	Standard lassen; Header `traceparent` setzt der OneAgent selbst; keine eigene Korrelations-ID nötig	eine Nachricht von `send()` bis in den Listener verfolgen
Alerting	die Regeln aus `rules.yml` als Metric Events anlegen: `UnderMinIsr` und `ControllerMissing` als feste Schwelle, sofort; `UnderReplicatedPartitions` 5 Minuten; `ConsumerLagHigh` als Schwelle plus Davis-Baseline	Davis findet Abweichungen, die harten Grenzen bleiben Regeln
Management Zone	je Kafka-Cluster und je Anwendungsteam	Alerts und Dashboards landen beim richtigen Team
Problem-Benachrichtigung	Ticket oder Chat mit Runbook-Link in der Beschreibung des Metric Events	Dynatrace nennt die Ursache, das Runbook die Maßnahme

Was entfällt: JMX-Exporter, Prometheus, Alertmanager und Grafana. Was bleibt: `kafka-consumer-groups.sh` für die Fehlersuche und die Shell-Skripte aus `labs/14-monitoring`.

14.11.2 Splunk: empfohlene Konfiguration

Bereich	Einstellung	Warum
Collector	Splunk OpenTelemetry Collector als Daemon auf den Broker-Knoten und als Deployment auf OpenShift	ein Sammler für Metriken, Logs und Traces
Broker-Metriken	`jmx`-Receiver gegen Port 9999 mit dem Kafka-Target-System, oder `prometheus`-Receiver gegen den vorhandenen Exporter auf 9101	der zweite Weg lässt das Lab-Setup unverändert
Lag von außen	`kafkametrics`-Receiver mit den Scrapern `brokers`, `topics`, `consumers`	Consumer-Group-Lag je Partition, auch wenn kein Consumer läuft
Spring-Boot-Pods	`prometheus`-Receiver gegen `/actuator/prometheus`; Micrometer bleibt	dieselben Metriken wie im Lab
Logs	Spring-Apps loggen JSON mit den Feldern `anwendung`, `instanz`, `topic`, `partition`, `offset`, `vertrag`, `korrelationsId`; Collector oder HEC nach Splunk	„Wo ist V-1001 v7?“ wird eine Suche über alle Pods
Topic-Inhalte	Splunk Connect for Kafka in einem eigenen Connect-Cluster, nur für Audit- und Status-Topics, eigener `splunk.hec.token`, `splunk.hec.ack.enabled=true`	Ereignisse zuverlässig im HEC, Lag der Connect-Gruppe überwachen
Alerting	die Regeln aus `rules.yml` als Detectors (Observability) oder gespeicherte Suchen mit Schwelle und Dauer; zusätzlich Log-Alerts wie „mehr als 3 ANHALTEN in 5 Minuten“	Metrik- und Logmuster in einem Werkzeug
Dashboards	Gliederung des Grafana-Dashboards übernehmen: Zustand, Last, Requests, Replikation, Speicher, Clients, DR	dieselbe Lesereihenfolge wie im Seminar

Ein Collector-Ausschnitt für die Metriken, der das Lab ohne Änderung abholt:

```
receivers:
  prometheus:
    config:
      scrape_configs:
        - job_name: kafka
          static_configs:
            - targets: ["kafka1:9101", "kafka2:9101", "kafka3:9101"]
        - job_name: spring-apps
          metrics_path: /actuator/prometheus
```

```
          static_configs:
            - targets: ["bestand:8081", "monitoring:8085"]
  kafkametrics:
    brokers: ["kafka1:9092", "kafka2:9092", "kafka3:9092"]
    scrapers: [brokers, topics, consumers]
exporters:
  signalfx:
    access_token: "${SPLUNK_ACCESS_TOKEN}"
    realm: "${SPLUNK_REALM}"
service:
  pipelines:
    metrics:
      receivers: [prometheus, kafkametrics]
      exporters: [signalfx]
```

Was entfällt: Grafana und Alertmanager; Prometheus wahlweise. Was bleibt: die Bordmittel für die Fehlersuche.

14.11.3 Confluent: empfohlene Konfiguration für das Monitoring

Confluent Platform bringt eigene Monitoring-Bausteine mit; sie ersetzen Exporter und Lag-Messung nicht, sondern ergänzen sie.

Bereich	Einstellung	Warum
Metriken aus dem Broker	JMX-Exporter oder OneAgent wie oben; zusätzlich `confluent.metrics.reporter` eingeschaltet, Ziel-Topic `_confluent-metrics` mit RF 3	Control Center und die Confluent-Alerts lesen aus diesem Topic
Confluent Cloud	Metrics API (`api.telemetry.confluent.cloud`) als Quelle für Prometheus, Dynatrace oder Splunk; Service-Account mit `MetricsViewer`	es gibt keinen Broker zum Instrumentieren
Control Center	für alle Teams nur lesend (RBAC-Rolle `DeveloperRead` oder `Operator` ohne Schreibrechte); Topic-Änderungen bleiben in Terraform	Lag, Gruppen und Topic-Konfiguration ansehen, nicht ändern
Lag	Control Center zeigt Lag je Gruppe von außen; für Alerts zusätzlich `kafkametrics`-Receiver oder Kafka Exporter	Control Center alarmiert nur grob; Schwellen und Runbooks gehören ins zentrale Werkzeug
Audit-Log	`confluent.security.event.logger` nach Splunk (HEC)	wer welches Topic wann angelegt oder gelesen hat
Alerts in Control Center	nur Cluster-weite Grundregeln (Broker down, unter-min-ISR); Anwendungs-Alerts in Dynatrace oder Splunk	kein zweites Alerting-System mit eigenen Schwellen
Dashboards	Confluent-Grafana-Dashboards (Confluent Platform Metrics) als Vorlage, Gliederung wie in diesem Kapitel	dieselbe Lesereihenfolge

14.11.4 Beide zusammen

Die übliche Aufteilung beim Kunden: Dynatrace für Metriken, Topologie, Traces und das Alerting auf Symptome; Splunk für Logs, Audit-Ereignisse und die fachliche Nachverfolgung; Confluent Control Center als Kafka-eigene Sicht auf Topics, Gruppen und Lag; Grafana nur dort, wo Teams eigene Prometheus-Instanzen betreiben. Die zehn Alert-Regeln aus dem Lab sind in beiden Werkzeugen die Vorlage; Schwellen und Wartezeiten sind werkzeugneutral.

14.12 Vom Messwert zum Alarm

Nicht jede Metrik braucht einen Alarm – die meisten dienen der Ursachensuche. Drei Regeln aus dem Buch:

1. **Glätten:** kurze Spitzen mit `for:` ignorieren.
2. **Justieren:** konservativ starten, am echten Verlauf nachschärfen.
3. **Abwarten:** Kafka heilt vieles selbst.

Auf Symptome alarmieren, die Nutzer spüren (Schreibstopp, wachsender Lag, Producer-Fehler); Ursachen gehören ins Dashboard. Wer zu oft grundlos geweckt wird, ignoriert irgendwann auch den echten Alarm.

14.12.1 Die Alert-Regeln des Labs

```
- alert: UnderMinIsr
  expr: sum(kafka_server_replicamanager_underminisrpartitioncount) > 0
  for: 1m
  labels:
    severity: critical
  annotations:
    summary: "Partitionen unter min.insync.replicas – Producer blockiert"
    runbook_url: "https://wiki.intern/runbooks/kafka-underminisr"
```

Regel	Ausdruck	for	Schwere
KafkaMetricsDown	up{job="kafka"} == 0	1 min	kritisch
UnderReplicatedPartitions	Summe > 0	5 min	kritisch
UnderMinIsr	Summe > 0	1 min	kritisch
ControllerMissing	Summe ActiveControllerCount ≠ 1	1 min	kritisch
RequestHandlerBusy	kleinster Idle-Anteil < 0,3	10 min	Warnung
LeaderImbalance	Imbalance > 0	15 min	Warnung
RetentionNearlyFull	RetentionSizeInPercent > 80 je Topic	5 min	Warnung
ConsumerLagHigh	records-lag-max > 1000 je Client	10 min	Warnung
ProducerErrors	record-error-rate > 0 je Client	1 min	kritisch
MirrorMakerLagging	Replikationslatenz > 30 s	5 min	Warnung

14.12.2 Hintergrund und Konsequenzen

- ControllerMissing **kann fälschlich auslösen,** wenn nur der Exporter des aktiven Controllers ausfällt (Summe 0). Immer zusammen mit KafkaMetricsDown lesen; im Alertmanager mit Inhibition Folgealarme unterdrücken.
- **Die Lag-Schwelle 1000 ist willkürlich.** In Produktion je Anwendung festlegen, besser in Zeit: „mehr als 15 Minuten Rückstand“. Für einen Puffer wie die Bestandsverwaltung ist Lag gewollt – Alarm erst, wenn er länger wächst als ein Peak dauert.
- **Der Lag-Alarm löst sich auf, wenn der Consumer stirbt.** Ohne Metrik hat die Regel keine Daten und geht auf „resolved“, obwohl der Lag weiter wächst. Deshalb Lag zusätzlich von außen messen oder auf fehlende Metriken alarmieren (absent()).

14.13 Vom Alarm zur Lösung: das Runbook

Ein Alarm ohne Anleitung hilft nachts um drei niemandem. Jeder Alarm bekommt ein Runbook, verlinkt in der Annotation runbook_url:

Abschnitt	Beispiel `UnderReplicatedPartitions`
Bedeutung	Mindestens eine Partition hat nicht alle Kopien in der ISR
Prüfen	`--describe --under-replicated-partitions`: welche Broker-ID fehlt? Läuft der Broker, ist er ausgesperrt?
Ursachen	Broker gestoppt, Platte voll oder langsam, Netzwerk, lange GC-Pausen
Handeln	Broker starten bzw. Ursache beheben; warten, bis die ISR voll ist; Leader-Wahl
Nicht tun	keinen weiteren Broker neu starten – das nimmt die letzte Reserve
Eskalieren	wenn `UnderMinIsr` dazukommt oder nach 30 Minuten keine Besserung

14.14 Lab: Dashboard lesen und Broker-Ausfall

1. Grafana öffnen: http://localhost:3000, Dashboard „Kafka Broker“. Zeile für Zeile lesen: Was steht wo, ist es gerade gut?
2. Gesundheitscheck per Kommandozeile – alle drei Ausgaben sollen leer sein:

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --describe --under-replicated-partitions
```

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --describe --under-min-isr-partitions
```

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --describe --unavailable-partitions
```

3. `docker stop kafka3` (PowerShell) und beobachten: Unterreplizierte Partitionen steigen, Leader wandern, der Controller bleibt bei 1. `KafkaMetricsDown` feuert nach einer Minute, `UnderReplicatedPartitions` erst nach fünf.
4. `docker start kafka3`, etwa 30 Sekunden warten, dann die Leader neu verteilen:

```
./kafka-leader-election.sh --bootstrap-server kafka1:9092 --election-type preferred --all-topic-partitions
```

14.15 Lab: Lag-Alarm auslösen

Eine Instanz der simulierten Bestandsverwaltung schafft 50 Änderungen pro Sekunde, die Quelle liefert 100. In `prometheus\rules.yml` bei `ConsumerLagHigh` zuerst `for: 10m` auf `for: 1m` setzen und neu laden:

```
curl.exe -X POST http://localhost:9090/-/reload
```

```
bash /opt/lab/bin/mit-metriken 9201 kafka-producer-perf-test.sh --bootstrap-server kafka1:9092 --topic bestand.aenderungen --num-records 100000 --record-size 500 --
```

```
throughput 100 --record-key-range 1000 --key-distribution random --command-
property client.id=quelle
```

```
bash /opt/lab/bin/bestandsverwaltung 1
```

Der Lag wächst um 50 pro Sekunde, nach etwa 20 Sekunden ist die Schwelle 1000 überschritten, eine Minute später feuert der Alarm. Dann die Instanz mit Strg+C beenden: Die Kurve in Grafana verschwindet, der Alarm wird „resolved“ – aber `kafka-consumer-groups.sh --describe --group bestand` zeigt, dass der Lag weiter wächst.

14.16 Zusammenfassung

- Vier Ebenen; bei Störungen von unten nach oben prüfen.
- Die Zustandswerte auswendig kennen: 0 bzw. 1.
- Spring-Anwendungen exportieren Client- und Fachmetriken über Micrometer.
- Alarm auf Symptome, mit `for:` geglättet, mit Runbook; Lag zusätzlich von außen messen.

15 Performance und Tuning

Zum Monatsabschluss erzeugt der Beitragslauf in wenigen Stunden Millionen Vertragsänderungen. Ob Kafka und die Anwendungen das schaffen, entscheiden eine Handvoll Stellschrauben. Die Regel für dieses Kapitel: **messen statt raten.** Jede Einstellung, die wir ändern, sehen wir sofort in den Zahlen des Messwerkzeugs und im Dashboard.

Die Beispiele zu diesem Kapitel

Die Messungen dieses Kapitels (`kafka-producer-perf-test.sh`, Consumer-Test, Ende-zu-Ende-Latenz, Quota) liegen als Skripte in `15-performance/shell`. `LasttestApp` im selben Modul ist der Versuch als Spring-Boot-Producer: dieselben Nachrichten mit anderen Werten für `linger.ms`, `batch.size`, `compression.type` und `acks`, Ergebnis in Nachrichten/s und MB/s.

15.1 Partitionen bestimmen die Parallelität

Pro Partition liest in einer Gruppe höchstens ein Consumer. **Die Partitionsanzahl ist die Obergrenze für die Parallelität.** Das Buch empfiehlt zwölf Partitionen als Ausgangspunkt und bei Bedarf zu verdoppeln. Nach oben gibt es Grenzen: als Richtwert etwa viertausend Partitionen pro Broker; jede Partition kostet Arbeitsspeicher, Dateihandles (die Doku empfiehlt mindestens 100.000 erlaubte Dateien) und Zeit beim Recovery.

15.1.1 Partitionszahl berechnen

Zieldurchsatz geteilt durch den Durchsatz **eines Consumers** – nicht des Brokers. Beispiel Beitragslauf: 600 Änderungen pro Sekunde, eine Instanz der Bestandsverwaltung schafft 50 → mindestens 12 Partitionen, mit Reserve für das Aufholen nach Störungen eher 24. Meist ist die Verarbeitung in der Anwendung der Engpass, nicht Kafka.

Teilbare Zahlen wählen: 12 lässt sich gleichmäßig auf 1, 2, 3, 4, 6 oder 12 Instanzen verteilen. Bei 10 Partitionen und 3 Instanzen bekommt eine 4, die anderen 3 – die eine wird zum Engpass.

15.1.2 Partitionen erhöhen – Vorsicht bei Keys

Die Partition ergibt sich aus Hash des Keys modulo Partitionsanzahl. **Ändert sich die Anzahl, landet derselbe Key in einer anderen Partition.** Der Vertrag V-1001 lag bisher in Partition 1, nach der Erhöhung auf sechs Partitionen in Partition 4. Ungelesene alte Änderungen in Partition 1 und neue in Partition 4 können sich überholen.

Sauberer Umzug auf mehr Partitionen:

1. Neues Topic mit der Zielanzahl anlegen.
2. Producer auf das neue Topic umstellen.
3. Altes Topic leer lesen lassen (Lag = 0).
4. Erst dann die Consumer auf das neue Topic umstellen.

Verringern geht gar nicht – weniger Partitionen heißt immer: neues Topic.

15.2 Producer: Batching

Der Producer schickt nicht jede Nachricht einzeln. Er sammelt pro Partition einen Batch und sendet, wenn `batch.size` voll ist oder `linger.ms` abgelaufen ist.

```
send() → Puffer je Partition → Batch voll oder linger.ms abgelaufen → Request
an den Leader
```

Der Standard von 16 KB ist klein; das Buch empfiehlt bis zu einem Megabyte und ein `linger.ms` um zehn Millisekunden. **Größere Batches bedeuten weniger Requests und mehr Durchsatz – oft sogar weniger Latenz,** weil der Broker weniger Anfragen abarbeiten muss.

Einstellung	Standard	Für den Beitragslauf
`batch.size`	16384	262144 bis 1048576
`linger.ms`	5 (seit Kafka 4.0)	10–50
`compression.type`	`none`	`lz4` oder `zstd`
`buffer.memory`	32 MB	größer bei vielen Partitionen
`max.in.flight.requests.per.connection`	5	5 (mit Idempotenz bleibt die Reihenfolge erhalten)

In Spring Boot:

```
spring.kafka.producer.batch-size=262144
spring.kafka.producer.compression-type=lz4
spring.kafka.producer.properties.linger.ms=20
spring.kafka.producer.buffer-memory=67108864
```

15.2.1 Hintergrund und Konsequenzen

- **Warum weniger Latenz trotz Warten:** Jeder Request kostet beim Broker feste Arbeit. Viele winzige Requests füllen die Request-Queue – dann wartet jede Nachricht länger als die paar Millisekunden `linger.ms`.
- `batch.size` **gilt pro Partition.** Ein Producer, der in 100 Partitionen schreibt, hält bis zu 100 Batches gleichzeitig; `buffer.memory` begrenzt den Gesamtspeicher.
- **Wann Batching nichts bringt:** bei sehr wenigen Nachrichten. Dann ist der Batch nie voll, und `linger.ms` bestimmt allein die zusätzliche Latenz.

15.2.2 Was ein Batch ist, und was nicht

Nach `send()` legt der Producer jede Nachricht in einen Puffer im Speicher der Anwendung, sortiert nach Ziel-Partition. Alles, was sich dort für dieselbe Partition ansammelt, wird zu einem Batch verpackt: ein Header, dann die Nachrichten hintereinander, gemeinsam komprimiert. `batch.size` ist die Obergrenze dieses Pakets in Bytes, `linger.ms` die Obergrenze der Wartezeit; was zuerst eintritt, schickt das Batch ab. Ein Request an den Broker enthält mehrere Batches, eines je Partition, die auf diesem Broker Leader ist.

Der Puffer liegt vollständig im Client. Der Broker weiß nichts von `batch.size`; er bekommt ein fertiges Batch und schreibt es so, wie es ist, in das Log der Partition. Deshalb bleibt die Kompression erhalten, und deshalb liest ein Consumer später batchweise. Eine Queue, in der der Broker Nachrichten für Consumer vorhält und nach dem Lesen entfernt, gibt es nicht; er hat eine Request-Warteschlange für eingehende Netzwerk-Requests (`RequestQueueTimeMs`, Kapitel 14) und das Log

auf der Platte. Der Producer-Puffer ist flüchtig: Beim Absturz der Anwendung ist er weg, darum zählt eine Nachricht erst als geschrieben, wenn der Callback den Offset meldet.

15.2.3 `linger.ms` und `batch.size` bestimmen

Nicht ausrechnen, sondern messen, nachdem die Frage geklärt ist, was wichtiger ist: Latenz oder Durchsatz. Bei wenig Verkehr ist `linger.ms` die Latenz, die man freiwillig dazugibt; bei viel Verkehr spielt es kaum eine Rolle, weil die Batches ohnehin voll werden.

Anwendung	`linger.ms`	Warum
Einzelne Ereignisse, Antwort wird erwartet (Vertragsänderung am Schalter)	0 bis 5	jede Millisekunde landet beim Nutzer; Default seit 4.0 ist 5
Laufender Strom, Latenz unwichtig (Beitragslauf, Import, Logs)	50 bis 100	große Batches, bessere Kompression, weniger Requests
Dazwischen (Quelle der Fallstudie, Verarbeitung dauert ohnehin Sekunden)	10 bis 20	kaum spürbar, Batches deutlich voller

`batch.size` (Default 16 KB) gehört dazu: Bei 1 KB Nachrichten ist das Batch nach 16 Stück voll, egal wie lange gewartet wird. Richtwert: Nachrichten pro Sekunde je Partition × Nachrichtengröße × `linger.ms` / 1000, also so groß, dass in der Wartezeit typischerweise ein Batch zusammenkommt; für den Beitragslauf mit 1 KB und 100 ms landet man bei 100 KB bis 1 MB. Obergrenze ist `max.request.size` (1 MB, am Broker `message.max.bytes`). Nebenwirkung: Der Producer reserviert je Partition Speicher aus `buffer.memory` (32 MB); 1 MB Batches auf 64 Partitionen passen nicht hinein, dann blockiert `send()` und `bufferpool-wait-ratio` steigt. Mit Keys verteilen sich die Nachrichten auf alle Partitionen, jedes Batch füllt sich entsprechend langsamer.

So misst man es (`15-performance`, Aufgabe 1): dieselbe Last mit 5, 20, 50, 100, 200 ms fahren und drei Zahlen notieren: Nachrichten/s aus der App, `batch-size-avg` und `request-latency-avg` aus Grafana „Clients“. Der Durchsatz steigt bis zu dem Wert, ab dem die Batches ohnehin voll sind; danach bringt Warten nichts, kostet aber Latenz. Lokal lag die Grenze bei 20 ms (24.000 → 67.000 Nachrichten/s). Dazu `kafka-e2e-latency.sh` mit demselben Wert, damit die bezahlte Latenz sichtbar wird. Liegt `batch-size-avg` dauerhaft weit unter `batch.size`, fehlt Last oder `linger.ms` ist zu klein; liegt er genau bei `batch.size`, ist der Puffer die Bremse.

15.2.4 Wo die Einstellungen stehen

Beim Producer, nie beim Broker oder Topic; `kafka-configs.sh` kennt `linger.ms` nicht, der Broker sieht nur fertige Batches. Jede Anwendung entscheidet für sich:

```
# Spring Boot: eigener Name für batch.size, linger.ms über properties.
spring.kafka.producer.batch-size=100000
spring.kafka.producer.properties.linger.ms=20
spring.kafka.producer.compression-type=lz4
```

```
// Plain Java (02-producer-plain)
einstellungen.put(ProducerConfig.BATCH_SIZE_CONFIG, "100000");
einstellungen.put(ProducerConfig.LINGER_MS_CONFIG, "20");
```

```
# Kafka-Werkzeuge
kafka-console-producer.sh ... --command-property linger.ms=20
kafka-producer-perf-test.sh ... --command-property acks=all \
    linger.ms=100 batch.size=100000
```

Beim Start überschreiben, ohne Build: `--spring.kafka.producer.properties.linger.ms=100`. Kafka Streams schreibt über einen eigenen Producer mit Präfix `spring.kafka.streams.properties.producer.linger.ms` und setzt selbst 100 ms als Default. Prüfen, ob es wirkt: `kafka_producer_batch_size_avg` und `records-per-request-avg` steigen.

15.3 Producer: Kompression und acks

Kompression wirkt pro Batch – deshalb hilft Batching auch der Kompression. `lz4` **oder** `zstd` **sind meist die beste Wahl.** Der Broker speichert die Daten komprimiert, erst der Consumer entpackt. Das spart Netz, Platte und Page Cache. JSON-Vertragsänderungen schrumpfen oft auf 20–30 %.

Verfahren	Kompression	CPU
`gzip`	stark	hoch
`snappy`, `lz4`	mittel	niedrig
`zstd`	fast wie gzip	deutlich niedriger als gzip

Kompression am Topic

`compression.type` am Topic steht standardmäßig auf `producer` – der Broker speichert, was kommt. Setzt man am Topic ein anderes Verfahren als der Producer, entpackt und packt der Broker jeden Batch neu. Das kostet Broker-CPU.

`acks`: null, eins oder all ist eine Abwägung zwischen Durchsatz und Zuverlässigkeit. Seit Kafka 3.0 sind `acks=all` und Idempotenz der Standard – und dabei sollte es für Vertragsänderungen bleiben.

15.4 Broker

Die Standardwerte sind auf Leistung ausgelegt. Die Kafka-Doku rät ausdrücklich davon ab, an den Flush-Einstellungen zu drehen – die Replikation schützt die Daten, nicht das fsync. Der Engpass ist meist die Platte: mehrere Platten, XFS, `noatime`.

- `num.replica.fetchers` (Standard 1): Threads zum Kopieren von anderen Brokern. Kommen Follower nicht hinterher, ist das ein Hebel.
- `num.network.threads` (3) und `num.io.threads` (8): erhöhen nur, wenn freie CPU da ist und die Idle-Anteile unter 0,3 liegen.
- **JVM-Heap klein** (einige GB), den Rest des Arbeitsspeichers dem Page Cache lassen. Faustregel für den Page Cache: Schreibdurchsatz × 30 Sekunden.
- **Langsame Consumer** lesen alte Daten von der Platte und verdrängen frische Daten aus dem Cache – ein Nachzügler kann den ganzen Broker bremsen.

15.4.1 Unter der Haube: Warum Kafka so schnell ist

- **Sequenzielles Schreiben:** Nachrichten werden nur ans Ende einer Datei angehängt. Auch Festplatten schreiben sequenziell mit voller Geschwindigkeit; SSDs profitieren durch weniger Schreibverstärkung.
- **Page Cache statt eigener Cache:** Kafka hält keine Nachrichten im JVM-Heap. Das Betriebssystem puffert Schreib- und Lesezugriffe; Consumer, die nah am Ende lesen, bekommen die Daten aus dem Arbeitsspeicher.
- **Batches von Ende zu Ende:** Der Producer bündelt, komprimiert einmal, der Broker speichert den Batch unverändert, der Consumer entpackt. Niemand dazwischen fasst einzelne Nachrichten an.
- **Zero-Copy:** Beim Lesen schickt das Betriebssystem die Bytes direkt aus dem Page Cache ins Netz (ohne TLS).
- **Partitionen:** Last verteilt sich auf Broker, Platten und Consumer – ohne Abstimmung zwischen ihnen.

Daraus folgt umgekehrt, was Kafka langsam macht: viele winzige Requests, Consumer weit hinter dem Ende (Lesen von der Platte), Neukomprimieren am Broker, ein zu großer Heap, der dem Page Cache Speicher nimmt.

15.5 Consumer

- `fetch.min.bytes` (Standard 1 Byte) und `fetch.max.wait.ms` (500 ms): größere Fetches sparen Requests, kosten etwas Latenz.
- `max.partition.fetch.bytes` (1 MB) begrenzt pro Partition.
- `max.poll.records` (Standard 500) ist ein Hebel für Stabilität, nicht für Durchsatz. Holt eine Instanz 500 Vertragsänderungen und braucht für jede 20 Millisekunden, ist sie zehn Sekunden beschäftigt – und gibt beim Rebalance ihre Partitionen nicht rechtzeitig ab. Rechnung: Datensätze × Verarbeitungszeit muss deutlich unter `max.poll.interval.ms` bleiben. Die `bestand-demo` setzt deshalb 10.

```
spring.kafka.consumer.max-poll-records=10
spring.kafka.consumer.fetch-min-size=65536
spring.kafka.consumer.fetch-max-wait=200ms
spring.kafka.listener.concurrency=3
```

Wenn die Verarbeitung der Engpass ist, hilft kein Kafka-Tuning. Dann: mehr Instanzen (bis zur Partitionszahl), Verarbeitung beschleunigen (Datenbank-Batching, Caches) oder Partitionen erhöhen – mit den Folgen von oben.

15.6 Messen mit Bordmitteln

Kafka bringt die Messwerkzeuge selbst mit:

Werkzeug	misst
`kafka-producer-perf-test.sh`	Durchsatz und Latenz beim Schreiben
`kafka-consumer-perf-test.sh`	Durchsatz beim Lesen
`kafka-e2e-latency.sh`	Zeit vom Schreiben bis zum Lesen, eine Nachricht nach der anderen

In Kafka 4.4 heißen die Client-Optionen einheitlich `--command-property`. Neu sind Aufwärmrunden mit `--warmup-records` und Keys über `--record-key-range` mit `--key-distribution`. Das Lab-Skript `mit-metriken` hängt den JMX-Exporter an das Werkzeug, sodass die Producer-Metriken live in Grafana erscheinen.

15.6.1 Hintergrund und Konsequenzen

- **Die Latenz im Producer-Perf-Test ist irreführend, wenn ohne Drosselung gemessen wird.** Mit `--throughput -1` füllt das Werkzeug den Puffer so schnell es kann; die gemessene Latenz ist dann vor allem Wartezeit im eigenen Puffer. Für Latenz-Messungen mit fester Rate messen (`--throughput 2000`), für Durchsatz ohne.
- **Perzentile lesen:** Die Ausgabe zeigt 50., 95., 99. und 99,9. Perzentil. Für den Betrieb zählt das 99.
- **Testdaten sind künstlich:** Der Perf-Test erzeugt Zufallsbuchstaben, die anders komprimieren als echte Vertragsänderungen. Mit `--payload-file` lassen sich echte Beispieldaten verwenden.

15.7 Lab: Perf-Test

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic perf --
partitions 6 --replication-factor 3
```

Lauf 1 – Basislinie:

```
bash /opt/lab/bin/mit-metriken 9201 kafka-producer-perf-test.sh --bootstrap-server
kafka1:9092 --topic perf --num-records 200000 --record-size 1000 --throughput -1
--warmup-records 20000 --command-property acks=all
```

Lauf 2 – getunt: größere Batches, warten, komprimieren.

```
bash /opt/lab/bin/mit-metriken 9201 kafka-producer-perf-test.sh --bootstrap-server
kafka1:9092 --topic perf --num-records 200000 --record-size 1000 --throughput -1
--warmup-records 20000 --command-property acks=all batch.size=100000 linger.ms=100
compression.type=lz4
```

Lauf 3 – zum Vergleich `acks=1`:

```
bash /opt/lab/bin/mit-metriken 9201 kafka-producer-perf-test.sh --bootstrap-server
kafka1:9092 --topic perf --num-records 200000 --record-size 1000 --throughput -1
```

```
--warmup-records 20000 --command-property acks=1 batch.size=100000 linger.ms=100
compression.type=lz4
```

Lauf 4 – mit 1000 verschiedenen Vertragsnummern als Keys:

```
bash /opt/lab/bin/mit-metriken 9201 kafka-producer-perf-test.sh --bootstrap-server
kafka1:9092 --topic perf --num-records 200000 --record-size 1000 --throughput -1
--record-key-range 1000 --key-distribution random --command-property acks=all
batch.size=100000 linger.ms=100 compression.type=lz4
```

Lesen und Ende-zu-Ende:

```
bash /opt/lab/bin/mit-metriken 9211 kafka-consumer-perf-test.sh --bootstrap-server
kafka1:9092 --topic perf --num-records 200000
```

```
./kafka-e2e-latency.sh kafka1:9092 perf 1000 all 1000
```

Bei `kafka-e2e-latency.sh` sind die Parameter positionsabhängig: Server, Topic, Anzahl, acks, Größe. Jeder Lauf schreibt rund 600 MB, mit RF 3 auf demselben Rechner also etwa 1,8 GB. Danach aufräumen:

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --delete --topic perf
```

15.7.1 Beobachten im Dashboard

Producer-Sicht (Zeile „Clients“): `batch-size-avg` und `records-per-request-avg` steigen mit Batching, `compression-rate-avg` sinkt mit lz4 – kleiner heißt stärker komprimiert. Broker-Sicht (Zeile „Requests“): Bei `acks=all` ist die Remote-Zeit deutlich sichtbar, bei `acks=1` fast null. Die freien Threads sollten über 0,3 bleiben.

15.7.2 Ergebnisse einordnen

Das Buch hat mit Batching den Durchsatz von 117 auf 438 MB pro Sekunde gesteigert und die Latenz von 165 auf 65 Millisekunden gesenkt. **Auf virtuellen Maschinen und Docker Desktop sind die absoluten Zahlen nicht aussagekräftig – die Richtung schon.** Drei Broker auf einem Rechner teilen sich Platte und CPU, die Replikation läuft ohne echtes Netz.

Lasttests für die Produktion: auf Hardware wie in Produktion, mit echten Nachrichtengrößen, mit RF und `acks` wie in Produktion, ausreichend lange und mit mehreren Producern parallel.

15.8 Quotas

Ein Client, der Amok läuft, kann einen Broker auslasten. Quotas begrenzen den Durchsatz je Client oder Benutzer:

```
./kafka-configs.sh --bootstrap-server kafka1:9092 --alter --entity-type clients --
entity-name beitragslauf --add-config producer_byte_rate=10485760
```

Quotas gelten **pro Broker**: 10 MB/s bei drei Brokern erlauben bis zu 30 MB/s insgesamt. Ein gebremster Client sieht `produce-throttle-time-avg` > 0.

15.9 Woran sehe ich das im Betrieb?

- **Bytes rein/raus pro Broker vergleichen:** Ein Broker mit deutlich mehr Last hat mehr Leader, eine heiße Partition oder mehr Partitionen.
- **Trend statt Momentaufnahme:** Wächst der Durchsatz über Monate, rechtzeitig planen – neue Broker brauchen Vorlauf, Umverteilung kostet Zeit.
- **Synthetik-Test:** `kafka-e2e-latency.sh` oder ein kleiner eigener Producer und Consumer regelmäßig laufen lassen. Er prüft den Weg so, wie eine Anwendung ihn erlebt.
- **Kapazitätsreserve:** Im Normalbetrieb deutlich unter der gemessenen Grenze bleiben – für Aufholen nach Störungen und Replikation nach einem Broker-Ausfall.

16 Sicherheit

Vertragsdaten, Bankverbindungen und Gesundheitsangaben aus der Risikoprüfung gehören zu den sensibelsten Daten einer Versicherung. Das Lab läuft der Einfachheit halber ohne Verschlüsselung und ohne Anmeldung – in Produktion ist das undenkbar. Dieses Kapitel zeigt die vier Schichten: Transport verschlüsseln, Clients authentifizieren, Rechte vergeben und sensible Felder selbst verschlüsseln.

16.1 Die vier Schichten

Schicht	Frage	Kafka-Mittel
Verschlüsselung der Übertragung	Kann jemand im Netz mitlesen?	TLS auf den Listenern
Authentifizierung	Wer ist dieser Client?	mTLS, SASL/SCRAM, SASL/OAUTHBEARER, Kerberos
Autorisierung	Was darf er?	ACLs je Benutzer, Topic, Gruppe
Verschlüsselung der Inhalte	Kann ein Administrator oder ein Backup die Daten lesen?	nicht in Kafka – im Client oder in einem Proxy

Kafka verschlüsselt **keine** Daten auf der Platte. Das übernimmt die Infrastruktur (verschlüsselte Volumes) – oder die Anwendung selbst für einzelne Felder.

16.2 Listener und Protokolle

Ein Broker kann mehrere Listener mit unterschiedlicher Sicherheit haben:

```
listeners=INTERN://:9092,EXTERN://:9094,CONTROLLER://:9093
advertised.listeners=\
    INTERN://kafka1.intern:9092,EXTERN://kafka1.extern.beispiel.de:9094
listener.security.protocol.map=\
    INTERN:SASL_SSL,EXTERN:SASL_SSL,CONTROLLER:SSL
inter.broker.listener.name=INTERN
controller.listener.names=CONTROLLER
```

Protokoll	Verschlüsselt	Authentifiziert
`PLAINTEXT`	nein	nein (nur im Lab)
`SSL`	ja	optional per Client-Zertifikat (mTLS)
`SASL_PLAINTEXT`	nein	ja – Passwörter im Klartext im Netz, nicht verwenden
`SASL_SSL`	ja	ja

16.3 Authentifizierung

Verfahren	Wie	Passt für
mTLS	Client weist sich mit Zertifikat aus; Benutzername = Distinguished Name	Plattformen mit eigener PKI, Kubernetes mit Cert-Manager
SASL/ SCRAM-SHA-512	Benutzer und Passwort, gespeichert als Hash im Cluster	einfacher Einstieg, Operator-verwaltete Benutzer
SASL/ OAUTHBEARER	Token eines Identity Providers (z. B. Keycloak, Entra ID)	zentrale Identitäten, kurze Token-Laufzeiten
SASL/ GSSAPI (Kerberos)	Kerberos-Tickets	bestehende Active-Directory-Landschaften

SCRAM-Benutzer anlegen:

```
./kafka-configs.sh --bootstrap-server kafka1:9092 --alter --add-config 'SCRAM-
SHA-512=[password=geheim]' --entity-type users --entity-name bestandsverwaltung
```

16.3.1 Spring Boot mit SASL_SSL und SCRAM

```
spring.kafka.bootstrap-servers=\
    kafka1.intern:9092,kafka2.intern:9092,kafka3.intern:9092
spring.kafka.security.protocol=SASL_SSL
spring.kafka.ssl.trust-store-location=file:/etc/kafka/truststore.p12
spring.kafka.ssl.trust-store-password=${KAFKA_TRUSTSTORE_PASSWORT}
spring.kafka.properties.sasl.mechanism=SCRAM-SHA-512
spring.kafka.properties.sasl.jaas.config=\
    org.apache.kafka.common.security.scram.ScramLoginModule required \
  username="bestandsverwaltung" password="${KAFKA_PASSWORT}";
```

Passwörter kommen aus Umgebungsvariablen oder einem Secret-Store, nie aus dem Repository.

16.4 Autorisierung mit ACLs

Mit einem Authorizer (`authorizer.class.name=org.apache.kafka.metadata.authorizer.StandardAuthorizer` im KRaft-Modus) braucht jeder Zugriff eine Erlaubnis. Für die Bestandsverwaltung:

```
./kafka-acls.sh --bootstrap-server kafka1:9092 --add --allow-principal
User:bestandsverwaltung --operation Read --topic bestand.vertragsaenderungen
```

```
./kafka-acls.sh --bootstrap-server kafka1:9092 --add --allow-principal
User:bestandsverwaltung --operation Read --group bestandsverwaltung
```

```
./kafka-acls.sh --bootstrap-server kafka1:9092 --add --allow-principal
User:bestandssystem --operation Write --operation Describe --topic
bestand.vertragsaenderungen
```

```
./kafka-acls.sh --bootstrap-server kafka1:9092 --list --topic
bestand.vertragsaenderungen
```

Anwendung	Rechte
Producer	`Write`, `Describe` am Topic; bei Transaktionen zusätzlich `Write` an der `TransactionalId`
Consumer	`Read` am Topic, `Read` an der Consumer Group
Kafka Streams	`Read`/`Write` an Eingangs- und Ausgangs-Topics, alle Rechte an Topics und Gruppe mit Präfix `<application.id>`
Kafbat UI (Produktion)	nur `Describe` und `Read`

Mit `--resource-pattern-type prefixed` gelten Regeln für alle Topics mit einem Präfix, etwa `bestand.` für das Bestandsteam.

16.4.1 Hintergrund und Konsequenzen

- `allow.everyone.if.no.acl.found` steht standardmäßig auf `false`: Ohne passende ACL ist alles verboten. Beim Einschalten des Authorizers also zuerst die ACLs anlegen, dann umschalten.
- **Super-User** (`super.users`) für Broker, Controller und Administration – sparsam vergeben.
- **ACLs als Code:** wie Topics über eine Pipeline oder als Operator-Ressource pflegen (Kapitel 17).

16.5 Felder verschlüsseln

Apache Kafka verschlüsselt keine einzelnen Felder. Wenn Administratoren, Backups oder nachgelagerte Systeme bestimmte Daten – Bankverbindung, Gesundheitsangaben – nicht im Klartext sehen dürfen, verschlüsselt der Producer sie selbst.

Ansatz	Wie	Bemerkung
eigener Serializer oder Interceptor	sensible Felder vor dem Senden verschlüsseln, im Deserializer entschlüsseln	volle Kontrolle, eigener Code
Confluent Client-Side Field Level Encryption	Felder im Schema markieren, Verschlüsselung über Regeln der Schema Registry und ein KMS	kommerziell
Proxy (z. B. Kroxylicious)	ein Proxy zwischen Clients und Broker verschlüsselt Nachrichten	keine Client-Änderung; verschlüsselt ganze Records, nicht einzelne Felder

16.5.1 Envelope Encryption

```
Master-Schlüssel (KMS, HSM, Vault)
        │ verschlüsselt
        ▼
Daten-Schlüssel je Kunde (DEK) ──► verschlüsselt das Feld "iban" in der Nachricht
        │
        └─ Schlüssel-ID im Header: "kek=kunde-4711, dek-version=3"
```

- Pro Kunde (oder pro Datensatz) ein Daten-Schlüssel, der selbst mit einem Master-Schlüssel im KMS verschlüsselt gespeichert ist.
- Die Schlüssel-ID steht im Header; der Consumer holt damit den passenden Schlüssel.
- **Crypto-Shredding:** Löscht man den Schlüssel eines Kunden, sind seine Daten in allen Topics, Backups und DR-Clustern unlesbar – eine Lösung für Löschpflichten nach DSGVO trotz unveränderlichem Log.

Das Modul `labs/16-feldverschluesselung` setzt das um: ein Serializer, der den `JacksonJsonSerializer` umhüllt und nur das Feld `iban` verschlüsselt (AES-GCM, zufälliger IV), ein Deserializer als Gegenstück und ein `Schluesseldienst`, der je Kunde einen Daten-Schlüssel aus einem Master-Schlüssel ableitet. In Produktion liefert das KMS die Schlüssel; der Rest bleibt gleich.

```
public class IbanVerschluesselung
        implements Serializer<Vertragsaenderung> {

    private final JacksonJsonSerializer<Vertragsaenderung> json =
            new JacksonJsonSerializer<>();
    private final Schluesseldienst schluessel =
            Schluesseldienst.instanz();

    @Override
    public void configure(Map<String, ?> configs, boolean isKey) {
        json.configure(configs, isKey);
        // Master-Schlüssel aus der Kafka-Konfiguration
        schluessel.setzeMasterschluessel(
                configs.get("app.master-schluessel").toString());
    }

    @Override
    public byte[] serialize(String topic, Headers headers,
            Vertragsaenderung aenderung) {
        if (aenderung != null && aenderung.daten().containsKey("iban")) {
            String id = schluessel.schluesselIdFuer(
                    aenderung.daten().get("kundennummer"));
            Map<String, String> daten = new HashMap<>(aenderung.daten());
            daten.put("iban",
                    schluessel.verschluessele(id, daten.get("iban")));
            headers.add("dek-id", id.getBytes(StandardCharsets.UTF_8));
```

```
            aenderung = new Vertragsaenderung(aenderung.vertragsnummer(),
                    aenderung.version(), aenderung.art(), daten,
                    aenderung.zeitpunkt());
        }
        return json.serialize(topic, headers, aenderung);
    }
}
```

Kafka erzeugt Serializer per Klassenname und ohne Spring – deshalb kommt der Master-Schlüssel über `spring.kafka.producer.properties.app.master-schluessel` in `configure()` an, und der `Schluesseldienst` ist eine statische Instanz. Der Test `FeldverschluesselungTest` prüft ohne Kafka: Im Topic steht die IBAN nicht im Klartext, der Deserializer liefert das Original zurück, dasselbe Feld ergibt jedes Mal ein anderes Chiffrat – und nach `loescheSchluessel("kunde-K-200")` schlägt das Entschlüsseln fehl (Crypto-Shredding).

Verschlüsselte Felder taugen nicht als Key

Wechselnde Chiffrate brechen Partitionierung und Compaction. Der Key bleibt eine nicht sensible fachliche Nummer (Vertragsnummer), sensible Inhalte stehen verschlüsselt im Wert. Key-Rotation und die Verfügbarkeit des KMS werden Teil des Betriebs – fällt das KMS aus, können Consumer nicht mehr entschlüsseln.

16.6 Quotas als Schutz

Quotas begrenzen Durchsatz und Request-Zeit je Benutzer oder Client und schützen den Cluster vor einem fehlerhaften Batch-Lauf (Kapitel 15):

```
./kafka-configs.sh --bootstrap-server kafka1:9092 --alter --entity-type users --
entity-name beitragslauf --add-config
producer_byte_rate=10485760,request_percentage=50
```

16.6.1 Einen Broker gegen Überlast und DoS absichern

Gegen einen Angreifer von außen schützt vor allem Nicht-Erreichbarkeit, gegen den eigenen lauten Nachbarn schützen Quotas. Beides gehört zusammen, denn im Unternehmensnetz ist der zweite Fall der häufige: ein Deployment mit Retry-Schleife legt den Cluster lahm.

Vor dem Broker: kein Broker-Port im Internet, Listener nur im internen Netz, Firewall auf die Client-Subnetze; Authentifizierung Pflicht (SASL/SCRAM oder mTLS) plus ACLs, damit ein unbekannter Client nicht bis zum Produce-Request kommt; Verbindungen begrenzen mit `max.connections.per.ip` (Default unbegrenzt) und `max.connections` je Listener; `connection.failed.authentication.delay.ms` bremst das Raten von Passwörtern.

Im Broker, das eigentliche Werkzeug: Quotas, alles Bordmittel von Apache Kafka, kein Confluent nötig.

```
kafka-configs.sh --bootstrap-server kafka1:9092 --alter \
  --entity-type clients --entity-name quelle-app \
  --add-config producer_byte_rate=10485760,\
consumer_byte_rate=52428800,request_percentage=50
```

Quota	Wirkung
`producer_byte_rate`, `consumer_byte_rate`	Bytes/s je Client oder Benutzer; der Broker verzögert die Antwort, der Client sieht `produce-throttle-time-avg` > 0
`request_percentage`	Anteil der Handler-Zeit, den ein Client belegen darf; hilft gegen Metadaten-Stürme und tausende kleine Requests
`--entity-default`	Quota für alle ohne eigenen Eintrag; ohne Default schützt man nur die, die man kennt
`controller_mutation_rate`	begrenzt das Anlegen von Topics und Partitionen; dazu `auto.create.topics.enable=false`

Größen begrenzen: `message.max.bytes` (1 MB) am Broker oder Topic gegen Riesen-Nachrichten, `socket.request.max.bytes` für den Request, `retention.bytes` je Topic, damit ein Flutschreiber die Platte nicht füllt (Alert `RetentionNearlyFull`).

Erkennen: Die Metriken aus Kapitel 14 sind die DoS-Anzeige. `RequestHandlerAvgIdlePercent` sinkt, `RequestQueueTimeMs` steigt, `BytesInPerSec` eines Brokers schießt hoch, `produce-throttle-time-avg` zeigt, wen die Quota bremst. Alert `RequestHandlerBusy` ist dafür da. Was Quotas nicht können: einen SYN-Flood oder Netzwerk-Sättigung abwehren, das bleibt Firewall und Netz. Und sie gelten pro Broker: 10 MB/s auf drei Brokern sind 30 MB/s. Confluent bietet darüber hinaus clusterweite Quotas und Mandanten-Isolation; für einen eigenen Cluster reichen die Bordmittel.

16.7 Checkliste

- ☐ Alle Listener mit TLS; kein `PLAINTEXT` außerhalb des Labs
- ☐ Jede Anwendung hat einen eigenen Benutzer
- ☐ Authorizer aktiv, ACLs nach dem Prinzip der geringsten Rechte
- ☐ Passwörter und Schlüssel aus einem Secret-Store
- ☐ Sensible Felder verschlüsselt, Key nicht sensibel
- ☐ Platten verschlüsselt, Backups und DR-Cluster eingeschlossen
- ☐ Kafbat UI und andere Werkzeuge in Produktion nur lesend
- ☐ Quotas für Batch-Läufe und unbekannte Clients

16.8 Weiterführende Links

- Kafka-Dokumentation, Security: https://kafka.apache.org/documentation/#security
- Kroxylicious: https://kroxylicious.io/

17 Betrieb

Ein Kafka-Cluster in einer Versicherung läuft jahrelang, wird gewartet, erweitert und aktualisiert – ohne dass die Bestandsverwaltung, das Inkasso oder das Schadenportal es merken. Dieses Kapitel sammelt, was dafür nötig ist: Plattform und Distribution, Hardware, Topics als Code, Wartung, Erweiterung, Upgrades und die Best Practices aus dem Seminar.

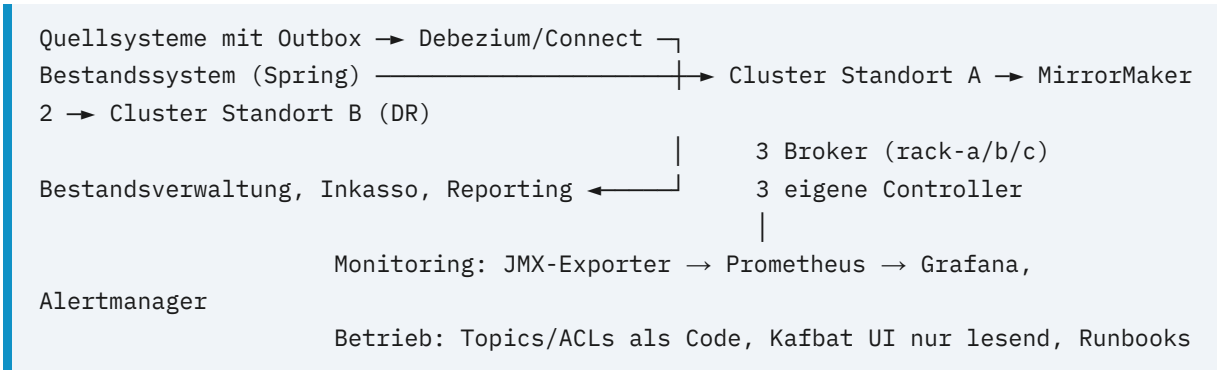
17.1 Apache Kafka, Confluent, KRaft und ZooKeeper

Diese Begriffe liegen auf verschiedenen Ebenen:

Begriff	Was es ist	Ebene
ZooKeeper	externer Koordinationsdienst, der früher Controller-Wahl und Metadaten verwaltete	intern in Kafka, seit Kafka 4.0 entfernt
KRaft	Kafka verwaltet seine Metadaten selbst über ein Raft-Protokoll in Controller-Knoten	intern in Kafka, heute Standard
Apache Kafka	das Open-Source-Projekt: Broker, Clients, Connect, Streams	Produkt
Confluent	Firma der ursprünglichen Kafka-Entwickler; vertreibt Kafka mit Zusatzfunktionen und Support	Distribution / Anbieter
Strimzi, Red Hat Streams for Apache Kafka	Operatoren, die Kafka auf Kubernetes und OpenShift betreiben	Betriebsform

Confluent ist also keine Alternative zu KRaft, sondern eine Ebene höher. Confluent Platform (selbst betrieben) basiert ab Version 8 auf Kafka 4.x und läuft nur mit KRaft; Version 7.x unterstützt noch ZooKeeper. In Confluent Cloud sieht man weder ZooKeeper noch KRaft. Zusätzlich bringt Confluent unter anderem eine Schema Registry, ein Control Center, Client-Side Field Level Encryption, rollenbasierte Rechte, Tiered Storage und Cluster Linking mit – teils unter kommerzieller Lizenz.

17.2 Zielbild für den Betrieb



- **Drei eigene Controller**, nicht kombiniert wie im Lab. 2N+1 Controller tolerieren N Ausfälle. Die Kafka-Doku nennt je etwa 5 GB Arbeitsspeicher und 5 GB Platte für das Metadaten-Log. Seit KIP-853 lassen sich Controller im laufenden Betrieb hinzufügen und entfernen.

- `broker.rack` gesetzt, damit die drei Kopien in drei Brandabschnitten liegen.
- **DR-Standort** mit MirrorMaker 2 (Kapitel 19).

17.3 Hardware und Betriebssystem

Bereich	Empfehlung
Betriebssystem	Linux; Windows wird für Broker nicht gut unterstützt
Dateien	mindestens 100.000 erlaubte Dateideskriptoren; `vm.max_map_count` erhöhen
Platten	mehrere Platten nur für Kafka (JBOD); kein RAID nötig, die Replikation schützt
Dateisystem	XFS mit `noatime`
Arbeitsspeicher	JVM-Heap einige GB, Rest für den Page Cache (≈ Schreibdurchsatz × 30 s)
Netz	Reserve für Replikation und Aufholen; unter 60 % Grundlast bleiben
Java	Broker ab Java 17, Clients ab Java 11
fsync	Standard lassen; Haltbarkeit kommt aus der Replikation

17.4 Kafka auf OpenShift und Kubernetes

Auf OpenShift betreibt man Kafka über einen Operator – Strimzi bzw. Red Hat Streams for Apache Kafka, oder Confluent for Kubernetes. Cluster, Topics und Benutzer werden zu Kubernetes-Ressourcen:

```
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: bestand.vertragsaenderungen
  labels:
    strimzi.io/cluster: bestand-cluster
spec:
  partitions: 12
  replicas: 3
  config:
    min.insync.replicas: "2"
    retention.ms: "1209600000"
```

```
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaUser
metadata:
  name: bestandsverwaltung
  labels:
    strimzi.io/cluster: bestand-cluster
spec:
  authentication:
    type: scram-sha-512
  authorization:
```

```
    type: simple
    acls:
      - resource: { type: topic, name: bestand.vertragsaenderungen }
        operations: [Read, Describe]
      - resource: { type: group, name: bestandsverwaltung }
        operations: [Read]
```

Der Operator übernimmt Rolling Updates, Zertifikate und das Anlegen von Topics und Benutzern; JMX-Exporter und ein Lag-Exporter lassen sich direkt in der Kafka-Ressource einschalten. Wichtig bleiben: persistente Volumes mit Reserve, Broker über Zonen verteilt (Rack Awareness über Zonen-Labels) und Cruise Control für die Lastverteilung.

17.5 Topics und ACLs als Code

Topics werden über Git und eine Pipeline angelegt, nie von Hand in Produktion – deshalb ist `auto.create.topics.enable` aus. Möglichkeiten:

Weg	Werkzeug
Operator-Ressourcen	`KafkaTopic`, `KafkaUser` (Strimzi), GitOps mit Argo CD
Infrastruktur als Code	Terraform-Provider für Kafka
Skript in der Pipeline	`kafka-topics.sh` / `kafka-configs.sh` / `kafka-acls.sh`, idempotent geschrieben
in der Anwendung	`NewTopic`-Beans (Kapitel 7) – für Teams, die ihre Topics selbst verantworten

Zu jedem Topic gehören festgelegt: Eigentümer, Partitionen, RF, `min.insync.replicas`, Retention, Schema und Kompatibilitätsregel.

17.6 Rolling Restart

1. Prüfen: keine unterreplizierten Partitionen, kein `UnderMinIsr`.
2. Einen Broker geordnet stoppen (Controlled Shutdown – der Broker gibt seine Leader ab).
3. Warten, bis er wieder läuft und `UnderReplicatedPartitions` wieder 0 ist.
4. Leader neu verteilen: `kafka-leader-election.sh --election-type preferred --all-topic-partitions`.
5. Erst dann den nächsten Broker.

Bei RF 3 und `min.insync.replicas=2` läuft das Schreiben während der ganzen Wartung weiter – aber ohne Reserve. Fällt in dieser Zeit ein zweiter Broker aus, stoppt das Schreiben.

17.7 Cluster erweitern und Partitionen umziehen

Neue Broker bekommen keine Daten von selbst. Umziehen mit `kafka-reassign-partitions.sh`:

```
./kafka-reassign-partitions.sh --bootstrap-server kafka1:9092 --topics-to-move-
json-file umzug.json --broker-list 1,2,3,4 --generate
```

```
./kafka-reassign-partitions.sh --bootstrap-server kafka1:9092 --reassignment-json-
file plan.json --execute --throttle 50000000
```

```
./kafka-reassign-partitions.sh --bootstrap-server kafka1:9092 --reassignment-json-
file plan.json --verify
```

`--throttle` begrenzt die Replikation in Bytes pro Sekunde, damit der Umzug den laufenden Betrieb nicht stört; `--verify` entfernt die Drosselung nach Abschluss. Mit Cruise Control lässt sich das automatisieren – nach Plattenbelegung, Netz und CPU.

Neu in Kafka 4.3: `cordoned.log.dirs` sperrt ein Log-Verzeichnis für neue Partitionen, etwa vor einem Plattentausch.

17.8 Upgrades

- Kafka 4.x läuft nur noch mit KRaft. ZooKeeper-Cluster werden vorher mit Kafka 3.9 als Brücke migriert.
- Neue Funktionen werden über **Feature-Versionen** freigeschaltet, nicht automatisch mit dem Binär-Upgrade. `kafka-features.sh describe` zeigt den Stand; nach dem Upgrade aller Knoten hebt `kafka-features.sh upgrade` die Metadaten-Version.
- Reihenfolge: Controller, dann Broker – jeweils als Rolling Restart. Clients können älter oder neuer sein als die Broker; die Kompatibilitätsmatrix der Doku beachten.
- Vorher im Test-Cluster mit realer Last und den Anwendungen der Teams proben.

17.9 Speicher: Kafka darf nie die Platte ausgehen

Eine volle Platte ist der schlimmste Fall: Der Broker fällt aus, und das Aufräumen ist mühsam. Deshalb:

- Topics mit `retention.bytes` begrenzen (gilt pro Partition!).
- `RetentionSizeInPercent` und die Plattenbelegung alarmieren.
- `kafka-log-dirs.sh` zeigt, was wo liegt.
- Tiered Storage legt alte Segmente in einen Objektspeicher – das macht lange Retention für Puffer und Replay günstig.

17.10 Best Practices

Machen:

- Schemas verwenden und Kompatibilitätsregeln festlegen (Kapitel 10).
- Keys fachlich wählen: Reihenfolge, wo nötig, und gleichmäßige Verteilung.
- Consumer idempotent bauen; Fehlerbehandlung je Topic bewusst entscheiden (Kapitel 9).
- Namenskonvention `<domäne>.<inhalt>` und Topics per Code anlegen.
- RF 3, `min.insync.replicas=2`, `acks=all` für wichtige Daten.
- Retention pro Topic bewusst festlegen.
- Mehrere Bootstrap-Adressen in jedem Client.

Vermeiden:

- `auto.create.topics.enable=true` in Produktion.
- Lange oder blockierende Verarbeitung in der Poll-Schleife – sie führt zu Rebalance-Stürmen.

- Sehr viele kleine Topics oder Partitionen „auf Vorrat“.
- Große Nachrichten (Claim-Check-Muster nutzen), Kafka als Datenbank für Ad-hoc-Abfragen, synchrones Request/Reply als Standardmuster.
- Unclean Leader Election.

17.11 Die Plattform beim Kunden: Confluent, Terraform und OpenShift

Im Lab läuft alles auf einem Rechner: drei Broker in Docker, die Anwendungen als JAR aus der Konsole. In der Produktion beim Kunden sieht dasselbe Bild so aus:

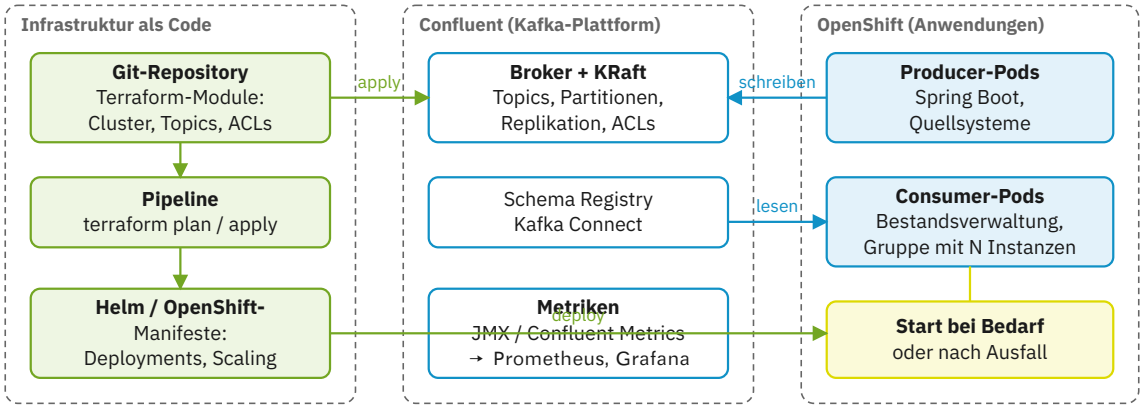


Abbildung 13: Confluent trägt Kafka, Terraform beschreibt die Plattform, OpenShift startet die Anwendungen

Baustein	Im Lab	In der Produktion
Kafka-Cluster	`apache/kafka:4.4.0` in Docker, KRaft kombiniert	Confluent Platform, KRaft, eigene Controller; Schema Registry und Connect dazu
Topics, ACLs, Quotas	`kafka-topics.sh`, `NewTopic`-Bean	Terraform-Module in Git, Pipeline mit `plan` und `apply`; nichts von Hand
Anwendungen	`java -jar` je Fenster	Pods auf OpenShift; Deployment und Skalierung über Manifeste, Start bei Bedarf oder nach Ausfall
Metriken	JMX-Exporter, Prometheus, Grafana aus `kafka-lab`	dieselben Metriknamen aus Confluent bzw. dem JMX-Exporter, Micrometer in den Pods, zentrales Prometheus oder Confluent Control Center
Consumer-Lag von außen	`kafka-consumer-groups.sh`	Lag-Exporter oder Control Center, weil die Consumer-Pods nicht immer laufen

Was sich dadurch an den Regeln des Buchs ändert: nichts an den Garantien, einiges an der Betonung.

- **Consumer, die nicht dauerhaft laufen.** Ein Pod, der erst bei Bedarf oder nach einem Ausfall startet, liest ab dem committeten Offset weiter (Kapitel 6). Zwei Dinge sind dafür

zu prüfen: `offsets.retention.minutes` muss länger sein als die längste Pause, sonst greift `auto.offset.reset`; und `group.instance.id` (Static Membership) lohnt sich, wenn OpenShift einen Pod innerhalb des Session-Timeouts ersetzt, damit kein Rebalance ausgelöst wird.

- **Lag messen, wenn niemand liest.** Die Client-Metrik `records-lag-max` existiert nur in einem laufenden Pod (Kapitel 14). Der Rückstand einer pausierten Gruppe ist nur von außen sichtbar; das Monitoring braucht deshalb einen Lag-Exporter gegen den Confluent-Cluster, und der Alarm sollte „Lag wächst, obwohl kein Consumer läuft“ unterscheiden von „Consumer läuft und kommt nicht nach“.
- **Skalierung ist eine Zahl im Manifest.** Mehr Pods bedeuten mehr Instanzen derselben Gruppe, höchstens so viele, wie das Topic Partitionen hat (Kapitel 6 und 18). Die Partitionszahl steht im Terraform-Modul; wer sie erhöht, ändert die Zuordnung der Keys (Kapitel 15). Beides gehört in dieselbe Änderung und dieselbe Freigabe.
- **Topics als Code, konsequent.** Weil Terraform die Topics anlegt, bleibt `auto.create.topics.enable` aus und die `NewTopic`-Bean aus den Beispielen entfällt in der Produktion; sie ist dort höchstens ein Test beim Start, ob das Topic existiert und die erwartete Partitionszahl hat.
- **Confluent statt Apache-Image.** Die Konfigurationsnamen, Metriken und Werkzeuge sind dieselben; Confluent bringt Schema Registry, Connect, Control Center und Support dazu (Abschnitt oben). Die Beispiele aus `labs` laufen unverändert gegen einen Confluent-Cluster, nur `bootstrap-servers`, TLS und SASL (Kapitel 16) kommen aus der Umgebung des Pods statt aus `application.properties`.

Für die Fallstudie (Kapitel 18) heißt das konkret: Die Bestandsverwaltung ist ein Deployment mit `replicas` zwischen 1 und der Partitionszahl, `max.poll.records` klein, Commit nach der Verarbeitung, und ein Lag-Alarm, der auch dann feuert, wenn OpenShift die Pods gerade nicht gestartet hat.

17.11.1 Confluent: empfohlene Einstellungen für die Plattform

Die Werte aus dem Kapitel „Wenn-Dann“ (Zielwerte für erhöhten Ausfallschutz) gelten für Confluent unverändert, weil darunter Apache Kafka läuft. Dazu kommen die Confluent-eigenen Bausteine:

Baustein	Einstellung	Warum
Terraform-Provider	`confluentinc/confluent` für Cloud, `Mongey/kafka` oder das Confluent-Ansible/REST-Setup für Platform; je Umgebung ein Service-Account mit eigenen API-Keys, Keys in Vault, nie im State im Klartext	Topics, ACLs, Schemas und Konten aus Git
Rechte	RBAC statt reiner ACLs: je Anwendung ein Service-Account mit `DeveloperRead`/`DeveloperWrite` auf Topic-Präfix und Gruppe; Menschen nur über Rollen, keine geteilten Keys	geringste Rechte, nachvollziehbar im Audit-Log
Schema Registry	Kompatibilität `BACKWARD` als Standard, `FULL` für Topics mit langer Retention; Schemas aus Git über Terraform; `auto.register.schemas=false` in Produktion	kein Consumer bricht durch eine Producer-Änderung
Self-Balancing Clusters	eingeschaltet, `confluent.balancer.heal.uneven.load.trigger=ANY_UNEVEN_LOAD` nur mit Netzreserve, sonst `EMPTY_BROKER`	neue Broker bekommen Partitionen ohne gemeinsame Arbeit, Reassignments brauchen Bandbreite
Tiered Storage	für Topics mit langer Retention (Audit, Replay) auf Objektspeicher; `confluent.tier.local.hotset.ms` nach Lesemuster	Platte des Brokers bleibt klein, Retention wird bezahlbar
Cluster Linking	statt MirrorMaker 2 für den zweiten Standort, Offsets werden mitgespiegelt; Failover-Übung je Quartal bleibt	weniger Betrieb als ein Connect-Cluster, gleiche Regeln wie Kapitel 19
Control Center	nur lesend, Alerts nur für Grundregeln; Lag-Alerts im zentralen Werkzeug (Kapitel 14)	eine Quelle für Alarme
Connect und ksqlDB	eigene Cluster je Zweck, nie auf den Broker-Knoten; Connect-Gruppen wie Consumer überwachen	ein hängender Connector darf den Broker nicht bremsen
Upgrades	Confluent-Versionen mit Kafka-Version abgleichen (Platform 8 = Kafka 4.x, nur KRaft); Rolling Restart wie oben, ein Broker nach dem anderen	KRaft-Pflicht ab Platform 8

17.12 Vom Testprojekt zur produktiven Anwendung

- ☐ Topic mit Eigentümer, Partitionen, RF 3, `min.insync.replicas=2`, Retention über die Pipeline angelegt
- ☐ Schema registriert, Kompatibilitätsregel gesetzt
- ☐ Eigener Benutzer, ACLs nach geringsten Rechten, TLS
- ☐ Producer: `acks=all`, Idempotenz, Ergebnis von `send()` ausgewertet, Outbox wo nötig
- ☐ Consumer: Commit nach der Verarbeitung, idempotent, `max.poll.records` passend, Error Handler entschieden
- ☐ Mehrere Instanzen getestet: Rebalance, Absturz, Neustart
- ☐ Metriken exportiert, Lag von außen gemessen, Alarme mit Runbook
- ☐ Offset-Reset und Replay geübt; nachgelagerte Systeme vertragen Duplikate
- ☐ Lasttest mit realistischen Daten; Kapazitätsreserve dokumentiert
- ☐ DR-Verhalten geklärt: Was passiert beim Ausfall eines Standorts?

17.13 Weiterführende Links

- Kafka-Dokumentation, Operations: https://kafka.apache.org/documentation/#operations
- Strimzi: https://strimzi.io/documentation/
- Cruise Control: https://github.com/linkedin/cruise-control

18 Fallstudie: Bestandsverwaltung

Ein Quellsystem liefert Vertragsänderungen an die Bestandsverwaltung, heute über eine Queue. Die Bestandsverwaltung arbeitet sie mit mehreren Consumern parallel ab. Zu Spitzenzeiten – Beitragsanpassung zum Jahreswechsel, Monatsabschluss – kommen mehr Änderungen an, als verarbeitet werden können. **Zwei Anforderungen: Kafka soll puffern, und auch bei einem Absturz muss die Reihenfolge pro Vertrag erhalten bleiben.** Dieses Kapitel setzt alle Bausteine des Buchs zu einer Lösung zusammen.

Die Beispiele zu diesem Kapitel

Die Fallstudie ist das Modul `18-fallstudie` (früher `bestand-demo`): Quelle, Bestandsverwaltung mit Versionen, Steuerung per HTTP für Last, Absturz und Poison Pill, Fehlerstrategie `anhalten` oder `dlt`. Was die Fallstudie nicht zeigt, zeigen `09a`, `09b` und `09c` am Code: Retry mit Backoff, Outbox, Kafka-Transaktionen.

18.1 Warum die Queue hier an Grenzen stößt

Bei einer klassischen Queue nimmt die nächste freie Instanz die nächste Nachricht. **Zwei Änderungen am selben Vertrag können also gleichzeitig auf zwei Instanzen laufen – und die zweite ist vielleicht zuerst fertig.** Dazu: Gelesen heißt weg. Kein Replay, keine Historie.

```
Queue:   V-1001 v1 → Instanz A (langsam) ─┐
         V-1001 v2 → Instanz B (schnell) ─┴→ Bestand: v2 vor v1 verbucht  ✗
```

18.1.1 Und die neuen Kafka-Queues?

Share Groups sind seit Kafka 4.2 produktionsreif, seit 4.4 mit Dead-Letter-Queue. Mehrere Consumer teilen sich dieselbe Partition, jede Nachricht wird einzeln bestätigt. **Aber: keine Reihenfolge-Garantie – genau wie die heutige Queue.** Ideal für unabhängige Aufgaben wie Dokumente erzeugen oder E-Mails versenden. Für die Bestandsverwaltung bleibt es bei der klassischen Consumer Group mit der Vertragsnummer als Key.

```
./kafka-configs.sh --bootstrap-server kafka1:9092 --alter --entity-type groups --
entity-name dokumente --add-config share.auto.offset.reset=earliest
```

```
./kafka-console-share-consumer.sh --bootstrap-server kafka1:9092 --topic
dokumente.erzeugen --group dokumente
```

```
./kafka-share-groups.sh --bootstrap-server kafka1:9092 --describe --group
dokumente
```

18.2 Kafka als Puffer

In Kafka schreibt der Producer in seinem Tempo, und die Consumer holen in ihrem Tempo. **Das Log ist der Puffer: Die Daten bleiben nach dem Lesen liegen, bis die Retention sie löscht.** Der Consumer Lag ist der Füllstand des Puffers.

Für den Übergang muss nicht alles auf einmal umgebaut werden: Ein MQ-Source-Connector kann die bestehende Queue nach Kafka übertragen (Kapitel 12), und die Consumer werden nacheinander umgestellt.

18.3 Den Puffer dimensionieren

Größe	Formel	Beispiel
Rückstand (Backlog)	(Zufluss – Abfluss) × Dauer des Peaks	(1200/s – 600/s) × 4 h = **8,64 Mio. Änderungen**
Speicher	Backlog × Nachrichtengröße × Replication Factor	8,64 Mio. × 2 KB × 3 ≈ **52 GB**
Abbauzeit	Backlog ÷ Überschuss nach dem Peak	8,64 Mio. ÷ (600/s – 100/s) ≈ **4,8 h**

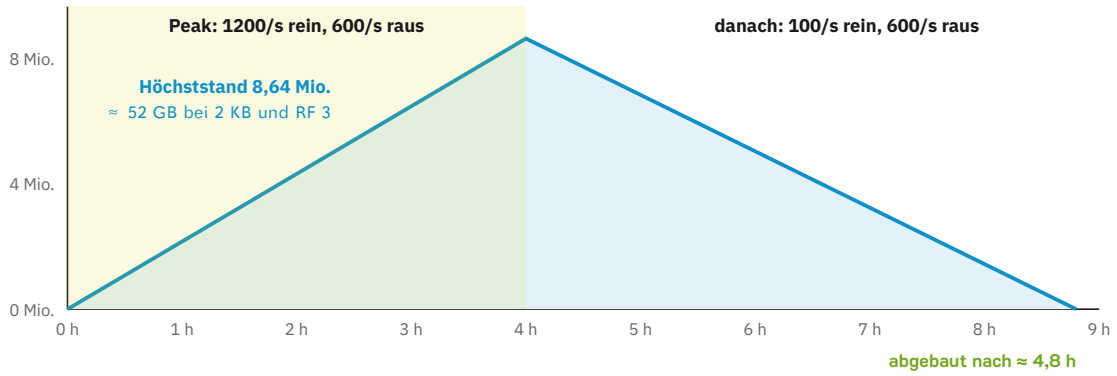


Abbildung 14: Rückstand während und nach dem Peak

Annahmen des Beispiels: Die Bestandsverwaltung schafft insgesamt 600 Änderungen pro Sekunde; im Peak kommen 1200 an, danach 100.

Die Retention muss Peak, Abbauzeit und eine Reserve abdecken – hier mindestens 9 Stunden, sinnvoll mehrere Tage, damit auch ein Ausfall übers Wochenende aufgeholt werden kann. `retention.bytes` setzen, damit die Platte nie voll läuft (pro Partition!), und `RetentionSizeInPercent` überwachen.

18.4 Reihenfolge pro Vertrag

Die Vertragsnummer wird der Key. Gleicher Key, gleiche Partition, und innerhalb einer Partition ist die Reihenfolge garantiert. V-1001 und V-1003 können in derselben Partition liegen – ihre Änderungen bleiben jeweils in Reihenfolge. Eine globale Reihenfolge über alle Verträge braucht die Bestandsverwaltung nicht. Und die Partitionen bestimmen die Parallelität: sechs Partitionen, bis zu sechs Instanzen.

18.5 Parallel verarbeiten, ohne die Reihenfolge zu brechen

- Pro Partition genau ein Thread, innerhalb der Partition streng nacheinander.
- In Spring Kafka: `concurrency` höchstens so groß wie die Anzahl Partitionen.
- `max.poll.records` klein halten (Kapitel 15).
- Das neue Gruppenprotokoll macht Rebalances ruhiger.

- **Nie die Datensätze an einen Thread-Pool weiterreichen – dann überholen sich die Änderungen.**

18.6 Lab: Skaliert die Bestandsverwaltung?

Die Quelle liefert 100 Änderungen pro Sekunde an 1000 Verträge, eine Instanz der simulierten Bestandsverwaltung schafft 50:

```
./kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic
bestand.aenderungen --partitions 6 --replication-factor 3
```

```
bash /opt/lab/bin/mit-metriken 9201 kafka-producer-perf-test.sh --bootstrap-server
kafka1:9092 --topic bestand.aenderungen --num-records 100000 --record-size 500 --
throughput 100 --record-key-range 1000 --key-distribution random --command-
property client.id=quelle
```

```
bash /opt/lab/bin/bestandsverwaltung 1
```

Nach je ein bis zwei Minuten eine weitere Instanz in einem neuen Fenster:

```
bash /opt/lab/bin/bestandsverwaltung 2
```

```
bash /opt/lab/bin/bestandsverwaltung 3
```

Den Lag alle 30 Sekunden abfragen und die Zuordnung ansehen:

```
./kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group
bestand --members
```

Instanzen	Abfluss	Puffer
1	50/s	läuft voll
2	100/s	Gleichgewicht – wächst nicht, schrumpft nicht
3	150/s	wird abgebaut

Erst mit drei Instanzen wird der Puffer abgebaut. Man braucht Überkapazität, nicht nur gleich viel Kapazität wie Zufluss – genau die Abbauzeit aus der Formel.

18.7 Wenn der Producer abstürzt

Fall	Folge	Lösung
Netzwerkfehler	Producer wiederholt	idempotenter Producer: keine Duplikate, keine Umsortierung (Standard seit Kafka 3)
Absturz vor der Bestätigung	Nachricht gilt als nicht geschrieben	Quelle sendet erneut – Outbox-Muster (Kapitel 9)
Absturz nach der Bestätigung	Nachricht ist sicher; das Relay schickt sie vielleicht noch einmal	Consumer verträgt Duplikate

18.8 Wenn der Consumer abstürzt

```
Instanz A: committet bis Offset 149 | verarbeitet 150 … 160 | ✗ Absturz
Instanz B:                          | übernimmt nach Rebalance, beginnt bei 150
                                    └─► 150 … 160 werden ein zweites Mal geliefert
```

Die Reihenfolge bleibt erhalten – aber 150 bis 160 werden doppelt verarbeitet. Das ist at-least-once. Deshalb: erst verarbeiten, dann committen, und die Verarbeitung idempotent machen – mit einer Versionsnummer pro Vertrag. Eine Änderung mit einer Version, die schon verbucht ist, wird ignoriert.

18.9 Die Poison Pill

Eine Änderung scheitert bei jedem Versuch. Endlos wiederholen blockiert die Partition. Nicht blockierend wiederholen und danach ins Dead-Letter-Topic – das ist der Standardweg in Spring. **Für die Bestandsverwaltung ist das falsch:** Die nächste Änderung am selben Vertrag würde die fehlerhafte überholen. **Hier gilt: blockierend wiederholen, dann den Container anhalten und alarmieren.** Ein Mensch entscheidet, wie es weitergeht. Für unabhängige Nachrichten bleibt das DLT richtig – aber dann immer überwachen. Die Umsetzung in Spring zeigt Kapitel 9.

18.10 Die Spring-Boot-Anwendung bestand-demo

Die `bestand-demo` setzt die Fallstudie als Spring-Boot-Anwendung um: Quelle und Bestandsverwaltung in einer App, Statusseite im Browser, Metriken für Prometheus.

```
bestand-demo/
├── bestand-demo.jar                          ← fertig gebaut, Java 17–25
├── src/main/java/de/konitzer/bestand/
│   ├── Quelle.java                           ← Producer, Version je Vertrag
│   ├── Bestandsverwaltung.java               ← Listener, Versionsprüfung
│   ├── KafkaKonfiguration.java               ← Topic als Code, Fehlerbehandlung
│   └── DemoController.java                   ← REST-Endpunkte für die Szenen
└── src/main/resources/application.properties ← alle Einstellungen, kommentiert
```

```
spring.kafka.producer.acks=all
spring.kafka.producer.properties.enable.idempotence=true
```

```
spring.kafka.consumer.group-id=bestandsverwaltung-app
spring.kafka.consumer.enable-auto-commit=false
spring.kafka.consumer.max-poll-records=10
spring.kafka.consumer.properties.group.protocol=consumer
# 3 Threads für 6 Partitionen, je Partition streng nacheinander
spring.kafka.listener.concurrency=3
spring.kafka.listener.ack-mode=batch
# Poison Pill: "anhalten" (richtig für abhängige Änderungen) oder "dlt"
# (zum
# Vergleich)
bestand.fehlerbehandlung=anhalten
```

18.10.1 Szenen

Szene	Aufruf (PowerShell)	Was man sieht
Start	`java -jar bestand-demo.jar`, Browser auf http://localhost:8081	Statusseite, 6 Partitionen
Peak	`curl.exe -X POST "http://localhost:8081/api/quelle/last?anzahl=6000&proSekunde=200"`	Lag steigt
zweite Instanz	`java -jar bestand-demo.jar --server.port=8082`	Partitionen 3 : 3, Lag sinkt
Absturz	`curl.exe -X POST http://localhost:8082/api/betrieb/absturz`	45 s Stille (Session-Timeout), dann übernimmt Instanz 1; im Log `DUPLIKAT`, „Reihenfolge verletzt“ bleibt 0
Poison Pill	`curl.exe -X POST "http://localhost:8081/api/quelle/fehler?vertrag=V-1001"`	drei Versuche, dann `ANHALTEN`; der Lag wächst, nichts geht verloren
Korrektur	…`/api/betrieb/korrigieren`, dann …`/api/betrieb/starten` (POST)	die wartenden Versionen werden in Reihenfolge verbucht
Vergleich DLT	`java -jar bestand-demo.jar --bestand.fehlerbehandlung=dlt`	Folgeänderung meldet `REIHENFOLGE VERLETZT`, fehlerhafte liegt im DLT

18.11 Fallstricke für die Reihenfolge

1. Partitionen nachträglich erhöhen.
2. Dead-Letter-Topic oder Retry-Topics für abhängige Änderungen.
3. Commit vor der Verarbeitung.
4. Asynchrone Verarbeitung im Thread-Pool.
5. Share Groups statt einer Consumer Group.

18.12 Den Puffer überwachen

Den Lag sehen Sie als Client-Metrik und von außen – und Sie brauchen beides, denn steht eine Instanz, verschwindet ihre Metrik. **Alarm ist nicht „Lag größer null“ – bei einem Puffer ist Lag gewollt.** Alarm ist: Der Lag wächst länger als ein Peak dauert, oder die Abbauzeit überschreitet

eine fachliche Frist (etwa: Beitragsänderungen müssen bis zum nächsten Inkassolauf verbucht sein). `RetentionSizeInPercent` zeigt, ob der Puffer an seine Grenze kommt.

18.13 Checkliste: Puffer mit Reihenfolge

- ☐ Key = Vertragsnummer
- ☐ Partitionen nach Zieldurchsatz geplant, mit Reserve – später nicht erhöhen
- ☐ Producer sicher: `acks=all`, Idempotenz, Outbox
- ☐ Commit nach der Verarbeitung, Verarbeitung idempotent (Version je Vertrag)
- ☐ `max.poll.records` passend zur Verarbeitungszeit
- ☐ Poison Pill: anhalten statt DLT
- ☐ Alarme auf wachsenden Lag, Abbauzeit und Retention-Füllstand
- ☐ Retention deckt Peak, Abbauzeit und Reserve

Sind alle acht Punkte erfüllt, bleibt die Reihenfolge pro Vertrag erhalten – auch wenn Producer oder Consumer abstürzen.

19 Referenzarchitektur und Desaster Recovery

Bisher ging es um einen Cluster. Zum Schluss geht der Blick auf die ganze Plattform: Welche Fehler können auftreten, woran erkennt man sie – und was passiert, wenn ein ganzes Rechenzentrum ausfällt?

Grundlage: Buch Zelenin/Kropp Kap. 14 und 16; Kafka-Dokumentation 4.4

19.1 Was kann schiefgehen – und woran sehe ich es?

Fehlerart	Erkennbar an	Reaktion
Netzwerk	Request-Latenz, ISR-Shrinks	Netz prüfen, Broker-Logs
ausgefallener Broker	unterreplizierte Partitionen, `KafkaMetricsDown`	Broker starten; Kafka gleicht in der Zwischenzeit aus
Speicher	`RetentionSizeInPercent`, Offline-Log-Verzeichnisse, volle Platte	Retention begrenzen, Platte tauschen (`cordoned.log.dirs`)
ganzes Rechenzentrum	alles rot	zweiter Standort

19.2 Ausfallsicher über Standorte

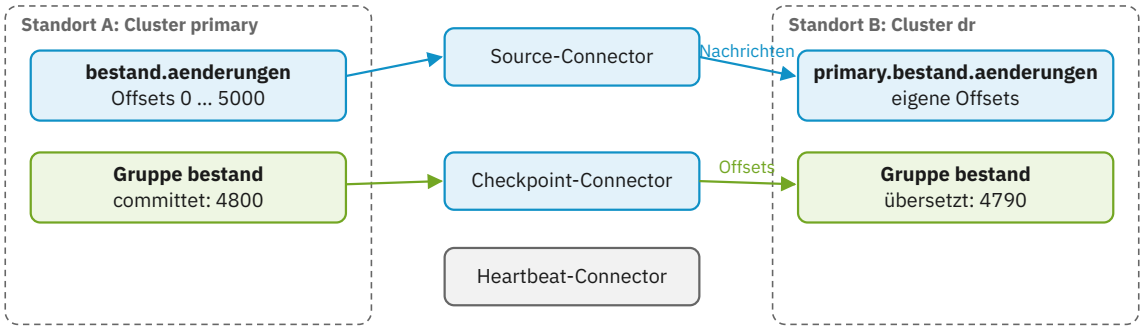
Variante	Voraussetzung	Eigenschaften
Stretched Cluster	mindestens drei Rechenzentren mit geringer Latenz	ein Cluster über alle Standorte; `broker.rack` = Rechenzentrum; synchron, kein Failover nötig
Zwei Cluster + MirrorMaker 2	beliebige Entfernung	asynchrone Kopie; bei Ausfall bewusster Failover, kleiner Datenverlust möglich

Mit `broker.rack` verteilt Kafka die Repliken über Racks oder Rechenzentren. Mit `client.rack` lesen Consumer vom nächstgelegenen Replikat – das spart Verkehr zwischen den Standorten. Bei nur zwei Rechenzentren reicht ein Stretched Cluster nicht: Der Standort mit der Mehrheit der Controller und Kopien ist ein Single Point of Failure.

19.3 MirrorMaker 2

MirrorMaker 2 ist ein Kafka-Bordmittel und besteht aus drei Connectoren:

Connector	Aufgabe
`MirrorSourceConnector`	kopiert Topics – im Ziel mit dem Präfix des Quell-Clusters, z. B. `primary.bestand.aenderungen`
`MirrorCheckpointConnector`	übersetzt die Offsets der Consumer Groups, denn im Ziel haben die Nachrichten andere Offsets
`MirrorHeartbeatConnector`	zeigt, dass die Strecke lebt



Failover: Consumer am Standort B mit derselben Gruppe starten – er liest ab dem übersetzten Offset.

Abbildung 15: MirrorMaker 2 spiegelt Nachrichten und übersetzt Offsets

Die Konfiguration im Lab:

```
clusters = primary, dr
primary.bootstrap.servers = kafka1:9092,kafka2:9092,kafka3:9092
dr.bootstrap.servers = kafka-dr:9092

primary->dr.enabled = true
primary->dr.topics = bestand\\..*, orders
dr->primary.enabled = false

# Offsets der Gruppen im Ziel nachführen – Voraussetzung für den Failover
primary->dr.sync.group.offsets.enabled = true
primary->dr.sync.group.offsets.interval.seconds = 5
primary->dr.emit.checkpoints.interval.seconds = 5
```

Gestartet wird es mit `connect-mirror-maker.sh mm2.properties` oder als Connectoren in einem Connect-Cluster. Im Lab steht der Replikationsfaktor auf 1, weil der DR-Cluster nur einen Knoten hat – in Produktion 3.

19.4 Lab: Spiegeln und Failover

Zweiten Standort starten (PowerShell, Ordner `kafka-lab`):

```
docker compose --profile dr up -d
```

```
docker exec -it kafka-dr bash
```

```
cd /opt/kafka/bin
```

Gespiegelte Topics heißen `primary.<topic>`:

```
./kafka-topics.sh --bootstrap-server kafka-dr:9092 --list
```

Die Gruppe `bestand` existiert im DR-Cluster, obwohl dort nie ein Consumer lief – **das ist die Übersetzung der Offsets**:

```
./kafka-consumer-groups.sh --bootstrap-server kafka-dr:9092 --describe --group
bestand
```

Failover: Alle Instanzen der Bestandsverwaltung am primären Standort beenden, etwa 10 Sekunden warten, bis der Checkpoint die Offsets übertragen hat. Dann dieselbe Gruppe am DR-Cluster starten:

```
./kafka-console-consumer.sh --bootstrap-server kafka-dr:9092 --topic
primary.bestand.aenderungen --group bestand --command-property
group.protocol=consumer --formatter-property print.key=true --formatter-property
print.partition=true
```

Sie liest dort weiter, wo die Bestandsverwaltung am primären Standort aufgehört hat – nicht von vorn, nicht vom Ende.

19.4.1 Hintergrund und Konsequenzen

- **Asynchron heißt:** Was in den letzten Sekunden vor dem Ausfall geschrieben, aber noch nicht gespiegelt wurde, fehlt am DR-Standort. Die Outbox der Quelle (Kapitel 9) kann diese Lücke schließen.
- **Offsets sind näherungsweise übersetzt:** Nach dem Failover kann eine Gruppe einige Nachrichten erneut lesen – idempotente Verarbeitung vorausgesetzt.
- **Topic-Namen ändern sich** (`primary.…`). Anwendungen müssen am DR-Standort das andere Topic lesen – per Konfiguration oder mit Mustern. Alternativ `IdentityReplicationPolicy` für gleiche Namen, dann aber ohne Schutz gegen Schleifen.
- **Failback ist der schwierigere Teil:** Nach der Rückkehr muss der primäre Standort die am DR-Standort geschriebenen Daten bekommen. Failover und Failback regelmäßig üben – wer es erst im Ernstfall probiert, hat ein Problem.

19.5 Desaster Recovery überwachen

Metrik	Bedeutung
Replikationslatenz	wie lange braucht eine Nachricht zum DR-Standort
Alter der gespiegelten Daten	wie aktuell ist der DR-Standort
Checkpoint-Latenz	wie aktuell sind die übersetzten Offsets

Die Alert-Regel `MirrorMakerLagging` schlägt an, wenn die Spiegelung länger als 30 Sekunden verzögert ist.

19.6 Backups in Kafka

- **Oft nicht nötig,** wenn die Quelle die Daten noch hat – bei der Bestandsverwaltung die Datenbank und die Outbox des Bestandssystems.
- **Snapshots einzelner Broker** sind zeitlich inkonsistent und ergeben keinen gemeinsamen Zeitpunkt.
- **MirrorMaker ist kein Backup gegen Bedienfehler:** Löscht jemand ein Topic, ist es im DR-Cluster bald auch weg. Für langfristige Archivierung: ein Sink in einen Objektspeicher oder Tiered Storage mit eigener Aufbewahrung.

19.7 Neu seit dem Buch

Thema	Was	Seit
Eligible Leader Replicas	sicherere Leader-Wahl ohne Unclean Election	Standard ab 4.1
Neues Consumer-Protokoll	ruhigere Rebalances, Broker koordiniert	produktionsreif ab 4.0, Standard ab 5.0
Share Groups	Queues ohne Reihenfolge	produktionsreif ab 4.2, DLQ ab 4.4
Tiered Storage	alte Segmente im Objektspeicher; lange Retention günstig	produktionsreif ab 3.9
neue Metriken	`RetentionSizeInPercent` (4.3), Platten-I/O (4.4), korrekte Idle-Werte im kombinierten Modus (4.2)	4.2–4.4

19.8 Ihr Zielbild

Drei Fragen, die jede Plattform für sich beantworten sollte:

1. Wo läuft Kafka – On-Prem, auf OpenShift mit einem Operator oder als Managed Service?
2. Wer verantwortet Monitoring und Alerting – das Plattformteam oder die Anwendungsteams?
3. Wie viele Rechenzentren gibt es – Stretched Cluster oder DR-Cluster?

Es gibt keine allgemein richtige Antwort – aber für die eigene Plattform sollte man sie kennen.

19.9 Weiterführende Links

- Kafka-Dokumentation, Geo-Replication: https://kafka.apache.org/documentation/#georeplication

20 Wenn-Dann: Situationen und ihre Folgen

Dieses Kapitel ist das Nachschlagewerk zum Seminar. Jede Situation steht als Wenn-Satz, dann folgt, was passiert, warum, woran man es erkennt, was zu tun ist und mit welchem Lab aus `labs` man es nachstellt. Die Reihenfolge folgt dem Seminar: Grundlagen, Zuverlässigkeit, Consumer Groups, Betrieb. Am Ende stehen die typischen Fehler beim Aufsetzen der Plattform, die Zielwerte für erhöhten Ausfallschutz und die Stellschrauben für mehr Durchsatz.

Die Folien dazu

Deck 10 „Wenn-Dann“ zeigt dieselben Situationen, eine je Karte. Im Rückblick an Tag 2 wird nur der Wenn-Satz gezeigt, die Gruppe sagt das Dann.

20.1 Grundlagen

20.1.1 Wenn der Producer ohne Key sendet

Dann verteilt der Sticky Partitioner die Nachrichten batchweise auf die Partitionen. Zwei Änderungen desselben Vertrags können in verschiedenen Partitionen landen und von verschiedenen Instanzen in beliebiger Reihenfolge verarbeitet werden.

Warum	Ohne Key gibt es nichts zu hashen. Kafka optimiert dann auf Durchsatz (volle Batches je Partition), nicht auf Reihenfolge.
Woran erkenne ich es	`03-consumer.ps1` zeigt Partition und Key; der Key ist `null`, die Partition wechselt. Fachlich: „Reihenfolge verletzt“, Version 3 vor Version 2.
Was tun	Den fachlichen Schlüssel als Key setzen (Vertragsnummer), nicht den Zeitstempel und nicht eine UUID je Nachricht.
Lab	`01-shell/02-producer.ps1` mit einer Zeile ohne Doppelpunkt; `02-producer-plain` meldet die Partition je Nachricht.

20.1.2 Wenn ein Consumer ohne Gruppe neu gestartet wird

Dann beginnt er als Fremder: Ohne gespeicherten Offset liest er nur Neues (oder mit `--from-beginning` alles); die Nachrichten dazwischen hat er nie gesehen.

Warum	Ohne `group.id` gibt es keinen Eintrag in `__consumer_offsets`. Der Console-Consumer bekommt eine Wegwerf-Gruppe, die nichts committet.
Woran erkenne ich es	`kafka-consumer-groups.sh --list` zeigt `console-consumer-12345`, jedes Mal eine andere; nach dem Neustart fehlen Nachrichten, obwohl sie im Topic liegen.
Was tun	Für Werkzeuge in Ordnung. Jede Anwendung, die verarbeitet, bekommt eine feste Gruppe mit dem Namen der Anwendung.
Lab	`01-shell/03-consumer.ps1` (ohne Gruppe) gegen `05-consumer-gruppe.ps1` (mit Gruppe), dazwischen Nachrichten senden.

20.2 Zuverlässigkeit

20.2.1 Wenn der Producer mit acks=1 sendet und der Leader direkt nach der Bestätigung ausfällt

Dann ist die Nachricht bestätigt, aber verloren: Der Leader hatte sie, kein Follower hatte sie kopiert, der neue Leader kennt sie nicht.

Warum	`acks=1` wartet nur auf den Leader. Die Follower holen die Nachricht erst danach ab. Ein Ausfall in diesem Fenster löscht sie, ohne dass der Producer es erfährt.
Woran erkenne ich es	Nicht zuverlässig. Im Monitoring höchstens `IsrShrinksPerSec` und ein Leader-Wechsel zur selben Zeit; die Lücke fällt erst fachlich auf.
Was tun	`acks=all` und `enable.idempotence=true` (Default seit Kafka 3.0), `min.insync.replicas=2`. `acks=1` nur für Daten, die verloren gehen dürfen.
Lab	`15-performance` mit `--spring.kafka.producer.acks=1` zeigt den Durchsatzgewinn, Kapitel 5 die Tabelle der Kombinationen.

20.2.2 Wenn der Producer mit acks=all sendet und weniger synchrone Kopien da sind als min.insync.replicas

Dann lehnt der Broker das Schreiben ab (`NOT_ENOUGH_REPLICAS`). Der Producer wiederholt bis `delivery.timeout.ms` (Standard 2 Minuten) und meldet dann im Callback einen Fehler. Die Nachricht ist nicht geschrieben.

Warum	`acks=all` heißt „alle Kopien im ISR“, und `min.insync.replicas` sagt, wie viele das mindestens sein müssen. Mit RF 3 und Minimum 2 darf ein Broker fehlen; fehlen zwei, verweigert Kafka lieber, als auf einer Einzelkopie zu bestätigen.
Woran erkenne ich es	Producer-Log `NOT_ENOUGH_REPLICAS … retrying`, Metrik `UnderMinIsrPartitionCount` > 0 (Alert UnderMinIsr), `kafka-topics.sh --describe --under-min-isr-partitions`.
Was tun	Fehlenden Broker zurückholen. Nicht `min.insync.replicas` senken, das tauscht Verfügbarkeit gegen Verlustrisiko. Anwendung: Fehler im Callback auswerten, sonst merkt es niemand.
Lab	`04-producer-spring`, Aufgabe 2 (`docker stop kafka3`, `min.insync.replicas=3`, `delivery.timeout.ms=5000`); `01-shell/04-topic-ansehen.ps1` zeigt die Isr-Spalte.

20.2.3 Wenn ein Broker ausfällt

Dann werden die Partitionen, deren Leader er war, innerhalb von Sekunden an Follower aus dem ISR übergeben. Producer und Consumer holen neue Metadaten und machen weiter. Schreiben läuft, solange `min.insync.replicas` erfüllt bleibt.

Warum	Replikation. Jede Partition hat drei Kopien, der Controller wählt aus den synchronen Kopien einen neuen Leader. Die Clients brauchen dafür keinen Code; `bootstrap.servers` mit mehreren Adressen reicht.
Woran erkenne ich es	`UnderReplicatedPartitions` > 0 (Alert nach 5 Minuten), `ActiveBrokerCount` 2, im Producer-Log ein paar `NOT_LEADER_OR_FOLLOWER … retrying`, danach Ruhe.
Was tun	Broker zurückholen. Nach dem Neustart ist er nirgends Leader: Preferred Leader Election verteilt die Last wieder (Alert LeaderImbalance).
Lab	`docker stop kafka3`, `01-shell/04-topic-ansehen.ps1 -Topic orders`, `14-monitoring/shell/01-gesundheit.sh`, dann `03-leader.sh`; `04-producer-spring --app.automatisch=true` läuft durch.

20.2.4 Wenn zwei von drei Brokern ausfallen

Dann steht der Cluster: Partitionen mit `min.insync.replicas=2` nehmen nichts mehr an, und im Lab fällt zusätzlich das KRaft-Quorum, sodass keine Leader-Wahl und keine Metadatenänderung mehr möglich ist.

Warum	RF 3 und Minimum 2 überleben genau einen Ausfall. Das Controller-Quorum braucht die Mehrheit; mit einem von drei Knoten gibt es keine.
Woran erkenne ich es	`ActiveControllerCount` 0 (Alert ControllerMissing), `OfflinePartitionsCount` > 0, UnderMinIsr; Producer melden `NOT_ENOUGH_REPLICAS`, Consumer bekommen Timeouts.
Was tun	Broker zurückholen, nichts umkonfigurieren. In Produktion eigene Controller-Knoten (3 oder 5), Broker über Zonen verteilt (`broker.rack`).
Lab	Nicht ausführen. Im Lab nie zwei Broker gleichzeitig stoppen.

20.3 Consumer Groups und Offsets

20.3.1 Wenn zwei Instanzen derselben Anwendung in derselben Gruppe laufen

Dann verteilt der Koordinator die Partitionen auf beide, jede Nachricht wird genau einmal gelesen, und fällt eine Instanz aus, übernimmt die andere. Das ist der gewollte Fall: Skalierung und Ausfallsicherheit mit demselben Mechanismus.

Warum	Jede Partition hat zu jedem Zeitpunkt genau ein Mitglied. Mitglieder sind Threads, keine Zeilen Code: die Anwendung zweimal starten oder `concurrency` erhöhen.
Woran erkenne ich es	Im Log `ZUGEWIESEN: {beitraege-0=…}` je Instanz, `ABGEGEBEN` beim Rebalance; `--members` zeigt die Verteilung.
Was tun	So viele Mitglieder wie nötig, höchstens so viele wie Partitionen. Welches Mitglied welche Partition bekommt, ist nicht Sache der Anwendung.
Lab	`06-consumer-groups` in zwei Fenstern, dann eines beenden; `01-shell/05-consumer-gruppe.ps1` zweimal.

20.3.2 Wenn mehr Mitglieder in der Gruppe sind als Partitionen

Dann bekommen die überzähligen Mitglieder nichts. Sie laufen, senden Heartbeats, verarbeiten aber keine Nachricht. Mehr Instanzen bringen ab diesem Punkt keinen Durchsatz.

Warum	Eine Partition ist die kleinste Einheit, die ein Mitglied bekommen kann; sie lässt sich nicht teilen.
Woran erkenne ich es	`--members` zeigt Mitglieder mit leerer Partitionsliste; im Log `ZUGEWIESEN: {}`; `records-lag-max` sinkt trotz neuer Instanz nicht.
Was tun	Die Partitionszahl ist die Obergrenze der Parallelität. Mehr Durchsatz braucht mehr Partitionen (vorher einplanen) oder schnellere Verarbeitung je Nachricht.
Lab	`06-consumer-groups --app.threads=3` plus ein viertes Fenster; Aufgabe 2 der ANLEITUNG.

20.3.3 Wenn zwei verschiedene Anwendungen versehentlich dieselbe Consumer Group benutzen

Dann teilen sie sich die Partitionen: Die Bestandsverwaltung sieht Partition 0 und 1, das Inkasso sieht Partition 2. Beide bekommen nur einen Teil der Nachrichten, ohne Fehlermeldung.

Warum	Für Kafka ist die Gruppe die Anwendung. Zwei Prozesse mit demselben `group.id` sind zwei Mitglieder derselben Anwendung, und Mitglieder teilen sich die Arbeit.
Woran erkenne ich es	`--members` zeigt zwei Mitglieder mit verschiedenen `client.id` oder Hosts; fachlich fehlen jeder Anwendung Verträge, scheinbar zufällig.
Was tun	Eine Gruppe je Anwendung, Name = Name der Anwendung, nie als Kopie aus einem anderen Projekt. Gruppennamen in der Topic-Dokumentation festhalten.
Lab	`05-consumer-spring` und `06-consumer-groups --app.gruppe=beitragsrechner-spring` gleichzeitig starten: jede App sieht nur einen Teil.

20.3.4 Wenn ein Mitglied abstürzt, ohne sich abzumelden

Dann merkt der Koordinator es erst nach `session.timeout.ms` (45 Sekunden). So lange steht die Partition still. Danach bekommt ein anderes Mitglied sie und beginnt beim letzten Commit: Alles, was das abgestürzte Mitglied verarbeitet, aber nicht committet hatte, kommt noch einmal.

Warum	Ein Prozess, der hart stirbt, sendet keine Leave-Nachricht. Kafka wartet, bis die Heartbeats ausbleiben. Der Commit ist die einzige Wahrheit über den Fortschritt.
Woran erkenne ich es	Lag der betroffenen Partition steigt 45 Sekunden lang; dann Rebalance; im Log der übernehmenden Instanz `DUPLIKAT`-Zeilen.
Was tun	Verarbeitung idempotent machen (Versionsnummer, Upsert); `session.timeout.ms` nur senken, wenn das Netz stabil ist; sauberes Herunterfahren bevorzugen (Spring stoppt den Container, das ist ein Leave).
Lab	`18-fallstudie`, Szene Absturz; `06-consumer-groups` Aufgabe 3 (`acknowledge()` weglassen).

20.3.5 Wenn der Consumer committet, bevor er verarbeitet hat

Dann gilt die Nachricht als erledigt, bevor sie es ist. Stürzt die Verarbeitung ab, ist die Nachricht für diese Gruppe verloren: at most once.

Warum	Der Offset sagt „bis hierher fertig“. Wer ihn vor der Arbeit setzt, verspricht etwas, das er noch nicht getan hat. Auto-Commit alle 5 Sekunden tut genau das, wenn die Verarbeitung in einem anderen Thread läuft.
Woran erkenne ich es	Fachlich fehlende Nachrichten ohne Lag und ohne Fehler im Log; schwer zu finden.
Was tun	`enable-auto-commit=false` und Commit nach der Verarbeitung (Spring: `ack-mode=record` oder `manual_immediate` mit `acknowledge()` am Ende). Nie Nachrichten an einen Thread-Pool weiterreichen und sofort committen.
Lab	`03-consumer-plain` zeigt `commitSync` nach der Schleife; `05-consumer-spring` Aufgabe 3.

20.3.6 Wenn die Verarbeitung eines Poll-Pakets länger dauert als max.poll.interval.ms

Dann wirft der Koordinator das Mitglied aus der Gruppe. Seine Partitionen gehen an ein anderes Mitglied, das dasselbe Paket noch einmal bekommt, ebenfalls zu lange braucht und ebenfalls fliegt: der Rebalance-Sturm. Fortschritt null, Duplikate viele.

Warum	`poll()` ist das Lebenszeichen. Wer fünf Minuten (Default) nicht pollt, gilt als hängend.
Woran erkenne ich es	Log `consumer poll timeout has expired`, Metrik `rebalance-total` steigt ständig, `time-between-poll-avg` nahe `max.poll.interval.ms`, Lag bewegt sich nicht.
Was tun	`max.poll.records` senken (die Fallstudie nimmt 10), Verarbeitung beschleunigen oder auslagern, `max.poll.interval.ms` nur als letztes Mittel erhöhen.
Lab	`03-consumer-plain` Aufgabe 4; Kommentar zu `max-poll-records` in `18-fallstudie/application.properties`.

20.3.7 Wenn eine Gruppe länger steht als offsets.retention.minutes

Dann löscht Kafka ihre Offsets (Standard nach 7 Tagen ohne Mitglied). Beim nächsten Start gibt es keinen Fortschritt mehr, und `auto.offset.reset` entscheidet: `latest` überspringt alles seitdem, `earliest` liest alles noch einmal.

Warum	Offsets ohne Gruppe sind für Kafka Abfall; die Retention räumt auf. Ein Dienst, der im Urlaub, in einer abgeschalteten Umgebung oder nach einem langen Incident steht, verliert seinen Platz.
Woran erkenne ich es	`kafka-consumer-groups.sh --describe` zeigt die Gruppe nicht mehr; nach dem Start plötzlich Lag 0 bei `latest` (Daten übersprungen) oder riesiger Lag bei `earliest`.
Was tun	`offsets.retention.minutes` länger als die längste geplante Pause; für selten laufende Jobs `earliest` plus idempotente Verarbeitung; auf OpenShift prüfen, wie lange Pods bewusst gestoppt bleiben.
Lab	Nicht in zwei Tagen nachstellbar; `01-shell/06-gruppe-ansehen.ps1` zeigt, welche Gruppen Kafka noch kennt.

20.4 Betrieb

20.4.1 Wenn der Lag wächst und wächst

Dann kommt die Verarbeitung dauerhaft nicht nach. Erst füllt sich der Puffer (gewollt), dann nähert sich die Leseposition dem ältesten Offset, und die Retention löscht Nachrichten, die nie gelesen wurden.

Warum	Zufluss größer als Abfluss. Ein Peak ist in Ordnung, wenn danach Überkapazität da ist; dauerhaft mehr Zufluss braucht mehr Parallelität.
Woran erkenne ich es	`records-lag-max` steigt linear; `records-lead-min` sinkt gegen 0 (Gefahr); Alert ConsumerLagHigh; von außen `kafka-consumer-groups.sh --describe`, auch wenn der Consumer steht.
Was tun	Instanzen oder Threads bis zur Partitionszahl, Verarbeitung je Nachricht beschleunigen, Retention so bemessen, dass Peak plus Abbauzeit hineinpassen (Kapitel 18, Backlog-Formel).
Lab	`06-consumer-groups --app.dauer-ms=2000` gegen `04-producer-spring --app.automatisch=true`; `14-monitoring/shell/07-lag-erzeugen.sh` für den Alert.

20.4.2 Wenn die Partitionszahl eines Topics nachträglich erhöht wird

Dann ändert sich die Zuordnung Key → Partition: V-1001 lag bisher in Partition 1 und landet künftig in Partition 4. Alte und neue Änderungen desselben Vertrags liegen in zwei Partitionen, und zwei Instanzen können sie überholen.

Warum	Die Partition ist `murmur2(Key)` modulo Partitionsanzahl. Ändert sich der Teiler, ändert sich das Ergebnis. Bestehende Nachrichten bleiben, wo sie sind.
Woran erkenne ich es	`10-partition-fuer-key.ps1` vor und nach der Änderung; die Partition springt. Fachlich: Versionslücken oder Duplikate bei genau den Verträgen, die den Übergang treffen.
Was tun	Partitionszahl von Anfang an großzügig wählen (12 als Standard) und als Entscheidung dokumentieren. Muss sie geändert werden: Producer pausieren, Consumer leer laufen lassen, dann erhöhen, dann Producer wieder starten.
Lab	`01-topic-anlegen.ps1 -Topic test6 -Partitionen 6`, dann `10-partition-fuer-key.ps1 -Topic test6` gegen das Ergebnis bei `beitraege`.

20.4.3 Wenn eine Nachricht bei jedem Versuch scheitert

Dann blockiert sie ihre Partition, solange die Anwendung sie wiederholt. Wird sie übersprungen oder in ein Dead-Letter-Topic geschoben, läuft die Partition weiter, aber die nächste Änderung desselben Vertrags überholt die fehlerhafte.

Warum	Reihenfolge je Partition heißt auch: Eine kaputte Nachricht hält alles dahinter auf. Es gibt keinen Weg ohne Nachteil, nur die Wahl, welchen man trägt.
Woran erkenne ich es	Log `Versuch 1 … 2 … 3 fehlgeschlagen`, Lag der einen Partition steigt, die anderen laufen; bei DLT: Nachrichten im Topic `…-dlt` mit Header `kafka_dlt-exception-cause-fqcn`.
Was tun	Vorübergehende Fehler mit Backoff wiederholen, fachliche nicht. Dann je Topic entscheiden: anhalten und alarmieren, wenn Änderungen voneinander abhängen; Dead-Letter-Topic, wenn Nachrichten unabhängig sind, mit Alert auf das DLT.
Lab	`09a-fehlerbehandlung`, erst anhalten, dann `--app.recoverer=dlt`; `18-fallstudie` Poison Pill.

20.4.4 Wenn eine Nachricht nicht gelesen werden kann

Dann wirft der Deserializer schon vor dem Listener eine Exception, und ohne Vorkehrung versucht der Container es endlos: Die Partition steht, das Log läuft voll.

Warum	Deserialisierung passiert in `poll()`, nicht in der Listener-Methode. Der normale Error Handler sieht die Nachricht nie, weil es nie eine Nachricht gab.
Woran erkenne ich es	Log `Error deserializing key/value` in Dauerschleife, Lag der Partition steht, CPU hoch.
Was tun	`ErrorHandlingDeserializer` vor den echten Deserializer schalten; dann bekommt der Error Handler die kaputte Nachricht als Fehler und kann sie überspringen oder ins DLT legen. Producer-Seite: Schema und Kompatibilitätsregel (Kapitel 10).
Lab	`08-consumer-json/application.properties`; `13-tests` BestandsverwaltungIT; Gegenprobe: Text-Nachricht mit `02-producer.ps1` in `bestand.vertragsaenderungen`.

20.4.5 Wenn die Platte eines Brokers voll läuft

Dann stoppt der Broker sich selbst. Seine Partitionen werden unterrepliziert, seine Leader wandern; bei Partitionen, deren einzige Kopie er hatte, steht die Verarbeitung.

Warum	Kafka schreibt sequenziell und löscht nur über Retention. Ohne Platz kann kein Segment mehr angelegt werden.
Woran erkenne ich es	Vorher: Platte über 80 Prozent, `RetentionSizeInPercent` > 80 (Alert RetentionNearlyFull), `kafka_log_log_size` je Topic. Nachher: `OfflineLogDirectoryCount` > 0, Broker-Log `No space left on device`.
Was tun	`retention.bytes` je Topic, damit die Summe unter die Platte passt; große Topics auf mehr Broker verteilen; 20 Prozent Reserve für Reassignments.
Lab	`15-performance/shell/09-aufraeumen.sh`; `14-monitoring/shell/04-promql.sh` für die Speichermetriken.

20.5 Typische Fehler beim Aufsetzen der Plattform

Die Plattform beim Kunden besteht aus Confluent, Terraform, OpenShift und Spring Boot (Kapitel 17). Die Fehler zeigen sich meist erst beim ersten Ausfall oder beim ersten Peak; alle sind in Modulen und Defaults vermeidbar.

20.5.1 Confluent und Terraform

Wenn	Dann	Was tun
Das Terraform-Modul legt Topics mit den Standardwerten an (1 Partition, RF 1)	keine Ausfallsicherheit; die Partitionszahl muss später erhöht werden, was die Key-Zuordnung bricht	Modul mit Pflichtwerten: Partitionen bewusst gewählt, `replication_factor` 3, `min.insync.replicas` 2, Retention; keine Defaults
`min.insync.replicas` steht nur im Broker-Default 1	`acks=all` schützt nicht: eine einzige Kopie reicht für die Bestätigung	Cluster-Default 2 setzen und im Topic-Modul erzwingen
`auto.create.topics.enable` bleibt eingeschaltet	jeder Tippfehler erzeugt ein Topic mit Standardwerten, an Terraform vorbei	ausschalten; Tippfehler werden als `UNKNOWN_TOPIC` sichtbar
ACLs fehlen oder erlauben als Prefix-ACL alles	die Spring-App liest nichts (`TopicAuthorizationException`) oder jede App darf alles	ACLs im selben Modul wie das Topic: je Anwendung ein Principal, Read auf Topic und Group, Write nur für Producer
Der Terraform-State wird von Hand umgangen (Topic im Control Center geändert)	der nächste `plan` meldet Drift; ein `apply` dreht Einstellungen zurück oder löscht ein Topic mit Daten	`lifecycle prevent_destroy`, Control Center nur lesend, Änderungen nur über die Pipeline
Die Schema Registry läuft mit Kompatibilität NONE	die erste Schema-Änderung bricht alle Consumer mit dem alten Schema	BACKWARD als Standard, Kompatibilität je Subject in Terraform, Schema-Änderung mit Consumer-Deployment koppeln
Broker ohne `broker.rack` über die Zonen verteilt	alle drei Kopien einer Partition können in derselben Zone liegen	`broker.rack` je Zone, Rack Awareness mit `--describe` prüfen

20.5.2 OpenShift und Spring Boot

Wenn	Dann	Was tun
`bootstrap-servers` steht fest im JAR oder enthält nur eine Adresse	jeder Broker-Wechsel ist ein Build; fällt genau dieser Broker aus, startet die App nicht	Adresse aus der Umgebung (`SPRING_KAFKA_BOOTSTRAP_SERVERS`), alle Broker oder ein DNS-Name
Die `group-id` wurde aus einem anderen Service kopiert	zwei Anwendungen teilen sich die Partitionen, jede sieht nur einen Teil	`group-id` = Name der Anwendung, in Terraform-ACLs dokumentiert
`terminationGracePeriodSeconds` ist kürzer als das Herunterfahren	der Pod wird hart beendet, ohne Leave: 45 Sekunden Stillstand, dann Duplikate	Grace Period größer als `max.poll.interval.ms` plus Verarbeitung; `spring.lifecycle.timeout-per-shutdown-phase` setzen
Der HorizontalPodAutoscaler erlaubt mehr Replicas als Partitionen	Pods ohne Partition kosten Ressourcen und bringen nichts	`maxReplicas` = Partitionszahl
Die Liveness-Probe schlägt während Start oder Rebalance zu	OpenShift startet den Pod neu, löst den nächsten Rebalance aus, die Gruppe kommt nie zur Ruhe	`startupProbe` mit genug Zeit, Liveness auf `/actuator/health`, Readiness getrennt
Consumer-Pods stehen länger als `offsets.retention.minutes`	die Offsets sind weg; `auto.offset.reset` überspringt oder wiederholt alles	Retention länger als die längste geplante Pause
Das Monitoring holt nur Broker-Metriken	der Lag ist unsichtbar, sobald die Pods nicht laufen	Lag-Exporter gegen den Cluster, Micrometer in den Pods, Alert auf die externe Messung
`enable-auto-commit` bleibt an, die Verarbeitung läuft asynchron	Commit vor der Verarbeitung: Verlust bei jedem Absturz, ohne Lag und ohne Fehler	`enable-auto-commit=false`, `ack-mode` record oder manual, Verarbeitung im Listener-Thread

20.6 Zielwerte für erhöhten Ausfallschutz

Preis dieser Werte: etwas mehr Latenz und Hardware. Gewinn: Der Cluster übersteht den Ausfall eines Brokers oder einer Zone ohne Datenverlust, die Anwendungen den Ausfall eines Pods ohne Stillstand.

20.6.1 Cluster und Topics

Einstellung	Standard	Bei erhöhtem Ausfallschutz	Warum
Broker und Zonen	3 Broker, eine Zone	mindestens 3 Broker über 3 Zonen, `broker.rack` je Zone	Rack Awareness verteilt die Kopien einer Partition auf Zonen
KRaft-Controller	kombiniert mit Brokern	3 oder 5 eigene Controller-Knoten über Zonen	Broker-Last stört die Metadaten nicht; Quorum überlebt einen Knotenausfall
`replication.factor`	1	3; `default.replication.factor=3` am Broker	zwei Kopien dürfen fehlen, bevor Daten weg sind
`min.insync.replicas`	1	2, am Broker als Default und im Topic-Modul	jede bestätigte Nachricht liegt auf zwei Brokern; bei RF 3 darf einer ausfallen
`unclean.leader.election.enable`	false	false, nie ändern; ELR aktiv lassen	ein Follower außerhalb der ISR darf nicht Leader werden
`offsets.topic.replication.factor`, `transaction.state.log.replication.factor`, `min.isr`	3, 3, 2	3, min.isr 2, prüfen	die internen Topics tragen Offsets und Transaktionen
`auto.create.topics.enable`	true	false	Topics nur mit bewussten Werten aus Terraform
Partitionen je Topic	1	nach Durchsatz und Parallelität, 12 als Standard; nie knapp	Obergrenze der Consumer-Parallelität; Änderung bricht Key-Zuordnung
`retention.bytes` und Plattenreserve	unbegrenzt	Summe unter 70 Prozent der Platte, Alert bei 80 Prozent	volle Platte stoppt den Broker

20.6.2 Producer

Einstellung	Standard	Bei erhöhtem Ausfallschutz	Warum
`acks`	all	all, bewusst ausgeschrieben	Bestätigung erst, wenn `min.insync.replicas` Kopien schreiben
`enable.idempotence`	true	true	keine Duplikate und keine Umsortierung bei Wiederholungen
`retries`, `delivery.timeout.ms`	max, 120000	`retries` lassen, Timeout nach SLA, nie unter `request.timeout.ms` plus `linger.ms`	der Producer überbrückt Leader-Wechsel selbst; danach muss der Fehler im Callback ankommen
`max.in.flight.requests.per.connection`	5 mit Idempotenz	5 mit Idempotenz	Reihenfolge bleibt trotz Pipelining
`bootstrap-servers`	ein Broker	alle Broker oder ein DNS-Name, aus der Umgebung	Start klappt, wenn ein Broker fehlt
Ergebnis von `send()`	oft ignoriert	immer auswerten, Fehler loggen und zählen (`record-error-rate`)	ohne Auswertung ist Datenverlust unsichtbar
`linger.ms`, `batch.size`	5, 16384	20 bis 100 ms, 64 bis 128 KB, lz4 oder zstd	gleiche Sicherheit, weniger Requests je Nachricht
Datenbank und Kafka	zwei Schreibzugriffe	Transactional Outbox (Modul 09b)	kein Zustand, in dem die DB gilt und das Ereignis fehlt
Kafka lesen, verarbeiten, schreiben	zwei Schritte	`transaction-id-prefix`, Consumer mit `read_committed` (Modul 09c)	Offset-Commit und Ausgabe gemeinsam oder gar nicht

20.6.3 Consumer

Einstellung	Standard	Bei erhöhtem Ausfallschutz	Warum
`enable-auto-commit`	true	false	Commit nur, wenn die Verarbeitung durch ist
`ack-mode`	batch	record oder manual_immediate	at least once mit kleinem Wiederholfenster
Idempotenz	keine	Versionsnummer je Schlüssel oder Upsert	nach jedem Absturz kommen Nachrichten doppelt
`max.poll.records`	500	10 bis 100	kleines Wiederholfenster, kein Rauswurf
`max.poll.interval.ms`	300000	so lassen, Verarbeitung darunter halten	sonst Rebalance-Sturm
`group.protocol`	classic	consumer (KIP-848)	Rebalance ohne Stillstand aller Partitionen

Einstellung	Standard	Bei erhöhtem Ausfallschutz	Warum
`group.instance.id`	leer	gesetzt, wenn Pods innerhalb des Session-Timeouts ersetzt werden	ein zurückkehrendes Mitglied behält seine Partitionen
`auto-offset-reset`	latest	earliest für Verarbeitung	fehlt der Offset, lieber wiederholen als überspringen
`concurrency`	1	höchstens Partitionen geteilt durch Instanzen	mehr Threads als Partitionen arbeiten nicht
`offsets.retention.minutes` (Broker)	7 Tage	länger als die längste geplante Pause	sonst verliert ein pausierter Dienst seinen Fortschritt
`isolation.level`	read_uncommitted	read_committed bei Transaktionen	abgebrochene Versuche bleiben unsichtbar

20.6.4 OpenShift, Monitoring und Desaster Recovery

Einstellung	Standard	Bei erhöhtem Ausfallschutz	Warum
`replicas` der Consumer-Deployments	1	mindestens 2 über Zonen (`podAntiAffinity`), PodDisruptionBudget `minAvailable` 1	ein Pod darf jederzeit sterben oder verschoben werden
`terminationGracePeriodSeconds`	30	größer als `max.poll.interval.ms` plus Verarbeitung	sauberer Leave statt 45 Sekunden Stillstand
Probes	Liveness auf Port	`startupProbe` großzügig, Liveness auf `/actuator/health`, Readiness getrennt	kein Neustart während Start oder Rebalance
HPA `maxReplicas`	beliebig	gleich Partitionszahl	darüber hinaus arbeitet niemand
Monitoring	Broker-Metriken	Lag-Exporter, Micrometer in den Pods, Alerts UnderMinIsr, ControllerMissing, UnderReplicatedPartitions, ConsumerLagHigh, ProducerErrors mit Runbook	der gefährlichste Lag entsteht, wenn kein Consumer läuft
Rolling Restart der Broker	beliebig	ein Broker nach dem anderen, weiter erst bei `UnderReplicatedPartitions` 0, danach Preferred Leader Election	zwei Broker gleichzeitig unter RF 3 bedeuten Schreibstopp
Zweiter Standort	keiner	MirrorMaker 2 mit Checkpoint-Connector, Alert MirrorMakerLagging, Failover-Übung je Quartal	Datenverlust auf die Replikationslatenz begrenzt (RPO), Wiederanlauf geprobt (RTO)
Terraform-State	lokal	Backend mit Locking und Versionierung, `prevent_destroy` auf Topics	ein verlorener State macht die Plattform unverwaltbar

20.7 Mehr Durchsatz zur Laufzeit

Zuerst messen, wo es klemmt. Consumer lassen sich sofort hinzufügen, Partitionen nur geplant, Broker nur mit Umzug. Reihenfolge im Betrieb: messen, Batching, Consumer bis zur Partitionszahl, dann Partitionen, zuletzt Broker.

Symptom	Engpass	Stellschraube
Lag wächst, Broker-Metriken ruhig (Idle über 0,3, Platte und Netz unauffällig)	Consumer zu langsam	mehr Mitglieder (Pods oder `concurrency`) oder schnellere Verarbeitung je Nachricht
Lag wächst, aber jedes Mitglied hat schon eine Partition	Partitionszahl ist die Obergrenze	mehr Partitionen, geplant
`RequestHandlerAvgIdlePercent` unter 0,3, `RequestQueueTimeMs` hoch, Platte oder Netz am Limit	Broker	mehr Broker und Partitionen dorthin verschieben
`record-queue-time-avg` hoch, `bufferpool-wait-ratio` über 0	Producer sendet ineffizient	Batching: `linger.ms`, `batch.size`, Kompression

Mehr Consumer: sofort, bis zur Partitionszahl. Ein weiterer Pod oder `concurrency` plus 1 ist nur ein Rebalance; mit `group.protocol=consumer` wandern nur die betroffenen Partitionen. Hilft es nicht, obwohl Partitionen frei waren, ist die Verarbeitung je Nachricht das Problem.

Mehr Partitionen: online möglich, bricht die Key-Zuordnung. `kafka-topics.sh --alter --partitions` dauert Sekunden, aber ab dann liegt V-1001 in einer anderen Partition als seine alten Änderungen. Mit Keys helfen neue Partitionen nur dem künftigen Verkehr: Alte Nachrichten bleiben liegen, ein bestehender Lag wird nicht verteilt, und an der Nahtstelle können zwei Mitglieder denselben Vertrag gleichzeitig lesen. Deshalb Producer pausieren, Consumer leer laufen lassen, erhöhen, Producer wieder starten. Für Streams-Joins müssen beide Eingangs-Topics gleich viele Partitionen haben.

Mehr Broker: nur mit Umzug, nur bei Broker-Engpass. Ein neuer Broker bekommt von selbst nichts; `kafka-reassign-partitions.sh` oder Cruise Control verschiebt Partitionen und kopiert dabei Daten über das Netz. Ein langsamer Consumer wird durch Broker nicht schneller.

Replikationsfaktor: skaliert die Sicherheit, nicht die Leser. Gelesen wird nur vom Leader einer Partition; Follower bedienen keine Consumer. Partitionen skalieren die Consumer, Broker skalieren die Last, der Replikationsfaktor skaliert die Sicherheit, und RF kann nie größer sein als die Brokerzahl.

Was oft mehr bringt als alle drei: Producer-Batching (`linger.ms` 20 bis 100, `batch.size` 64 bis 128 KB, lz4; im Lab von 24.000 auf 67.000 Nachrichten pro Sekunde mit `15-performance`), Batch-Verarbeitung im Listener (`ack-mode=batch`, ein Datenbankaufruf je Paket statt je Nachricht), `max.poll.records` und `fetch.min.bytes` passend zur Verarbeitung.

20.8 Weiterführende Links

- Kafka-Dokumentation, Konfiguration von Broker, Producer und Consumer: https://kafka.apache.org/documentation/#configuration
- Spring for Apache Kafka, Referenz: https://docs.spring.io/spring-kafka/reference/
- Apache Camel, Kafka-Komponente: https://camel.apache.org/components/next/kafka-component.html
- Confluent, Best Practices für Produktion: https://docs.confluent.io/platform/current/kafka/deployment.html
- Strimzi, Betrieb auf Kubernetes und OpenShift: https://strimzi.io/documentation/

Anhang: Befehlsreferenz

Alle Kafka-Befehle laufen im Lab im Container `kafka1` (`docker exec -it kafka1 bash`, dann `cd /opt/kafka/bin`) und beginnen mit `./`. Jeder Befehl braucht `--bootstrap-server kafka1:9092`; in den Tabellen ist das mit … abgekürzt.

Docker und Lab

Befehl (PowerShell, Ordner `kafka-lab`)	Wirkung
`docker compose up -d`	alle Container starten
`docker compose --profile dr up -d`	zusätzlich DR-Cluster und MirrorMaker 2
`docker compose ps`	Status
`docker exec -it kafka1 bash`	Shell im Broker
`docker stop kafka3` / `docker start kafka3`	Broker anhalten / starten
`docker compose logs --tail 30 kafka1`	Log-Zeilen
`docker compose --profile dr down`	alles stoppen und entfernen (Daten weg)

Topics

Befehl	Wirkung
`./kafka-topics.sh … --list`	alle Topics
`./kafka-topics.sh … --create --topic bestand.vertragsaenderungen --partitions 6 --replication-factor 3`	Topic anlegen
`./kafka-topics.sh … --describe --topic bestand.vertragsaenderungen`	Leader, Replicas, Isr, Elr je Partition
`./kafka-topics.sh … --describe --under-replicated-partitions`	fehlende Kopien
`./kafka-topics.sh … --describe --under-min-isr-partitions`	Schreibstopp für `acks=all`
`./kafka-topics.sh … --describe --unavailable-partitions`	Partitionen ohne Leader
`./kafka-topics.sh … --alter --topic t --partitions 12`	Partitionen erhöhen (Vorsicht bei Keys)
`./kafka-topics.sh … --delete --topic t`	Topic löschen
`./kafka-get-offsets.sh … --topic t`	Log-Ende je Partition
`./kafka-log-dirs.sh … --describe --topic-list t`	Größe je Partition und Broker
`./kafka-dump-log.sh --files <segment>.log --print-data-log`	Segment-Datei lesen

Konfiguration

Befehl	Wirkung
`./kafka-configs.sh … --alter --entity-type topics --entity-name t --add-config min.insync.replicas=2`	Topic-Einstellung setzen
`./kafka-configs.sh … --alter --entity-type topics --entity-name t --delete-config retention.ms`	auf Standard zurück
`./kafka-configs.sh … --describe --entity-type topics --entity-name t --all`	alle Werte mit Herkunft
`./kafka-configs.sh … --alter --entity-type clients --entity-name c --add-config producer_byte_rate=1048576`	Quota
`./kafka-configs.sh … --alter --entity-type groups --entity-name g --add-config share.auto.offset.reset=earliest`	Share-Group-Startposition

Schreiben und lesen

Befehl oder Option	Wirkung
`./kafka-console-producer.sh … --topic t`	jede Zeile eine Nachricht
`--reader-property parse.key=true --reader-property key.separator=:`	Eingabe `key:value`
`--command-property acks=all`	Client-Einstellung
`./kafka-console-consumer.sh … --topic t`	nur neue Nachrichten
`--from-beginning`	ab Anfang (nur ohne gespeicherten Offset)
`--group g`	als Mitglied einer Consumer Group, mit Commits
`--formatter-property print.key=true --formatter-property print.partition=true --formatter-property print.offset=true`	Key, Partition, Offset ausgeben
`--partition 0 --offset earliest`	eine bestimmte Partition ohne Gruppe
`./kafka-console-share-consumer.sh … --topic t --group g`	als Mitglied einer Share Group

Consumer Groups und Offsets

Befehl	Wirkung
`./kafka-groups.sh … --list`	alle Gruppen mit Typ und Protokoll
`./kafka-consumer-groups.sh … --describe --group g`	Offsets und Lag je Partition
`./kafka-consumer-groups.sh … --describe --group g --members`	Mitglieder und Zuordnung
`./kafka-consumer-groups.sh … --describe --group g --state`	Gruppenstatus
`./kafka-consumer-groups.sh … --group g --topic t --reset-offsets --to-datetime 2026-11-02T10:00:00.000 --dry-run`	Reset simulieren
… `--execute` statt `--dry-run`	Reset ausführen (Gruppe gestoppt)
… `--dry-run --export > stand.csv`, später `--from-file stand.csv --execute`	Stand sichern und zurückholen
`./kafka-consumer-groups.sh … --delete --group g`	Gruppe samt Offsets löschen
`./kafka-share-groups.sh … --describe --group g`	Share Group ansehen

Cluster und KRaft

Befehl	Wirkung
`./kafka-metadata-quorum.sh … describe --status`	aktiver Controller, Voter, Observer
`./kafka-metadata-quorum.sh … describe --replication`	Stand jedes Knotens im Metadaten-Log
`./kafka-metadata-quorum.sh --bootstrap-controller kafka1:9093 describe --status`	direkt bei den Controllern fragen
`./kafka-leader-election.sh … --election-type preferred --all-topic-partitions`	Leader auf Preferred Leader zurück
`./kafka-reassign-partitions.sh … --reassignment-json-file plan.json --execute --throttle 50000000`	Partitionen umziehen
`./kafka-features.sh … describe`	Feature-Versionen des Clusters
`./kafka-acls.sh … --add --allow-principal User:u --operation Read --topic t`	Recht vergeben
`./kafka-jmx.sh --jmx-url service:jmx:rmi:///jndi/rmi://kafka1:9999/jmxrmi --object-name <mbean> --one-time true`	eine Metrik direkt lesen

Messen

Befehl	Wirkung
`./kafka-producer-perf-test.sh … --topic perf --num-records 200000 --record-size 1000 --throughput -1 --command-property acks=all`	Schreibdurchsatz
`--record-key-range 1000 --key-distribution random`	mit Keys (neu in 4.4)
`--warmup-records 20000`	Aufwärmphase ausklammern
`./kafka-consumer-perf-test.sh … --topic perf --num-records 200000`	Lesedurchsatz
`./kafka-e2e-latency.sh kafka1:9092 perf 1000 all 1000`	Ende-zu-Ende-Latenz (Server, Topic, Anzahl, acks, Größe)
`bash /opt/lab/bin/mit-metriken <port> <werkzeug> …`	Werkzeug mit JMX-Exporter starten (Lab)

Spring-Boot-Einstellungen

Schlüssel	Empfehlung
`spring.kafka.bootstrap-servers`	mehrere Broker
`spring.kafka.producer.acks`	`all`
`spring.kafka.producer.properties.enable.idempotence`	`true`
`spring.kafka.producer.compression-type`	`lz4` oder `zstd`
`spring.kafka.producer.transaction-id-prefix`	nur für Kafka-Transaktionen
`spring.kafka.consumer.group-id`	Name der Anwendung
`spring.kafka.consumer.auto-offset-reset`	`earliest`
`spring.kafka.consumer.enable-auto-commit`	`false`
`spring.kafka.consumer.max-poll-records`	passend zur Verarbeitungszeit
`spring.kafka.consumer.properties.group.protocol`	`consumer`
`spring.kafka.listener.ack-mode`	`batch` (Standard) oder `record`
`spring.kafka.listener.concurrency`	≤ Partitionen
`spring.kafka.streams.application-id`	Name der Streams-Anwendung
`spring.kafka.streams.replication-factor`	3
`spring.kafka.streams.properties.processing.guarantee`	`exactly_once_v2`

Anhang: Glossar

Begriff	Bedeutung
`acks`	worauf der Producer vor der Bestätigung wartet: `0`, `1` oder `all`
At-least-once	jede Nachricht wird mindestens einmal verarbeitet; Duplikate möglich
At-most-once	höchstens einmal; Verlust möglich
Bootstrap-Server	Einstiegsadresse eines Clients; danach spricht er direkt mit den Leadern
Broker	Server, der Partitionen speichert und Clients bedient
CDC (Change Data Capture)	Änderungen aus dem Transaktionslog einer Datenbank als Ereignisse lesen, z. B. mit Debezium
Claim-Check-Muster	große Inhalte im Objektspeicher ablegen, nur die Referenz über Kafka schicken
Co-Partitioning	gleiche Partitionsanzahl und gleicher Key in Topics, die zusammengeführt werden
Commit	Speichern des Fortschritts einer Consumer Group in `__consumer_offsets`
Compaction	Kafka behält je Key nur die letzte Nachricht
Consumer Group	Consumer mit derselben `group.id`; teilen sich die Partitionen
Consumer Lag	Abstand zwischen Log-Ende und committetem Offset
Controller	verwaltet Metadaten, wählt Leader; mehrere bilden das KRaft-Quorum
Crypto-Shredding	Daten unlesbar machen, indem man ihren Schlüssel löscht
Dead-Letter-Topic (DLT)	Topic für Nachrichten, die nicht verarbeitet werden konnten
ELR (Eligible Leader Replicas)	Repliken außerhalb der ISR, die sicher alle bestätigten Daten haben
Exactly-once	genau einmal – in Kafka nur innerhalb von Kafka, mit Transaktionen
Group Coordinator	Broker, der eine Consumer Group koordiniert
High Watermark	höchster Offset, den alle ISR-Mitglieder haben; Grenze für Consumer
Idempotenter Producer	verhindert Duplikate durch Wiederholungen; Standard seit Kafka 3.0
Idempotente Verarbeitung	doppelte Zustellung ändert das Ergebnis nicht (Version, Upsert, Dedupe)
ISR (In-Sync Replicas)	Repliken, die mit dem Leader aktuell sind
Key	bestimmt die Partition einer Nachricht
KRaft	Kafkas eigenes Quorum-Verfahren für Metadaten; ersetzt ZooKeeper

Begriff	Bedeutung
`KStream` / `KTable`	Kafka Streams: Strom einzelner Ereignisse / aktueller Stand je Key
Leader / Follower	Leader bedient Clients, Follower kopieren
`min.insync.replicas`	Mindestzahl aktueller Kopien für Schreibvorgänge mit `acks=all`
MirrorMaker 2	spiegelt Topics und Offsets zwischen Clustern
Offset	Position einer Nachricht innerhalb einer Partition
Outbox	Ereignis in derselben Datenbank-Transaktion wie die fachliche Änderung speichern
Partition	Teil eines Topics; geordnetes, nur anwachsendes Log
Poison Pill	Nachricht, die bei jedem Versuch scheitert
Preferred Leader	erste Replik in der Liste; bevorzugter Leader
Rebalance	Neuverteilung der Partitionen einer Consumer Group
Replication Factor	Anzahl der Kopien einer Partition
Retention	wie lange bzw. wie viel Kafka aufbewahrt
Schema Registry	speichert Schemas und prüft ihre Kompatibilität
Segment	Datei, in der ein Teil einer Partition liegt
Share Group	Gruppe mit Queue-Semantik: Einzelbestätigung, keine Reihenfolge
Sticky Partitioner	verteilt Nachrichten ohne Key batchweise über Partitionen
Tiered Storage	alte Segmente in einem Objektspeicher
Tombstone	Nachricht mit Key und Wert `null`; löscht den Key in kompaktierten Topics
Topic	benannter Datenstrom einer Art
Unclean Leader Election	Leader-Wahl auch außerhalb der ISR; Datenverlust möglich